

Limbaje formale si translatoare - introducere

February 23, 2021

Outline

Administrativ

- ▶ Examen final - 60%: ≥ 4 pentru a se aduna laboratorul
- ▶ Laborator - 40%: ≥ 5 pt intrare in examen
 - ▶ Colocviu Lex si Yacc
 - ▶ Proiect (individual sau echipe de 2 studenti)
 - ▶ teme
- ▶ puncte suplimentare: Kahoot
- ▶ anca.marginean@cs.utcluj.ro
- ▶ Registration key Moodle - *Chomsky_X*
 - ▶ Camelia Pinte
 - ▶ Vanessa Mercea

Bibliografie

- ▶ Curs introductiv - Capitolul 1. Michael Scott "Programming language pragmatics" third edition
- ▶ **I.A.Letia, E.S.Chifu "Limbaje formale si translatoare"**
- ▶ J.E. Hopcroft, R. Motwani, J.D. Ullman "Introduction to Automata Theory, Languages, and Computation"
- ▶ A.V. Aho, M.S. Lam, R. Sethi, J.D. Ullman : "Compilers: Principles, Techniques, and Tools" (dragon book)
- ▶ M. Sipster: "Introduction to the theory of computation" (third edition)

Outline

Motivatie

Evolutia limbajelor de programare

Compilare - Interpretare. Fazele unui compilator

Limbaje ezoterice

```
primesTo m = sieve [2..m]
    where
        sieve (x:xs) = x : sieve (xs \\ [x,x+x..m])
        sieve [] = []
```

```
import java.util.LinkedList;
import java.util.BitSet;

public class Sieve{
    public static LinkedList<Integer> sieve(int n){
        LinkedList<Integer> primes = new LinkedList<Integer>
            ();
        BitSet nonPrimes = new BitSet(n+1);

        for (int p = 2; p <= n ; p = nonPrimes.nextClearBit(
            p+1)) {
            for (int i = p * p; i <= n; i += p)
                nonPrimes.set(i);
            primes.add(p);
        }
        return primes;
    }
}
```

Limbaje formale si translatoare

- ▶ Un limbaj e un set legal de propozitii

Limbaje formale si translatoare

- ▶ Un limbaj e un set legal de propozitii
- ▶ O propozitie e o secventa de simboluri

Limbaje formale si translatoare

- ▶ Un **limbaj** e un set legal de propozitii
- ▶ O **propozitie** e o secventa de simboluri
- ▶ Un **simbol** poate fi un caracter, un cuvant, un semn de punctuatie, ...

Limbaje formale si translatoare

- ▶ Un **limbaj** e un set legal de propozitii
- ▶ O **propozitie** e o secventa de simboluri
- ▶ Un **simbol** poate fi un caracter, un cuvant, un semn de punctuatie, ...
- ▶ Un **limbaj formal** este un limbaj definit de un **set finit de reguli** neambigue care delimiteaza propozitiile legale de cele ilegale

Limbaje formale si translatoare

- ▶ Un **limbaj** e un set legal de propozitii
- ▶ O **propozitie** e o secventa de simboluri
- ▶ Un **simbol** poate fi un caracter, un cuvant, un semn de punctuatie, ...
- ▶ Un **limbaj formal** este un limbaj definit de un **set finit de reguli** neambigue care delimiteaza propozitiile legale de cele ilegale

Obiective curs

- ▶ Cum se poate descrie un limbaj formal?
- ▶ Cum se poate recunoaste si prelucra un limbaj?
- ▶ Automate finite deterministe si nedeterministe, automate stiva pentru parsare
- ▶ + Elemente introductive de procesare a limbajului natural

Donalt Knuth(1938-): programming - "the art of telling another human being what one wants the computer to do"

Limbaje formale si translatoare - definire limbaj de programare si implementare

- ▶ specificam limbajul de programare folosind modele formale - gramatici si automate
- ▶ transformam aceste modele formale intr-o implementare

Implementare limbaje de programare: 3 strategii

- ▶ interpretare - *source* $\xrightarrow{\text{interpret}}$ *actions/results*
- ▶ compilare - *source* $\xrightarrow{\text{translates}}$ *machine / assembly language* $\xrightarrow{\text{execute}}$ *actions/results*
- ▶ hibrid
 - ▶ Just-In-Time(JIT) compilers - interpreteaza parti din program, compileaza alte parti in timpul executiei
 - ▶ compilatoare care translateaza programul in *alte limbaje de programare*(precum C) sau limbaj intermediar (Java *bytecode*) pentru care exista un translator sau compilator

Observatie: strategia este specifica implementarii unui limbaj, si nu unui limbaj (exista interpretoare C si compilatoare Lisp)

Outline

Motivatie

Evolutia limbajelor de programare

Compilare - Interpretare. Fazele unui compilator

Limbaje ezoterice

Evolutia limbajelor de programare

Machine language - instructiuni in binar sau hexazecimal care controleaza direct unitatea centrala de procesare (CPU)

```
55 89 e5 53 83 ec 04 83 e4 f0 e8 31 00 00 00 89 c3 e8 2a 00
00 00 39 c3 74 10 8d b6 00 00 00 00 39 c3 7e 13 29 c3 39 c3
75 f6 89 1c 24 e8 6e 00 00 00 8b 5d fc c9 c3 29 d8 eb eb 90
```

Listing 1: gcd in hexazecimal

Limbaj de asamblare

- ▶ one-to-one correspondence: mnemonics - machine language instructions
- ▶ programare dependenta de setul de instructiuni al masinii

```
pushl    %ebp
movl     %esp, %ebp
pushl    %ebx
subl     $4, %esp
andl     $-16, %esp
call     getint
movl     %eax, %ebx
call     getint
cmpl     %eax, %ebx
je       C
A: cmpl   %eax, %ebx

jle      D
subl     %eax, %ebx
B:  cmpl  %eax, %ebx
jne      A
C:  movl  %ebx, (%esp)
call     putint
movl     -4(%ebp), %ebx
leave
ret
D:  subl  %ebx, %eax
jmp      B
```

Limbaj masina - limbaj asamblare

```
55 89 e5 53 83 ec 04 83 e4 f0 e8 31 00 00 00 89 c3 e8 2a 00  
00 00 c9 c3 74 10 8d b6 00 00 00 00 39 c3 7e 13 29 c3 39 c3  
75 f6 89 1c 24 e8 6e 00 00 00 8b 5d fc c9 c3 29 d8 eb eb 90
```

Correspondenta one-to-one *cmpl %eax, %ebx* e reprezentat de
secventa 39 c3

1950 Fortran - first high-level programming language

*FOR*mula *TRAN*slator; Rapid urmat de Lisp si Algol
expresii aritmetice, If, Do, Goto

```
10  if (a .EQ. b) goto 20
    if (a .LT. b) then
        a = b - a
    else
        b = a - b
    endif
    goto 10
20 end
```

Evolutie

- ▶ COBOL(1959) - Common Business-Oriented Language; type declarations, record types, file manipulation
- ▶ LISP - McCarthy, MIT, 1958; functional: recursive
- ▶ APL (array manipulation) - IBM, 1960; imperative, matrix-centric; symbols α
- ▶ Algol, Pascal(1970), Clu, Modula, Ada - Imperative, block-structured language, formal syntax definition, structured programming
- ▶ SNOBOL, Icon - string processing languages
- ▶ 1964 - BASIC - Beginner's All-purpose Symbolic Instruction Code; programming for the masses; la Dartmouth College
 - ▶ 1975 - Altair BASIC (Bill Gates, Paul Allen) - Micro-Soft
- ▶ C(1969)- procedural, imperative
- ▶ Simula, Smalltalk, C++, Java (1991), C# - object oriented
- ▶ ML, Miranda, Haskell(1990) - functional languages with types
- ▶ sh, awk(1977), perl, tcl, python(1989), php - scripting langs
- ▶ SQL(1974 IBM) - database queries
- ▶ Prolog(1972) - logic programming language

Clasificarea limbajelor

declarative

functional	Lisp/Scheme, ML, Haskell
dataflow	Id, Val
logic, constraint-based	Prolog, spreadsheets
template-based	XSLT

imperative

von Neumann	C, Ada, Fortran, ...
scripting	Perl, Python, PHP, ...
object-oriented	Smalltalk, Eiffel, Java, ...

Declarativ - *ce* trebuie sa faca

Imperativ - *cum* trebuie sa faca

Preluat din Michael Scott "Programming language pragmatics" third edition

Popularitatea Limbajelor

- ▶ TIOBE index - “The Importance of Being Earnest”
<https://www.tiobe.com/tiobe-index/>
- ▶ PYPL Index : The PYPL PopularitY of Programming Language Index <http://pypl.github.io/PYPL.html>

Outline

Motivatie

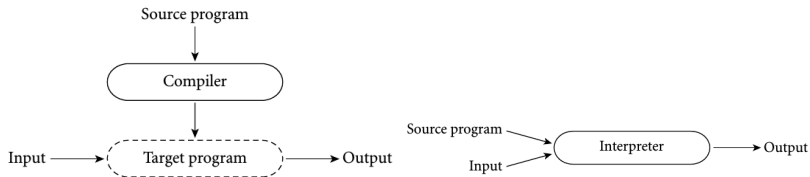
Evolutia limbajelor de programare

Compilare - Interpretare. Fazele unui compilator

Limbaje ezoterice

Compiler vs Interpretor

- ▶ Compiler: **translatarea** dintr-un **limbaj de nivel inalt** in limbaj asamblare sau masina
- ▶ Object code - rezultatul compilarii

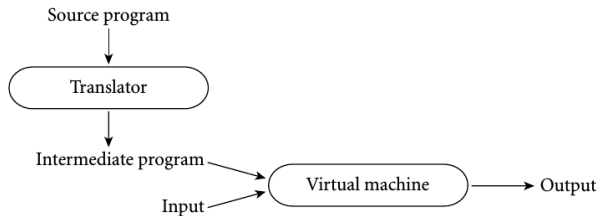


Compilerul nu face parte din executie; interpretorul citeste instructiuni mai mult sau mai putin una cate una si le executa

Comparatie

- ▶ Interpretarea - mai mare flexibilitate si mesaje de eroare mai bune
- ▶ Compilarea - performanta mai buna (decizii la momentul compilarii)
exemplu: GHC optimizare tail-recursive calls - apelul recursiv e ultimul statements din functie

Combinare compilare - interpretare



- ▶ Java bytecode - rezultatul compilarii codului sursa; executat in Java virtual machine (JVM)

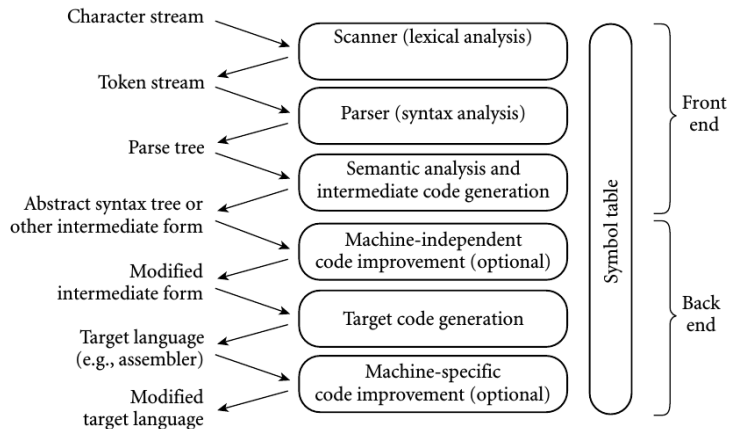
Preprocesare - eliminarea comentariilor, spatiile

Observatii

Asamblor - traducere din asamblare in cod masina (initial folosind o mapare 1 la 1)

Compiler - traducere din limbaj de nivel inalt in asamblare sau cod masina; substantial mai complicat decat asamblorul (nu exista mapare 1 la 1 intre sursa si target)

Fazele unui compilator



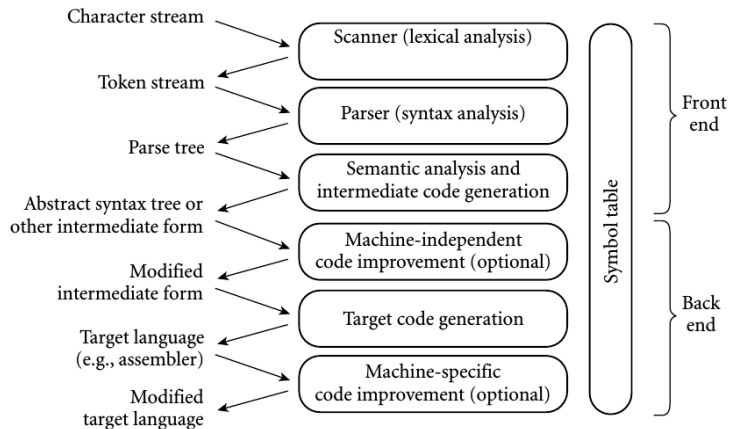
```
int main() {  
    int i = getint(), j = getint();  
    while (i != j) {  
        if (i > j) i = i - j;  
        else j = j - i;  
    }  
    putint(i);  
}
```

Scanare

- ▶ Scanarea (analiza lexicala) - citeste caracterele si le grupeaza in token-i (cea mai mica unitate cu sens a unui program)
- ▶ elimina de obicei si comentariile si spatiile albe
- ▶ reduce dimensiunea inputului pentru parser (numarul de caractere e mult mai mare decat cel de tokeni)

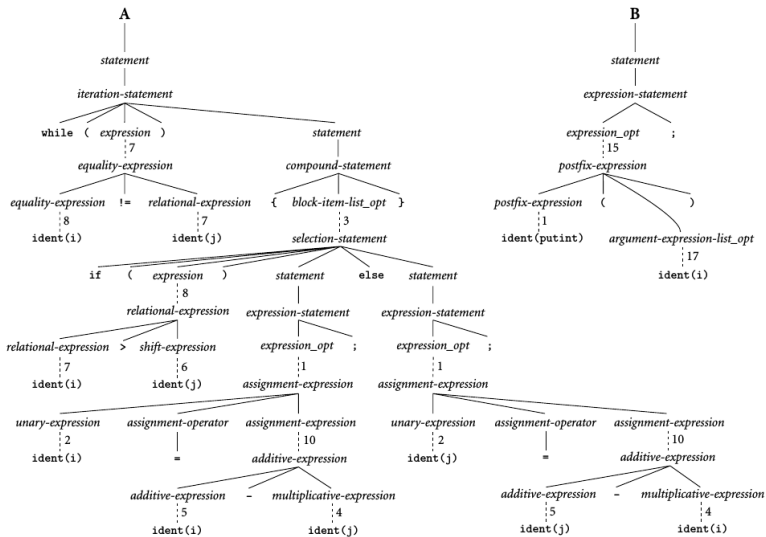
```
int    main    (    )    {    int    i    =  
getint (    )    ,    j    =    getint (      
    )    ;    while (    i    !=    j    )  
{    if    (    i    >    j    )    i  
=    i    -    j    ;    else    j    =  
j    -    i    ;    }    putint (    i  
)    ;    }
```

Fazele unui compilator - parsare



Parsare

- ▶ Parsarea - organizeaza token-ii in arbori de parsare (parse tree)
- ▶ cum formeaza tokenii un program
- ▶ analizor sintactic - derivator sau parser
- ▶ Arbore de parsare(derivare): radacina e programul, frunzele sunt token-ii de la analizorul lexical



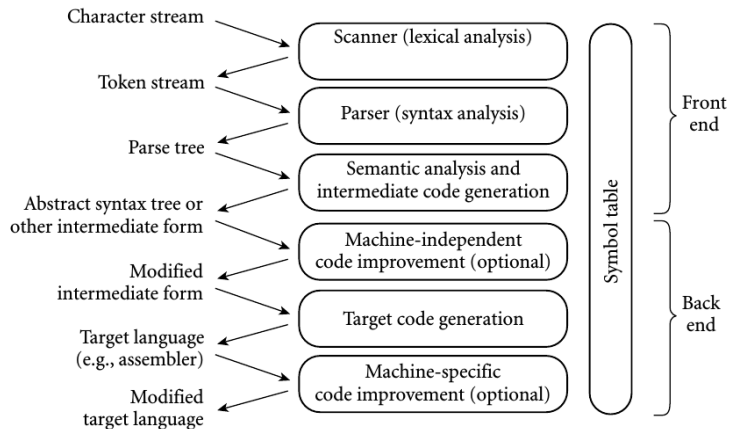
Observatii: Analizor lexical + sintactic

- ▶ Scanarea (analiza lexicala) si parsarea (analiza sintactica) - verifica daca toti token-ii sunt corecti (well formed) si secventa de token-i corespunde sintaxei limbajului

Intrebare - **cum definim limbajul?**

- ▶ Set de productii (gramatici), diagrame sintactice, BNF (Backus Naur Form)

Fazele unui compilator - analiza semantica



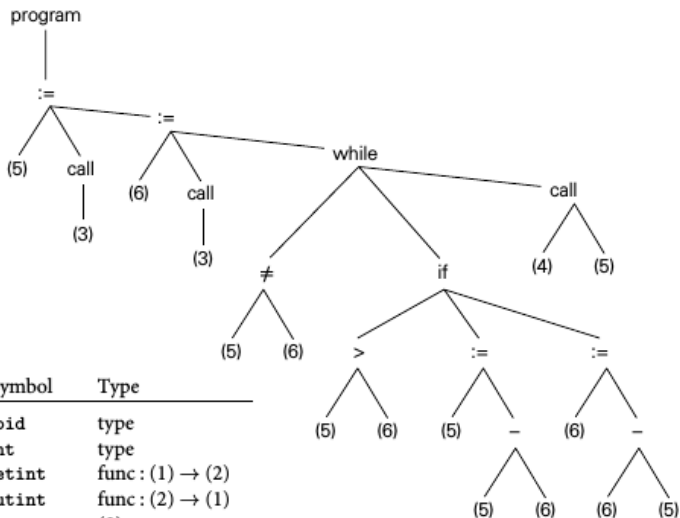
Analiza semantica si Generarea de cod intermediar

Descoperirea sensului programului

- ▶ folosirea aceluiasi identificator - aceeasi entitate; **tabela de simboluri (symbol table)**
- ▶ tipurile identificatorilor si ale expresiilor
- ▶ verificarea unor reguli semantice: ex - nu se aduna un string cu un intreg, procedurile sunt chemate cu nr corect de argumente (static rules)
- ▶ Observatie: exista si reguli semantice care nu pot fi verificate la momentul compilarii, ci doar la momentul rularii; ex: nu se acceseaza un element dintr-un array din afara limitelor acestuia (dynamic rules)

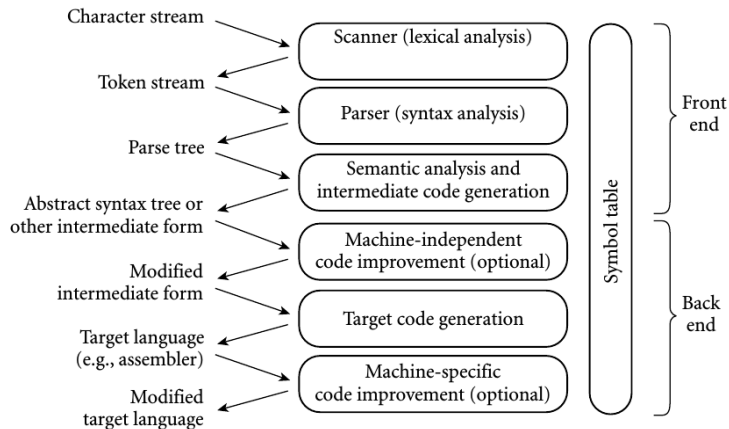
In multe compilatoare: actiuni la momentul identificarii unui pas particular intr-o regula gramaticala (semantic action routines)

Arbore sintactic (abstract syntax tree, sau syntax tree) - mai concis decat **arborele de derivare** (parse tree) (uneori numit **concrete syntax tree**)



Index	Symbol	Type
1	void	type
2	int	type
3	getint	func: (1) → (2)
4	putint	func: (2) → (1)
5	i	(2)
6	j	(2)

Fazele unui compilator - Optimizare si generare cod



Generarea de cod target. Optimizare

Pornind de la arborele sintactic si tabela de simboluri - cod limbaj de asamblare sau masina

Optimizare cod - independent sau dependent de masina

Front-end. Back-end

Front-end - verifica daca un program este corect scris in termenii sintaxei si semanticii limbajului de programare

Back end - responsabil cu translatarea sursei in code target (asamblare/masina)

Alt exemplu

$$position = initial + rate * 60$$

1. Analiza lexicala

Alt exemplu

$$position = initial + rate * 60$$

1. Analiza lexicala

Lexeme	Tokeni
position	ID
=	=
initial	ID
+	+
rate	ID
*	*
60	NUM

2. Parsare/Analiza sintactica

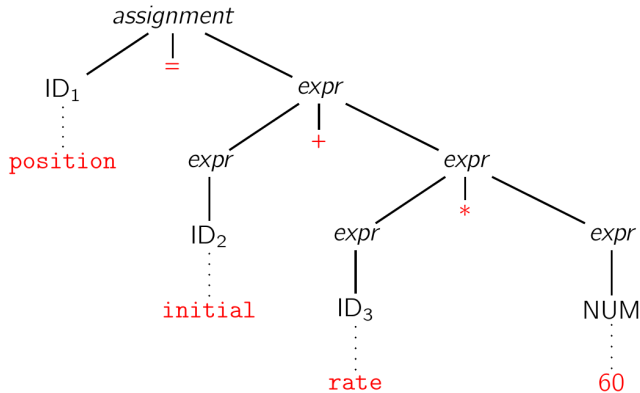
```
<assignment> -> ID "=" <expr>  
<expr>        -> ID | NUM | <expr><op><expr> | ( < expr > )  
<op>          -> + | - | * | /
```

2. Parsare/Analiza sintactica

<assignment> -> ID "=" <expr>

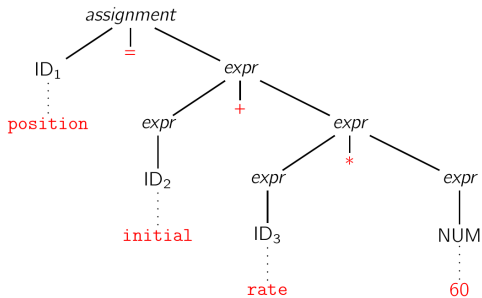
<expr> -> ID | NUM | <expr><op><expr> | (< expr >)

<op> -> + | - | * | /



3. Analiza semantica: verificare de tipuri si alte informatii

Generare cod intermediar din arborele de parsare



```
temp1 = inttoreal(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

4. Exemplu de optimizare

```
temp1 = inttoreal(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

```
temp1 = id3*60.0
id1 = id2+ temp1
```

5. Generare de cod

MOVF id3, R2 The F stands **for** floating-point instruction
MULF #60.0, R2 The # means that 60.0 is **a** constant
MOVF id2, R1 The first and second operand of **each**
 instruction
ADDF R2, R1 specify **a** source and **a** destination
MOVF R1, id1

position := initial + rate * 60

lexical analyzer

$id_1 := id_2 + id_3 * 60$

syntax analyzer

id_1
 $:=$
 id_2 $+$ id_3 $*$ 60

semantic analyzer

id_1
 $:=$
 id_2 $+$ id_1 $*$ $inttoreal$
 60

intermediate code generator

temp1 := inttoreal(60)
 temp2 := id3 * temp1
 temp3 := id2 + temp2
 id1 := temp3

code optimizer

temp1 := id3 * 60.0
 id1 := id2 + temp1

code generator

MOVF id3, R2
 MULF #60.0, R2
 MOVF id2, R1
 ADDF R2, R1
 MOVF R1, id1

SYMBOL TABLE

1	position	...
2	initial	...
3	rate	...
4		

Outline

Motivatie

Evolutia limbajelor de programare

Compilare - Interpretare. Fazele unui compilator

Limbaje ezoterice

Esolang - esoteric programming language

Brainfuck - 8 caractere

```
+++++++ [ >+++++>+++++++>+++<<<- ] >+ . >+ . ++++++  
..+++ .>+ . <<+++++++ .> .+++ .----- .----- .>+ .
```

Listing 2: "Hello world"

LOLCODE

```
HAI 1.2
CAN HAS STDIO?
PLZ OPEN FILE "LOLCATS.TXT"?
    AWSUM THX
        VISIBLE FILE
    O NOES
        INVISIBLE "ERROR!"
KTHXBYE
```

LOLCODE

```
HAI 1.2
BTW Greetz a friend
I HAS A animal
GIMMEH animal
BOTH SAEM animal AN "cat"
O RLY?
    YA RLY
        VISIBLE "Hello cat"
        VISIBLE "Nice to meet you"
    MEBBE BOTH SAEM animal AN "mouse"
        VISIBLE "Hello mouse"
        VISIBLE "Nice to eat you"
    NO WAI
        VISIBLE "Hello stranger"
OIC
KTHXBYE
```

Exemple de limbaje

- ▶ Geometric figure drawing
- ▶ Music manipulation
- ▶ Table manipulation
- ▶ Finance language
- ▶ A graph language
- ▶ text-based adventure game

<http://www.cs.columbia.edu/~sedwards/classes.html>

`http://www.99-bottles-of-beer.net/toplist.html`

- ▶ See the toplist
- ▶ brainfuck code, dna#,
- ▶ lisp, haskell, python, c++

Semantic - observatii

- ▶ Ceva poate fi sintactic corect, dar fara sens

The rock jumped thorough the hairy planet

- ▶ Sau ambiguu

The chickens are ready to eat.

- ▶ Instrumente pentru reprezentare
 - ▶ Siruri de rescriere
 - ▶ Gramatici - ierarhia lui Chomsky
 - ▶ Derivari si arbori de derivare
- ▶ Gramatici regulate si automate finite
 - ▶ Automate finite
 - ▶ Diagrame de stare si expresii regulate
- ▶ Gramatici independente de context si automate stiva:
Automate stiva
- ▶ Analiza sintactica descendenta:
 - ▶ $LL(k)$
 - ▶ eliminare recursivitate stanga
 - ▶ Factorizare stanga
 - ▶ gramatici $LL(k)$ tari
 - ▶ **Derivator $LL(1)$** - segmente de program
- ▶ Analiza sintactica ascendenta
 - ▶ $LR(k)$
 - ▶ **Derivator $LR(0)$** - functia de tranzitie
 - ▶ $SLR(1)$
- ▶ Elemente de Procesare a limbajului natural - NLP

Rezumat

Motivatie

Evolutia limbajelor de programare

Compilare - Interpretare. Fazele unui compilator

Limbaje ezoterice

Exemplu de proiect ani anteriori

Limbaj propriu pentru desenare (autor Timotei Molcut)

Elemente de limbaje formale. Instrumente pentru reprezentare

2 Martie 2021

- ▶ Gramaticile formale, in particular gramaticile independente de context, sunt uneltele cele mai utilizate pentru a reprezenta clar structura programelor, sub forma arborilor de derivare.
- ▶ Pornind de la gramatici, se pot specifica automate care accepta programe concrete.
- ▶ Automatele pot fi modificate pentru a genera o codificare acceptabila din arborii de derivare.

Outline

Siruri si sisteme de rescriere

Gramatici

Ierarhia lui Chomsky

Arbori de derivare

Siruri si sisteme de rescriere

- ▶ Un limbaj = set de stringuri
- ▶ Definirea formala a limbajului = raspuns formal pt “Care stringuri sunt admise de catre Limbaj?”

Siruri si sisteme de rescriere cont

Alfabet (vocabulary) V - un set de simboluri

ex: $V=1,2,3,4,5,6,7,8,9,0$

String un string peste alfabetul V = secventa(sir finit) de simboluri din alfabetul V

ex: 2018

Stringul vid: ϵ

$$\epsilon\chi = \chi\epsilon = \chi$$

V^* - multimea tuturor stringurilor peste V

$$V^+ = V^* \setminus \{\epsilon\}$$

Limbaj L peste V este orice subset al lui V^*

Propozitii - elementele limbajului

Numarul de propozitii dintr-un limbaj poate fi infinit

Exemple:

$$V = \{a, b, c, \dots a\}; \quad L = \{\text{cuvintele limbii engleze}\}$$

$$V = \{0, 1\}; \quad L = \{\varepsilon, 01, 010, 0101, 01010, 010101, \dots\}$$

Cum putem defini propozitiile unui limbaj? Ne trebuie o reprezentare formală

proces de generare

Derivare

Relatie de derivare $\Rightarrow^+$ binara, tranzitiva pe V^*

$$L = \{\chi | \zeta \Rightarrow^+ \chi, \zeta \text{ un anumit sir din } V^*\}$$

Sistem formal $(V, \Rightarrow^+)$

definire derivare prin enumerare?? – NU

Productii Un sir finit de perechi (σ, τ) de siruri din V^*

Definesc relatia de derivare

Generam un string pornind de la alt string

Inchiderea tranzitiva a relatiei finite descrise de catre productii =
relatia de derivare

Derivare cont.

Sistem de rescriere (V, P) , V vocabular, P set finit de productii

$$\sigma \rightarrow \tau, \sigma, \tau \in V^*$$

Derivare directa $\Rightarrow$ Un sir χ este derivabil direct din π : $\pi \Rightarrow \chi$ daca exista sirurile $\sigma, \tau, \mu, \nu \in V^*$ a.i.

$$\sigma \rightarrow \tau \in P,$$

$$\pi = \mu\sigma\nu,$$

$$\chi = \mu\tau\nu.$$

Derivare $\Rightarrow^+$ Un sir χ este derivabil din sirul π : $\pi \Rightarrow^+ \chi$ daca exista sirurile $\rho_0, \dots, \rho_n \in V^*$, $n \geq 1$ a.i.

$$\pi = \rho_0, \chi = \rho_n,$$

$$\rho_{i-1} \Rightarrow \rho_i, i = \overline{1, n}.$$

Secventa $\rho_0, \dots, \rho_n$ = derivare de lungime n .

χ reductibil direct la π daca χ derivabil direct din π

Expresii aritmetice: Fie sistemul de rescriere (V, P)

unde

- ▶ $V = \{+, *, (,), i, E, T, F\}$
- ▶ cu productiile P
 1. $E \rightarrow T$
 2. $E \rightarrow E + T$
 3. $T \rightarrow F$
 4. $T \rightarrow T * F$
 5. $F \rightarrow i$
 6. $F \rightarrow (E)$

Derivari cu lungimea lor:

$$E \Rightarrow T \quad 1$$

$$T \Rightarrow T * F \quad 1$$

$$T * F \Rightarrow T * i \quad 1$$

$$E \Rightarrow^* T * i \quad 3$$

$$TiE \Rightarrow^* iii \quad 5$$

$$E \Rightarrow i + i * i \quad 8$$

Outline

Siruri si sisteme de rescriere

Gramatici

Ierarhia lui Chomsky

Arbori de derivare

Gramatici

Definitie generativa a unui limbaj: Un cuadru (T, N, Z, P) este o gramatica pentru limbajul $L(G)$

$$L(G) = \{\chi \in T^* \mid Z \Rightarrow^+ \chi\}$$

daca

- ▶ T si N disjuncte, formeaza impreuna vocabularul
- ▶ $(T \cup N, P)$ este un sistem de rescriere
- ▶ $Z \in N$

Doua gramatici sunt echivalente daca $L(G_1) = L(G_2)$

- ▶ T - multimea terminalelor
- ▶ N - multimea nonterminalelor sau a variabilelor sintactice
- ▶ Z - nonterminal anume, simbol de start
- ▶ in limbaj - sirurile derivabile din simbolul de start si care constau doar din terminalelor

Gramatica G genereaza limbajul L

Exemplul 1

Fie gramatica $G = (\{0, 1\}, \{S\}, S, P = \{S \rightarrow 0S1, S \rightarrow \varepsilon\})$

Care dintre stringurile de mai jos $\in L(G)$?

- ▶ 01
- ▶ 010
- ▶ 1
- ▶ 01010
- ▶ ε
- ▶ $0^n 1^n$

Exemplul 1

Fie gramatica $G = (\{0, 1\}, \{S\}, S, P = \{S \rightarrow 0S1, S \rightarrow \varepsilon\})$
Care dintre stringurile de mai jos $\in L(G)$?

- ▶ 01
- ▶ 010
- ▶ 1
- ▶ 01010
- ▶ ε
- ▶ $0^n 1^n$

$$S \Rightarrow^* 0^n 1^n$$

$$L(G) = \{0^n 1^n \mid n \geq 0\}$$

Exemplul 2

Fie gramatica $G = (T, N, S, P)$ unde

- ▶ $T = \{a, b\}$
- ▶ $N = \{S, A, B\}$
- ▶ cu productiile P
 1. $S \rightarrow AB$
 2. $A \rightarrow aA$
 3. $A \rightarrow \varepsilon$
 4. $B \rightarrow bB$
 5. $B \rightarrow \varepsilon$

Care stringuri apartin limbajului $L(G)$?

- ▶ ε
- ▶ a
- ▶ b
- ▶ $aaabb$

Exemplul 2

Fie gramatica $G = (T, N, S, P)$ unde

- ▶ $T = \{a, b\}$
- ▶ $N = \{S, A, B\}$
- ▶ cu productiile P
 1. $S \rightarrow AB$
 2. $A \rightarrow aA$
 3. $A \rightarrow \varepsilon$
 4. $B \rightarrow bB$
 5. $B \rightarrow \varepsilon$

Care stringuri apartin limbajului $L(G)$?

- ▶ ε
- ▶ a
- ▶ b
- ▶ $aaabb$

$$\begin{aligned} S &\Rightarrow^1 AB \Rightarrow^2 aAB \Rightarrow^2 aaAB \Rightarrow^2 aaaAB \\ &\Rightarrow^3 aaaB \Rightarrow^4 aaabB \Rightarrow^4 aaabbB \Rightarrow^5 aaabb \end{aligned}$$

Exemplul 3

Fie gramatica $G = (T, N, S, P)$ unde

- ▶ $T = \{a\}$
- ▶ $N = \{S, N, Q, R\}$
- ▶ cu productiile P
 1. $S \rightarrow QNQ$
 2. $QN \rightarrow QR$
 3. $RN \rightarrow NNR$
 4. $RQ \rightarrow NNQ$
 5. $N \rightarrow a$
 6. $Q \rightarrow \varepsilon$

Expresii aritmetice:

$$G_1 = (T, N, E, P)$$

- ▶ $T = \{+, *, (,), i\}$
- ▶ $N = \{E, T, F\}$
- ▶ cu productiile P
 1. $E \rightarrow T$
 2. $E \rightarrow E + T$
 3. $T \rightarrow F$
 4. $T \rightarrow T * F$
 5. $F \rightarrow i$
 6. $F \rightarrow (E)$

derivare pt i; dar pt $i*i$?

Expresii aritmetice: Gramatici echivalente

Doua gramatici sunt **echivalente** daca $L(G_1) = L(G_2)$

$G_1 = (T, N, E, P)$

- ▶ $T = \{+, *, (,), i\}$
- ▶ $N = \{E, T, F\}$
- ▶ cu productiile P
 1. $E \rightarrow T$
 2. $E \rightarrow E + T$
 3. $T \rightarrow F$
 4. $T \rightarrow T * F$
 5. $F \rightarrow i$
 6. $F \rightarrow (E)$

$G_2 = (T, N, E, P)$

- ▶ $T = \{+, *, (,), i\}$
- ▶ $N = \{E, E', T, T', F\}$
- ▶ cu productiile P
 1. $E \rightarrow T$
 2. $E \rightarrow Te'$
 3. $E' \rightarrow +T$
 4. $E' \rightarrow +TE'$
 5. $T \rightarrow F$
 6. $T \rightarrow FT'$
 7. $T' \rightarrow *F$
 8. $T' \rightarrow *FT'$
 9. $F \rightarrow i$
 10. $F \rightarrow (E)$

Outline

Siruri si sisteme de rescriere

Gramatici

Ierarhia lui Chomsky

Arbori de derivare

Ierarhia lui Chomsky

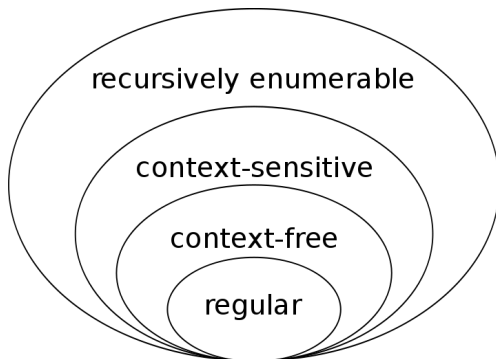
Conceptul de **Clasificarea gramaticilor** - 1950, **Noam Chomsky** - parintele lingvisticii moderne, unul dintre fondatorii stiintelor cognitive

- Modalitate de a descrie complexitatea structurala a unor propozitii particulare din limbajul natural

- ▶ Limbajele clasificate in functie de gramatica care le genereaza: **constrangeri asupra productiilor gramaticii definesc diferite clase de gramatici/limbaje**

Ierarhia lui Chomsky

- ▶ Gramatica de tip 0
- ▶ Gramatica de tip 1 - **dependenta de context**
- ▶ Gramatica de tip 2 - **independenta de context**
- ▶ Gramatica de tip 3 - **regulata**



Gramatica de Tip 0

$$G = (T, N, Z, P)$$

- ▶ cele mai generale gramatici
- ▶ fiecare productie are forma

$$\sigma \rightarrow \tau, \sigma \in V^+, \tau \in V^*$$

Gramatica din exemplul 3 este de Tip 0 (si nu si de tip 1,2,3)

$$RN \rightarrow NNR$$

$$RQ \rightarrow NNQ$$

Gramatica de Tip 1 - Dependente de context

$$G = (T, N, Z, P)$$

- fiecare productie are forma

$$\mu X \nu \rightarrow \mu \chi \nu, \quad \mu, \nu \in V^*, X \in N, \chi \in V^+$$

Context-sensitive (dependenta de context) - contextul lui X

Gramatica de Tip 1 - Dependente de context

$$G = (T, N, Z, P)$$

- fiecare productie are forma

$$\mu X \nu \rightarrow \mu \chi \nu, \quad \mu, \nu \in V^*, X \in N, \chi \in V^+$$

Context-sensitive (dependenta de context) - contextul lui X

Gramatica de Tip 2 - Independentă de context

$$G = (T, N, Z, P)$$

- ▶ fiecare producție are forma

$$X \rightarrow \chi, X \in N, \chi \in V^*$$

Gramatica de Tip 2 - Independentă de context

$$G = (T, N, Z, P)$$

- ▶ fiecare producție are forma

$$X \rightarrow \chi, X \in N, \chi \in V^*$$

- ▶ Context-free grammars - suficient de puternice pentru a descrie sintaxa limbajelor de programare
- ▶ permit construirea unor algoritmi eficienți de parsare:
determină dacă un string este sau nu generat din gramatica

gramatica din Exemplul 1 este Context-free grammar.

$$S \rightarrow 0S1 \text{ DA}$$

$$QN \rightarrow QR \text{ și } RN \rightarrow NNR \text{ NU}$$

Gramatica de Tip 3 - Regulate

$$G = (T, N, Z, P)$$

- fiecare productie are forma

$$X \rightarrow t, \quad X \in N, \quad t \in T \cup \{\varepsilon\}$$

sau

$$X \rightarrow tY, \quad X, Y \in N, \quad t \in T$$

Gramatica de Tip 3 - Regulate

$$G = (T, N, Z, P)$$

- ▶ fiecare productie are forma

$$X \rightarrow t, \quad X \in N, \quad t \in T \cup \{\varepsilon\}$$

sau

$$X \rightarrow tY, \quad X, Y \in N, \quad t \in T$$

- ▶ Regular grammars: de obicei folosite pt a defini structura lexicala a limbajelor de programare

Ierarhia lui Chomsky - rezumat

tip 0

$$\sigma \rightarrow \tau$$

$$\sigma \in V^+, \tau \in V^*$$

*dependenta de
context*

$$\mu X \nu \rightarrow \mu \chi \nu$$

$$\mu, \nu \in V^*, X \in N, \chi \in V^+$$

*independenta de
context*

$$X \rightarrow \chi$$

$$X \in N, \chi \in V^*$$

regulata

$$X \rightarrow t,$$

$$X \rightarrow tY,$$

$$X \in N, t \in T \cup \{\varepsilon\} \text{ sau}$$

$$X, Y \in N, t \in T$$

Ierharhia Chomsky - concluzii

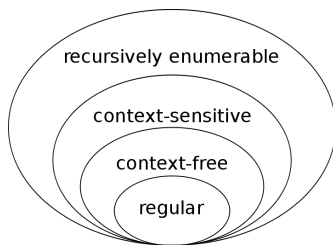
- ▶ productia $S \rightarrow \varepsilon$ productii ε
Admise in gramaticile independente de context, regulate - limbajele se pot descrie printr-o gramatica si fara productia ε
- ▶ Every Regular Language is Context-Free, every Context-Free Language is Context-Sensitive and every Context-Sensitive Language is a Type 0 Language.
- ▶ fiecare simbol din vocabular apare in derivarea cel putin a unei propozitii; (nu exista simboluri inutile)

Ierarhia lui Chomsky - concluzii

- ▶ Gramatica de tip 0
- ▶ Gramatica de tip 1 - **dependenta de context**
- ▶ Gramatica de tip 2 - **independenta de context**
- ▶ Gramatica de tip 3 - **regulata**

Pt compilatoare: gramaticile regulate si independente de context

- ▶ Simboluri fundamentale ale limbajului (identificatori, constante..): gramatici regulate
- ▶ Structura programului: gramatici independente de context



Outline

Siruri si sisteme de rescriere

Gramatici

Ierarhia lui Chomsky

Arbori de derivare

Derivare si arbori - G1 - expresii aritmetice

$P = (E \rightarrow T, E \rightarrow E + T, T \rightarrow F, T \rightarrow T * F, F \rightarrow i, F \rightarrow (E))$

Derivari pentru $i+i*i$

Derivare stanga	Arbitrar	Derivare dreapta
E	E	E
$E + T$	$E + T$	$E + T$
$T + T$	$E + T * F$	$E + T * F$
$F + T$	$T + T * F$	$E + T * i$
$i + T$	$T + F * T$	$E + F * i$
$i + T * F$	$T + F * i$	$E + i * i$
$i + F * F$	$F + F * i$	$T + i * i$
$i + i * F$	$i + F * i$	$F + i * i$
$i + i * i$	$i + i * i$	$i + i * i$

Derivare si arbori - G1 - expresii aritmetice

$P = (E \rightarrow T, E \rightarrow E + T, T \rightarrow F, T \rightarrow T * F, F \rightarrow i, F \rightarrow (E))$

Derivari pentru $i+i*i$

Derivare stanga	Arbitrar	Derivare dreapta
E	E	E
$E + T$	$E + T$	$E + T$
$T + T$	$E + T * F$	$E + T * F$
$F + T$	$T + T * F$	$E + T * i$
$i + T$	$T + F * T$	$E + F * i$
$i + T * F$	$T + F * i$	$E + i * i$
$i + F * F$	$F + F * i$	$T + i * i$
$i + i * F$	$i + F * i$	$F + i * i$
$i + i * i$	$i + i * i$	$i + i * i$

La ce se refera Structura conferita de gramatica unui sir?

- ▶ ? Secventa pasilor de derivare
- ▶ ? Relatia ce arata din ce subsir este derivat un anumit nonterminal

$i*i$ este derivat tot timpul din T

Derivare si arbori - G1 - expresii aritmetice

$P = (E \rightarrow T, E \rightarrow E + T, T \rightarrow F, T \rightarrow T * F, F \rightarrow i, F \rightarrow (E))$

Derivari pentru $i+i*i$

Derivare stanga	Arbitrar	Derivare dreapta
E	E	E
$E + T$	$E + T$	$E + T$
$T + T$	$E + T * F$	$E + T * F$
$F + T$	$T + T * F$	$E + T * i$
$i + T$	$T + F * T$	$E + F * i$
$i + T * F$	$T + F * i$	$E + i * i$
$i + F * F$	$F + F * i$	$T + i * i$
$i + i * F$	$i + F * i$	$F + i * i$
$i + i * i$	$i + i * i$	$i + i * i$

La ce se refera Structura conferita de gramatica unui sir?

- ▶ **NU** Secventa pasilor de derivare
- ▶ **DA** Relatia ce arata din ce subsir este derivat un anumit nonterminal

$i*i$ este derivat tot timpul din T

Derivare cont.

- ▶ $T \Rightarrow^+ i * i$ - unitate semantica: operatorul $*$ se aplica operanzilor i
- ▶ gramatica - **structura semantica relevanta** fiecarei propozitii din limbaj
- ▶ Daca $E \rightarrow E + T, T \rightarrow T * F$ schimbam in $E \rightarrow E * T, T \rightarrow T + F$
multimea de siruri va fi aceeaasi cu G_1 , dar structura propozitiilor va fi alta: adunarile mai prioritare decat inmultirile

Derivari cont.

Fie gramatica $G = (T, N, Z, P)$.

- Sirul $\chi \in V^+$ este o fraza pentru X a lui $\mu\chi\nu$ daca si numai daca

$$Z \Rightarrow^* \mu X \nu \Rightarrow^* \mu \chi \nu$$

unde $\mu, \nu \in V^*, X \in N$

- Sirul $\chi \in V^+$ este o fraza simpla a lui $\mu\chi\nu$ daca si numai daca

$$Z \Rightarrow^* \mu X \nu \Rightarrow \mu \chi \nu$$

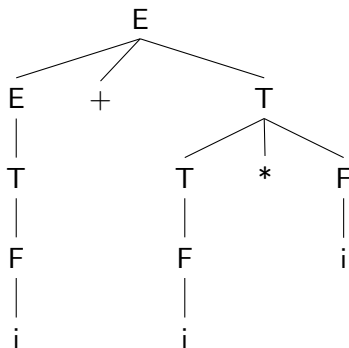
Subsirurile derivate din nonterminale singulare se numesc fraze.

Obs: fraza nu consta numai din terminale:

$E \Rightarrow^* E + T \Rightarrow^* E + F * F$. Deci $F * F$ derivat din T

Set de fraze - Arborele de derivare

- ▶ Toate cele trei derivari pt gramatica expresiilor aritmetice sunt echivalente: confera acelasi set de fraze.
- ▶ Arborele de derivare - reprezentarea intregului set de derivari echivalente; structura frazala



- ▶ din arbore de parsare:
orice sir din orice derivare
a unei propozitii -
taietura = nr minim de
noduri care intrerup
calea de la radacina catre
frunze
- ▶ exemplu: T, +, T, *, F,
E, +, T

Arbori de derivare

- ▶ parse tree - metoda vizuala de a descrie orice derivare dintr-o gramatica independenta de context (Context-free grammar CFG)
- ▶ fiecare nod are un label
- ▶ radacina este simbolul de start al gramaticii
- ▶ daca un nod n , etichetat cu A are cel putin un descendent, A este in N
- ▶ daca nodurile $n_1, n_2, \dots, n_k$ sunt descendentii unui nod n , cu etichetele $A_1, A_2, \dots, A_k$ atunci

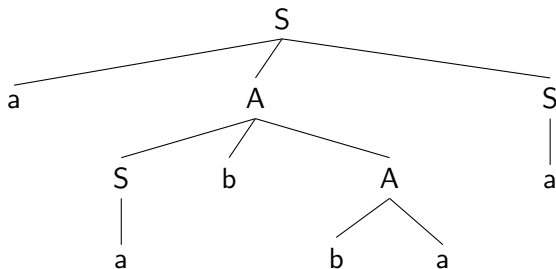
$$A \rightarrow A_1, A_2, \dots, A_k$$

este o productie in P

Exemplu

Fie $G = (\{a, b\}, \{S, A\}, S, P)$

- ▶ $S \rightarrow aAS$
- ▶ $S \rightarrow a$
- ▶ $A \rightarrow SbA$
- ▶ $A \rightarrow ba$
- ▶ $A \rightarrow SS$



Exemplu

Fie $G = (\{a, b\}, \{S, A\}, S, P)$

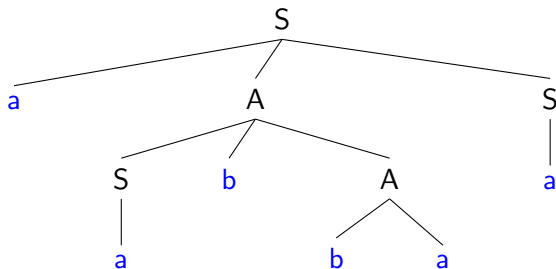
► $S \rightarrow aAS$

► $S \rightarrow a$

► $A \rightarrow SbA$

► $A \rightarrow ba$

► $A \rightarrow SS$



$S \Rightarrow^* aabbbaa$

$S \rightarrow aAS \rightarrow aSbAS \rightarrow aabAS \rightarrow aabbbaS \rightarrow aabbbaa$

Stanga la dreapta frunzele: propozitie (rezultatul arborelui de derivare)

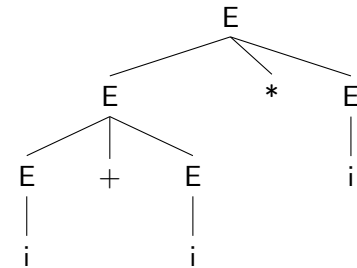
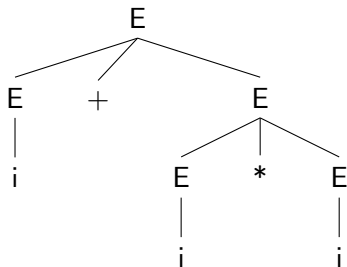
Ambiguity

Let $G_4 = (\{+, *, i\}, \{E\}, E, P)$

► $E \rightarrow E + E$

► $E \rightarrow E * E$

► $E \rightarrow i$



$2+3*4??$

Ambiguitate

“Look at the dog with one eye”

- ▶ O propozitie este ambigua daca derivarile sale pot fi descrise prin cel putin doi arbori de derivare distincti.
- ▶ O gramatica este ambigua daca in limbajul generat exista cel putin o propozitie ambigua
- ▶ O gramatica este ambigua daca genereaza mai mult de o derivare cea mai din stanga pentru vreo propozitie

Rezumat

Siruri si sisteme de rescriere

Gramatici

Ierarhia lui Chomsky

Arbori de derivare

Exemplu

Fie $G = (\{the, a, reads, walks, kid, robot\},$
 $\{S, NounPhrase, Predicate, Article, Noun, Verb\}, S, P)$

- ▶ $S \rightarrow NounPhrase Predicate$
- ▶ $NounPhrase \rightarrow Article Noun$
- ▶ $Predicate \rightarrow Verb$
- ▶ $Article \rightarrow the$
- ▶ $Article \rightarrow a$
- ▶ $Verb \rightarrow reads$
- ▶ $Verb \rightarrow walks$
- ▶ $Noun \rightarrow kid$
- ▶ $Noun \rightarrow robot$

Gramatici regulate. Automate finite

09.08.2021

Outline

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Introducere in automate. Automate deterministe finite

- ▶ Letia & Chifu 2.2: 2.2.1
- ▶ capitolul 1.1: Finite automata, FORMAL DEFINITION OF A FINITE AUTOMATON , EXAMPLES OF FINITE AUTOMATA, FORMAL DEFINITION OF COMPUTATION
“Introduction to the Theory of computation” 3rd edition,
Michael Sipser
- ▶ Introduction to Automata Theory, Languages, and Computation sections 2.1, 2.2, Ullman

Automate finite

- ▶ modele pentru calculatoare cu extrem de putina memorie
- ▶ colectie finita de **stari** cu **reguli de tranzitie** care determina trecerea dintr-o stare in alta

Reprezentarea FA - State diagram

- ▶ noduri
- ▶ arce - indica tranzitia starilor
- ▶ etichete (labels) pe arce care definesc ce cauzeaza tranzitia

Exemplu: Recunoasterea cuvintelor care se termina in “.ing”

ingest, reading

Automat → Cod

1. citește următorul input
2. decide starea următoare
3. sare la începutul codului pentru acea stare

```
2: /* i seen */
   c = getNextInput();
   if (c=='n') goto 3;
   else if (c=='i') goto 2;
   else goto 1;
3:  /* "in" seen */
...

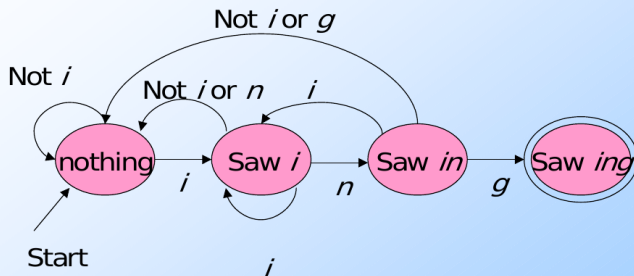
```

Automat → Cod

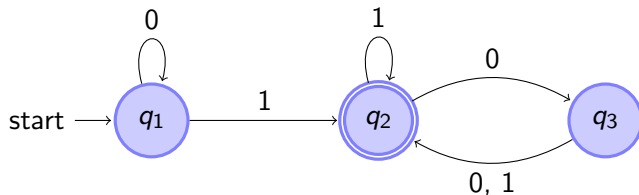
1. citește următorul input
2. decide starea următoare
3. sare la începutul codului pentru acea stare

```
2: /* i seen */  
  c = getNextInput();  
  if (c=='n') goto 3;  
  else if (c=='i') goto 2;  
  else goto 1;  
3: /* "in" seen */  
...
```

de fapt: expresii regulate `.ing`



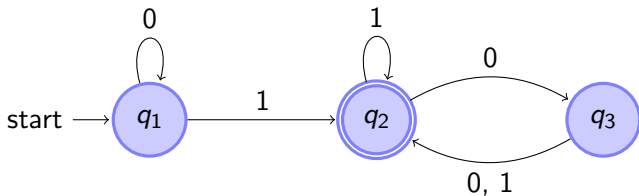
Exemplu: Automat



- ▶ 3 stari; start state, accept state
- ▶ transitions

Automatul primește un input string și produce *accept* sau *reject*.
fie 1101:

1. Start in q_0
2. Citeste 1 și urmează tranziția q_1 to q_2
3. Citeste 1 și urmează tranziția q_2 to q_2
4. Citeste 0 și urmează tranziția q_2 to q_3
5. Citeste 1 și urmează tranziția q_3 to q_2
6. *accept* deoarece se afla în starea accept q_2 la sfârșitul input-ului

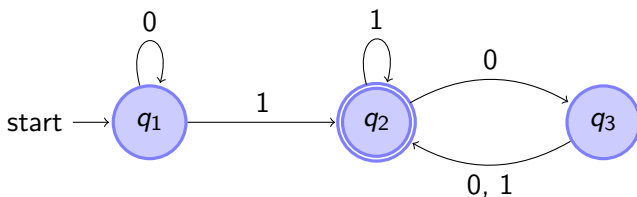


- ▶ Accepta 1, 01, 11, 0101010101?
- ▶ Dar 100, 0100, 110000, 0101000000?
- ▶ dar 0, 10, 101000?

care sunt toate stringurile pe care automatul le accepta?

Setul tuturor sirurilor recunoscute de an automat A : $L(A)$

$L(A) = ?$



- Accepta 1, 01, 11, 0101010101? DA
- Dar 100, 0100, 110000, 0101000000? Da
- dar 0, 10, 101000? le respinge

care sunt toate stringurile pe care automatul le accepta?

Setul tuturor sirurilor recunoscute de an automat A : $L(A)$

$L(A) = \{w | w \text{ contine cel putin un } 1 \text{ si se termina cu un numar par de } 0\text{-uri dupa ultimul } 1\}$

Table of Contents

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Automat finit determinist (Deterministic Finite Automaton) - *formal* definition

$$(\Sigma, Q, \delta, q_0, F)$$

- ▶ un alfabet de intrare Σ - set de simboluri
- ▶ un set finit de stari Q
- ▶ o functie de tranzitie δ
- ▶ o stare de start q_0
- ▶ un set de stari finale $F \subseteq Q$ (final state, accepting states)

Functia de tranzitie $\delta : Q \times \Sigma \rightarrow Q$: $\delta(q, a)$ starea in care automatul DFA trece cand este in starea q si primeste ca input a .

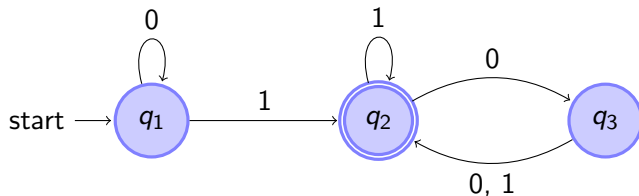
Setul tuturor sirurilor recunoscute de un automat A :

$$L(A) = \{w | A \text{ accepta } w\}$$

Descrierea formală a automatului:

$$D_1 = (\{0, 1\}, \{q_1, q_2, q_3\}, \delta, q_1, \{q_2\})$$

δ		0	1
$\rightarrow$	q_1	q_1	q_2
*	q_2	q_3	q_2
	q_3	q_2	q_2



Acceptare 011 ? $\exists \delta(q_1, 0)$:

$$\delta(q_1, 0) = q_1; \delta(q_1, 1) = q_2; \delta(q_2, 1) = q_2 \in F$$

s-a gasit secventa de stari: q_1, q_1, q_2

Definitie formală a calculului

Fie $A = (\Sigma, Q, \delta, q_0, F)$ și $w = w_1 w_2 \dots w_n$, $w_i \in \Sigma$. Automatul recunoaște w dacă există o secvență $r_0, r_1, \dots, r_n \in Q$:

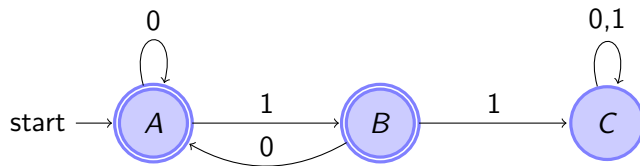
Definitie formală a calculului

Fie $A = (\Sigma, Q, \delta, q_0, F)$ și $w = w_1 w_2 \dots w_n$, $w_i \in \Sigma$. Automatul recunoaște w dacă există o secvență $r_0, r_1, \dots, r_n \in Q$:

- ▶ $r_0 = q_0$
- ▶ $\delta(r_i, w_{i+1}) = r_{i+1}$ pt $i = 0, \dots, n-1$
- ▶ $r_n \in F$

Exemplu:

Ce accepta?

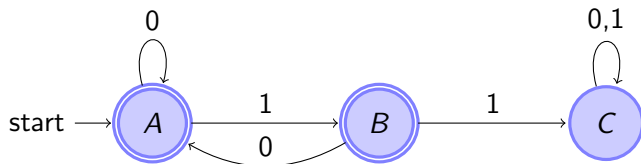


Extended $\hat{\delta}$

$$\begin{aligned}\hat{\delta}(A, 011) &= \delta(\delta(\delta(A, 0), 1), 1) = \\ &\delta(\delta(A, 1), 1) = \delta(B, 1) = C\end{aligned}$$

Exemplu:

Ce accepta?



Extended $\hat{\delta}$

$$\begin{aligned}\hat{\delta}(A, 011) &= \delta(\delta(\delta(A, 0), 1), 1) = \\ &\delta(\delta(A, 1), 1) = \delta(B, 1) = C\end{aligned}$$

Accepta toate stringurile care nu includ doua simboluri consecutive
1

Table of Contents

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

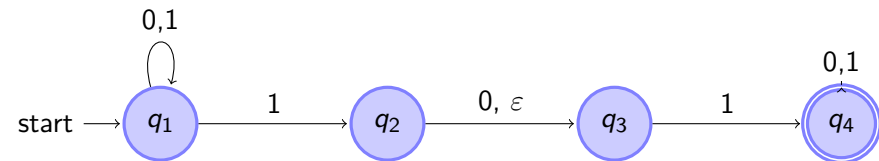
Automat finit nedeterminist - *formal* definition

$$(\Sigma, Q, \delta, q_0, F)$$

- ▶ un alfabet de intrare Σ - set de simboluri
- ▶ un set finit de stari Q
- ▶ o functie de tranzitie δ
- ▶ o stare de start q_0
- ▶ un set de stari finale $F \subseteq Q$ (final state, accepting states)

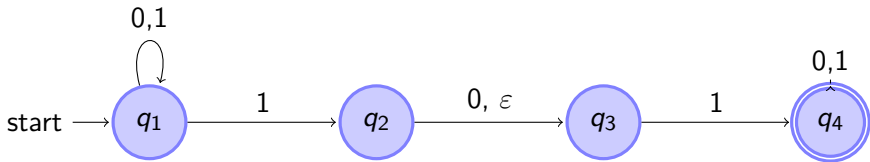
Functia de tranzitie $\delta : Q \times \Sigma_\epsilon \rightarrow P(Q)$: $\delta(q, a)$ starea/**starile** in care automatul NFA poate trece cand este in starea q si primeste ca input a .

Exemplu - automat nedeterminist



Input: 010110

Exemplu - automat nedeterminist



Input: 010110 Calcul nedeterminist: accept/reject

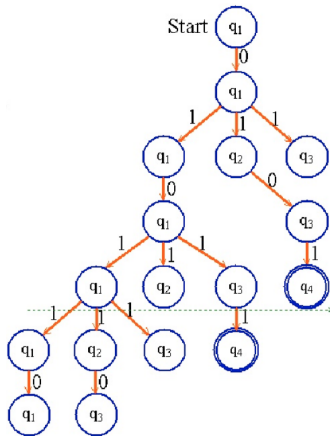


Table of Contents

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Outline

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Automat finit - definitie formală ca sistem de rescriere

Automat finit (finite automaton, finite state acceptor):

$$A = (T, Q, R, q_0, F)$$

- ▶ Q set nevid - setul starilor interne
- ▶ $(T \cup Q, R)$ sistem de rescriere; $T \cap Q = \emptyset$
- ▶ $q_0 \in Q$ - starea initiala
- ▶ $F \subseteq Q$ - stari finale
- ▶ fiecare element din R are forma $qt \rightarrow q'$, $q, q' \in Q, t \in T$

Definitie formală *Automat finit*

- ▶ automatul A accepta/recunoaste setul de stringuri

$$L(A) = \{\tau \in T^* \mid q_0\tau \Rightarrow^* q, q \in F\}$$

- ▶ Doua automate A și A' sunt **echivalente** dacă și numai dacă $L(A) = L(A')$

Interpretare

- ▶ mașina care citește la intrare un input string; citește simbol cu simbol și își schimbă starea internă
- ▶ automatul se află în starea q când sirul curent din derivare este $q\tau$
- ▶ automatul face o tranziție din q în q' dacă $\tau = t\chi$ și $?? \in R$
 $q\tau = qt\chi \Rightarrow ???$
- ▶ fiecare tranziție șterge un simbol din stringul de intrare

Definitie formală *Automat finit*

- ▶ automatul A accepta/recunoaste setul de stringuri

$$L(A) = \{\tau \in T^* \mid q_0\tau \Rightarrow^* q, q \in F\}$$

- ▶ Doua automate A și A' sunt **echivalente** dacă și numai dacă $L(A) = L(A')$

Interpretare

- ▶ mașina care citește la intrare un input string; citește simbol cu simbol și își schimbă starea internă
- ▶ automatul se află în starea q când sirul curent din derivare este $q\tau$
- ▶ automatul face o tranziție din q în q' dacă $\tau = t\chi$ și $qt \rightarrow q' \in R$ $q\tau = qt\chi \Rightarrow q'\chi$
- ▶ fiecare tranziție șterge un simbol din stringul de intrare

Exemplu automat vazut ca sistem de rescriere

$$A = (T = \{0, 1\}, Q = \{q_0, q_1\}, R, q_0, F = \{q_1\})$$

$$R = \{ \begin{array}{l} q_0 1 \rightarrow q_1 \\ q_0 0 \rightarrow q_0 \\ q_1 1 \rightarrow q_0 \\ q_1 0 \rightarrow q_1 \end{array} \}$$

Intrebare: 1001 apartine limbajului automatului? Dar 10?

?Exista derivarea

$$q_0 1001 \Rightarrow^* q_1$$

Table of Contents

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Teorema

- Pentru fiecare gramatica regulata G exista un automat finit A a.i. $L(A) = L(G)$.

Reamintire: gramatica regulata (din ierarhia lui Chomsky)
 $G = (T, N, Z, P)$

- fiecare productie are forma

$$X \rightarrow t, \quad X \in N, \quad t \in T \cup \{\varepsilon\}$$

sau

$$X \rightarrow tY, \quad X, Y \in N, \quad t \in T$$

Construirea AF pentru gramatica regulata G

- **Algoritm** Construirea automatului $A = (T, N \cup \{f\}, R, Z, F)$, $f \notin N$ pentru gramatica $G = (T, N, Z, P)$.
1. daca $X \rightarrow t \in P$, $X \in N, t \in T$, atunci $Xt \rightarrow f \in R$
 2. daca $X \rightarrow tY \in P$, $X, Y \in N, t \in T$, atunci $Xt \rightarrow Y \in R$
 3. $F = \{f\} \cup \{X | X \rightarrow \varepsilon \in P\}$

Gramatica pentru constante reale - gramatica regulata

Fie gramatica G_3

- ▶ $T = \{n, ., +, -, E\}$
- ▶ $N = \{C, F, I, X, S, U\}$
- ▶ $P = \{$

$C \rightarrow n, C \rightarrow nF, C \rightarrow .I,$

$F \rightarrow .I, F \rightarrow ES,$

$I \rightarrow n, I \rightarrow nX,$

$X \rightarrow ES,$

$S \rightarrow n, S \rightarrow +U, S \rightarrow -U,$

$U \rightarrow n\}$

Exemple de derivare:

- ▶ $C \Rightarrow n$
- ▶ $C \Rightarrow .I \Rightarrow .n$
- ▶ $C \Rightarrow nF \Rightarrow n.I \Rightarrow n.nX \Rightarrow n.nES \Rightarrow n.nE + U \Rightarrow n.nE + n$

FA pentru G_3

Gramatica regulata

- ▶ $T = \{n, ., +, -, E\}$
- ▶ $N = \{C, F, I, X, S, U\}$
- ▶ $P = \{$
 $C \rightarrow n, C \rightarrow nF, C \rightarrow .I,$
 $F \rightarrow .I, F \rightarrow ES,$
 $I \rightarrow n, I \rightarrow nX,$
 $X \rightarrow ES,$
 $S \rightarrow n, S \rightarrow +U, S \rightarrow -U,$
 $U \rightarrow n\}$

Automat finit

- ▶ $T = \{n, ., +, -, E\}$
- ▶ $Q = \{C, F, I, X, S, U, q\}$
- ▶ $P = \{$
 $Cn \rightarrow q, Cn \rightarrow F, C. \rightarrow I,$
 $F. \rightarrow I, FE \rightarrow S,$
 $In \rightarrow q, In \rightarrow X,$
 $XE \rightarrow S,$
 $Sn \rightarrow q, S+ \rightarrow U, S- \rightarrow U,$
 $Un \rightarrow q\}$
- ▶ $q_0 = C$
- ▶ $F = \{q\}$

Derivare $n.n$

Gramatica

$$C \Rightarrow nF \Rightarrow n.l \Rightarrow n.n$$

Automat

$$Cn.n \Rightarrow F.n \Rightarrow ln \Rightarrow q$$

Pentru orice $Z\tau\chi \Rightarrow^* X\chi \Rightarrow^* q$, $\tau, \chi \in T^*$, $X \in N$, $\tau\chi \in L(A)$,
 $q \in F$,

starea X specifica nonterminalul din G care ar fi trebuit utilizat
pentru derivarea lui χ

Derivare $n.n$

Gramatica

$$C \Rightarrow nF \Rightarrow n.l \Rightarrow \textcolor{red}{n.n}$$

Automat

$$\textcolor{red}{C}n.n \Rightarrow F.n \Rightarrow ln \Rightarrow q$$

Pentru orice $Z\tau\chi \Rightarrow^* X\chi \Rightarrow^* q$, $\tau, \chi \in T^*$, $X \in N$, $\tau\chi \in L(A)$,
 $q \in F$,

starea X specifica nonterminalul din G care ar fi trebuit utilizat
pentru derivarea lui χ

Derivare $n.n$

Gramatica

$$C \Rightarrow nF \Rightarrow n.l \Rightarrow n.n$$

Automat

$$Cn.n \Rightarrow F.n \Rightarrow ln \Rightarrow q$$

Pentru orice $Z\tau\chi \Rightarrow^* X\chi \Rightarrow^* q$, $\tau, \chi \in T^*$, $X \in N$, $\tau\chi \in L(A)$,
 $q \in F$,

starea X specifica nonterminalul din G care ar fi trebuit utilizat
pentru derivarea lui χ

Proprietati ale automatului

Pentru orice $Z\tau\chi \Rightarrow^* X\chi \Rightarrow^* q$, $\tau, \chi \in T^*$, $X \in N$, $\tau\chi \in L(A)$,
 $q \in F$,

starea X specifica nonterminalul din G care ar fi trebuit utilizat
pentru derivarea lui χ

Demonstratie prin inductie

- ▶ daca $\tau\chi \in L(G)$. afirmatia este adevarata pentru Z stare initiala
- ▶ proprietatea ramana adevarata pana la starea finala Q , care nu genereaza alte simboluri

Fiecare propozitie din $L(G)$ apartine lui $L(A)$ si invers

Evitarea backtrackingului

- ▶ Automatul generat este nedeterminist: stare / cu inputul n sunt mai multe tranzitii posibile
- ▶ la implementare: backtracking necesar in cazul unei decizii incorecte
- ▶ motive pentru evitarea backtrackingului:
 - ▶ timpul necesar parsarii unui string cu backtracking poate creste exponential cu lungimea stringului
 - ▶ daca automatul nu accepta stringul, stringul va fi recunoscut drept incorect. Pinpointingul (tratarea erorilor) devine dificil cu backtracking
 - ▶ deoarece in compilator, tranzitiilor de stare le sunt asociate anumite actiuni, la revenire ar trebui anulara acelor actiuni

Automat finit determinist

Un automat este determinist daca fiecare derivare poate fi continuata prin cel mult o mutare.

→ Determinist daca Partile stanga ale tuturor productiilor sunt distincte

Poate fi definit prin **Tabelul de stare** (state table): q, t contine q' daca si numai daca $qt \rightarrow q' \in R$

Backtrackingul poate fi intotdeauna evitat cand se recunosc stringuri pentru limbaje regulate

Automat finit determinist (deterministic finite automaton)

Pentru orice gramatica regulata G , exista un automat finit **determinist** A (DFA) a.i. $L(A) = L(G)$

Algoritm construire DFA

Idee: construim un automat pentru gramatica $G = (T, N, Z, P)$ a.i. in timpul acceptarii unei propozitii din $L(G)$, starea la fiecare pas sa mentioneze elementul N utilizat pentru a deriva restul stringului.

Daca $X \rightarrow tU$ si $X \rightarrow tV \in P$,

atunci cand t este urmatorul simbol, restul stringului poate fi derivat atat din U cat si din V

dar pentru a avea DFA, R trebuie sa contina o singura productie $Xt \rightarrow q'$

deci starea q' trebuie sa contina un set de nonterminale - acelea care puteau fi utilizate pentru derivarea restului sirului

Algoritm construire DFA pt $G=(T,N,Z,P)$

$A = (T, Q, R, q_0, F)$, q reprezinta $N_q \subseteq N \cup \{f\}$, $f \notin N$

1. initial $Q = \{q_0\}$ si $R = \emptyset$, $N_{q_0} = \{Z\}$
2. pentru $q \in Q$ netratat se efectueaza pasii 3-5 pentru fiecare $t \in T$
3. fie $next(q, t) = \{U | \exists X \in N_q \text{ a.i. } X \rightarrow tU \in P\}$
4. daca exista un $X \in N_q$ a.i. $X \rightarrow t \in P$, atunci adauga f la $next(q, t)$ daca nu era deja adaugat;
daca exista $X \in N_q$ a.i. $X \rightarrow \varepsilon \in P$ atunci adauga f la N_q
5. daca $next(q, t) \neq \emptyset$, atunci fie q' starea ce reprezinta $N_{q'} = next(q, t)$. Adauga q' la Q si $qt \rightarrow q'$ in R
6. daca toate starile din Q au fost considerate, atunci $F = \{q | f \in N_q\}$ si terminat; altfel continua cu pasul 2

DFA

	n	$.$	$+$	$-$	E	N
q_0	q_1	q_2				$\{C\}$
q_1		q_2			q_3	$\{f, F\}$
q_2	q_4					$\{I\}$
q_3	q_5		q_6	q_6		$\{S\}$
q_4					q_3	$\{f, X\}$
q_5						$\{f\}$
q_6	q_5					$\{U\}$

► $T = \{n, ., +, -, E\}$, $F = \{q_1, q_4, q_5\}$

► $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

► $P =$

$q_0 n \rightarrow q_1, q_0 . \rightarrow q_2,$

$q_1 . \rightarrow q_2, q_1 E \rightarrow q_3,$

$q_2 n \rightarrow q_4$

$q_3 n \rightarrow q_5, q_3 + \rightarrow q_6, q_3 - \rightarrow q_6,$

$q_4 E \rightarrow q_3$

$q_6 n \rightarrow q_5\}$

DFA

	n	$.$	$+$	$-$	E	N
q_0	q_1	q_2				$\{C\}$
q_1		q_2			q_3	$\{f, F\}$
q_2	q_4					$\{I\}$
q_3	q_5		q_6	q_6		$\{S\}$
q_4					q_3	$\{f, X\}$
q_5						$\{f\}$
q_6	q_5					$\{U\}$

► $T = \{n, ., +, -, E\}$, $F = \{q_1, q_4, q_5\}$

► $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

► $P =$

$q_0 n \rightarrow q_1, q_0 . \rightarrow q_2,$

$q_1 . \rightarrow q_2, q_1 E \rightarrow q_3,$

$q_2 n \rightarrow q_4$

$q_3 n \rightarrow q_5, q_3 + \rightarrow q_6, q_3 - \rightarrow q_6,$

$q_4 E \rightarrow q_3$

$q_6 n \rightarrow q_5\}$

Diagrama de stare

Fie $T = \{n, ., +, -, E\}$, $F = \{q_1, q_4, q_5\}$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

$P = \{q_0 n \rightarrow q_1, q_0 . \rightarrow q_2,$

$q_1 . \rightarrow q_2, q_1 E \rightarrow q_3,$

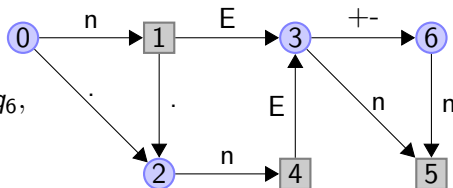
$q_2 n \rightarrow q_4$

$q_3 n \rightarrow q_5, q_3 + \rightarrow q_6, q_3 - \rightarrow q_6,$

$q_4 E \rightarrow q_3$

$q_6 n \rightarrow q_5\}$

Diagrama de stare



cale care incepe in q_0 si se termina intr-o stare finala $\in L(A)$

n	.n	n.n
nEn	nE+n	nE-n
.nEn	.nE+n	.nE-n
n.nEn	n.nE+n	n.nE-n

Diagrama de stare

Fie $A = (T, Q, R, q_0, F)$ un automat finit,

- ▶ $D = \{(q, q') | \exists t, qt \rightarrow q' \in R\}$,
- ▶ $f : (q, q') \rightarrow \{t | qt \rightarrow q' \in R\}$ o mapare de la D la $P(T)$

Graful directionat (Q, D) cu etichetele muchiilor $f((q, q'))$ este diagrama de stare a automatului A

Pentru fiecare automat finit A exista o gramatica regulata G a.i.
 $L(A) = L(G)$

Din automatul $A = (T, Q, R, q_0, F)$ construim gramatica
 $G = (T, Q, q_0, P)$:

$$P = \{q \rightarrow tq' | qt \rightarrow q' \in R\} \cup \{q \rightarrow \varepsilon | q \in F\}$$

Gramatici pentru automat

$$F = \{q_1, q_4, q_5\} \quad P = \{ \begin{array}{l} q_0 n \rightarrow q_1, q_0 \cdot \rightarrow q_2, \\ q_1 \cdot \rightarrow q_2, q_1 E \rightarrow q_3, \\ q_2 n \rightarrow q_4 \\ q_3 n \rightarrow q_5, q_3 + \rightarrow q_6, q_3 - \rightarrow q_6, \\ q_4 E \rightarrow q_3 \\ q_6 n \rightarrow q_5 \end{array} \}$$

Productii gramatica

$$\begin{array}{l} q_0 \rightarrow nq_1 | \cdot q_2, \\ q_1 \rightarrow \cdot q_2 | Eq_3 | \varepsilon, \\ q_2 \rightarrow nq_4 \\ q_3 \rightarrow nq_5 | + q_6 | - q_6, \\ q_4 \rightarrow Eq_3 | \varepsilon \\ q_5 \rightarrow \varepsilon \\ q_6 \rightarrow nq_5 \end{array}$$

Productii gramatica fara productii ε

$$\begin{array}{l} q_0 \rightarrow n | nq_1 | \cdot q_2, \\ q_1 \rightarrow \cdot q_2 | Eq_3, \\ q_2 \rightarrow n | nq_4 \\ q_3 \rightarrow n | + q_6 | - q_6, \\ q_4 \rightarrow Eq_3 \\ q_6 \rightarrow n \end{array}$$

Gramatici pentru automat

$$F = \{q_1, q_4, q_5\} \quad P = \{ \begin{array}{l} q_0 n \rightarrow q_1, q_0 \cdot \rightarrow q_2, \\ q_1 \cdot \rightarrow q_2, q_1 E \rightarrow q_3, \\ q_2 n \rightarrow q_4 \\ q_3 n \rightarrow q_5, q_3 + \rightarrow q_6, q_3 - \rightarrow q_6, \\ q_4 E \rightarrow q_3 \\ q_6 n \rightarrow q_5 \end{array} \}$$

Productii gramatica

$$\begin{array}{l} q_0 \rightarrow nq_1|.q_2, \\ q_1 \rightarrow .q_2|Eq_3|\epsilon, \\ q_2 \rightarrow nq_4 \\ q_3 \rightarrow nq_5|+q_6|-q_6, \\ q_4 \rightarrow Eq_3|\epsilon \\ q_5 \rightarrow \epsilon \\ q_6 \rightarrow nq_5\} \end{array}$$

Productii gramatica fara productii ϵ

$$\begin{array}{l} q_0 \rightarrow n|nq_1|.q_2, \\ q_1 \rightarrow .q_2|Eq_3, \\ q_2 \rightarrow n|nq_4 \\ q_3 \rightarrow n|+q_6|-q_6, \\ q_4 \rightarrow Eq_3 \\ q_6 \rightarrow n\} \end{array}$$

- ▶ Pentru orice gramatica regulata G , exista un automat finit A
a.i. $L(A) = L(G)$
- ▶ Pentru fiecare automat finit A exista o gramatica regulata G
a.i. $L(A) = L(G)$

Gramaticile regulate si automatele finite sunt echivalente

DFA vs NFA

Ambele automate, deterministe si nedeterministe, sunt capabile sa recunoasca toate limbajele regulate:

$$L(NFA) = L(DFA)$$

Diferenta principala: spatiu vs timp:

- ▶ DFA sunt mai rapide decat NFA
- ▶ DFA sunt exponential mai mari decat NFA

FA sunt folosite ca mdoele pentru:

- ▶ software for designing digital circuits
- ▶ lexical analyzer of a compiler
- ▶ software for verifying finite state systems, such as communication protocols: exemplul cu planeta

Rezumat

Introducere in automate. Automate deterministe finite

DFA - deterministic finite automaton

NFA - nondeterministic finite automaton

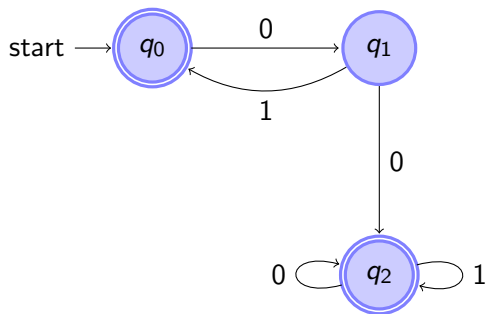
Gramatici regulate si automate finite

Automate finite vazute ca sisteme de rescriere

Automate finite pentru gramatici regulate

Extended Example - ullman slides

- ▶ On a distant planet, there are three species, a, b, and c. Any two different species can mate. If they do:
 1. The participants die.
 2. Two children of the third species are born.
- ▶ The planet fails if at some point all individuals are of the same species. Then, no more breeding can take place.
- ▶ State = sequence of three integers the numbers of individuals of species a, b, and c.



Gramatici regulate. Automate finite

March 23, 2021

Outline

Gramatici regulate 2. Expresii regulate

Diagrama de stare. Expresii regulate

- ▶ Letia & Chifu 2.2: 2.2.2

Regular expression - expresii regulate

Expresiile regulate descriu limbajele regulate

Fie V un vocabular si simbolurile $E, \varepsilon, +, *, (,) \notin V$.

Un string ρ peste $V \cup \{E, \varepsilon, +, *, (,)\}$ este o **expresie regulata** peste V daca

1. ρ este un simbol peste V sau unul dintre simbolurile E, ε , sau
2. ρ este de forma $(X + Y)$, (XY) , $(X)^*$, unde X si Y sunt expresii regulate.

Descriere expresii regulate

- ▶ $E = \emptyset$ este limbajul empty
- ▶ $\varepsilon = \{\varepsilon\}$ este limbajul format din stringul empty
- ▶ $v, v \in V$ descrie limbajul $\{v\}$
- ▶ $(X + Y) = \{w | w \in X \text{ sau } w \in Y\}$
- ▶ $XY = \{\chi\gamma | \chi \in X \text{ si } \gamma \in Y\}$
- ▶ operatorul $*$ inchidere (Kleene closure):

$$X^* = \varepsilon + X + XX + XXX + \dots$$

expresii regulate 2

- ▶ Parantezele se pot omite
- ▶ $*$ este operator unar, cu prioritate mai mare decat oricare operator binar
- ▶ $+$ are prioritate mai mica decat concatenarea

$W + XY^*$ este echivalent cu $(W + (X (Y^*)))$

Exemple expresii regulate

► $01 = \{01\}$

Exemple expresii regulate

- ▶ $01 = \{01\}$
- ▶ $01 + 0 = \{01, 0\}$ - in lex |

Exemple expresii regulate

- ▶ $01 = \{01\}$
- ▶ $01 + 0 = \{01, 0\}$ - in lex |
- ▶ $0(1 + 0) = \{01, 00\}$

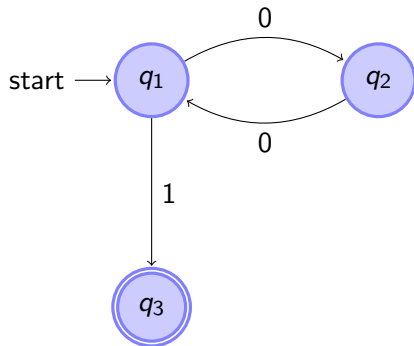
Exemple expresii regulate

- ▶ $01 = \{01\}$
- ▶ $01 + 0 = \{01, 0\}$ - in lex |
- ▶ $0(1 + 0) = \{01, 00\}$
- ▶ $0^* = \{\varepsilon, 0, 00, 000, \dots\}$

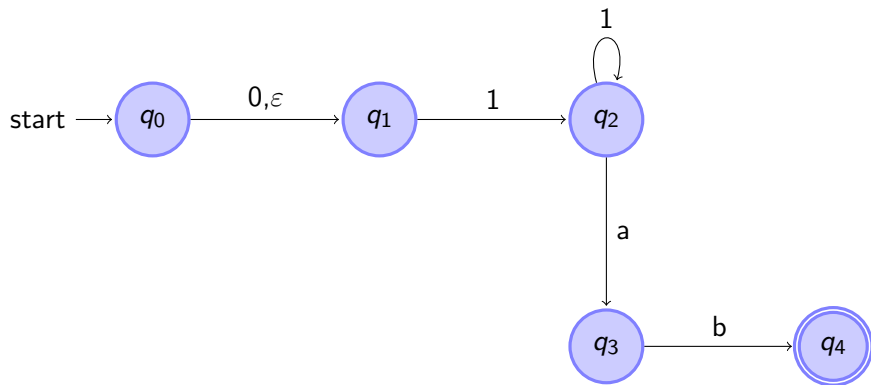
Exemple expresii regulate

- ▶ $01 = \{01\}$
- ▶ $01 + 0 = \{01, 0\}$ - in lex |
- ▶ $0(1 + 0) = \{01, 00\}$
- ▶ $0^* = \{\varepsilon, 0, 00, 000, \dots\}$
- ▶ $(0 + 10)^*(\varepsilon + 1)$ Toate stringurile de 0 si 1 fara doua consecutive 1

Exemplu $(00)^*1$



Exemplu $0?1^+ab$



Operatori aditionali: ? +

Nu permit definirea unor limbaje aditionale, dar permit exprimarea mai usoara a expresiilor regulate

- ▶ Operatorul optional: ?

Daca R este o expresie regulata, $R? = \varepsilon + R$

- ▶ Operatorul +

Daca R este o expresie regulata $R^+ = RR^*$:

$$L(R^+) = L(R) \cup L(RR) \cup L(RRR) \cup \dots$$

Proprietati algebrice ale expresiilor regulate

$$X + Y = Y + X$$

comutativitate

$$(X + Y) + Z = Z + (Y + Z)$$

asociativitate

$$X(YZ) = (XY)Z$$

$$X(Y + Z) = XY + XZ$$

distributivitate

$$(X + Y)Z = XZ + YZ$$

$$X + \emptyset = \emptyset + X = X$$

identitate

$$X\varepsilon = \varepsilon X$$

$$X\emptyset = \emptyset X = X$$

zero

$$X + X = X$$

idempotentă

$$(X^*)^* = X^*$$

$$X^* = \varepsilon + XX^*$$

$$X^* = X + X^*$$

$$\varepsilon^* = \varepsilon$$

$$\emptyset^* = \varepsilon$$

Echivalenta expresii regulate - automate finite

Fie R o expresie regulata care descrie un subset $S \subseteq T^*$.

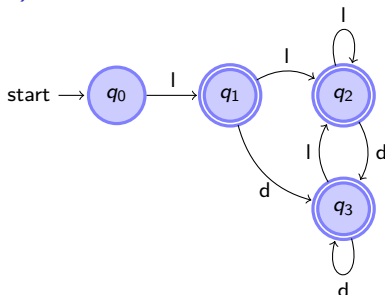
Exista un automat finit determinist $A = (T, Q, P, q_0, F)$ a.i.
 $L(A) = S$.

Construire automat

1. $R = l(l + d)^*$
2. $R' = 1(2 + 3)^*$ - o noua expresie in care se inlocuiesc elementele lui T din R cu simboluri distincte
Aparitii multiple ale aceluiasi element - simboluri diferite
3. $R' = 01(2 + 3)^*$ - se adauga un prefix (un simbol distinct)
Daca $R = E$ atunci R' este doar simbolul de start
4. starile automatului corespund submultimilor setului de simboluri.
5. Se inspecteaza pe rand starile lui Q si daca e necesar, se adauga stari noi:
pentru $\forall q \in Q$ si $\forall t \in T$, fie q' corespondentul setului de simboluri din R' :
 - ▶ care inlocuiesc pe t si
 - ▶ urmeaza unui simbol din setul corespunzator lui qDaca setul corespunzator lui q' nu e vid, se adauga $qt \rightarrow q'$ la P si se include q' in Q .
6. setul F de starile finale = toate starile care includ un simbol final posibil al lui R'

$$I(I + d)^*. \quad R' = 01(2 + 3)^*$$

	I	d	
q_0	q_1		$\{0\}$
q_1	q_2	q_3	$\{1\}$
q_2	q_2	q_3	$\{2\}$
q_3	q_2	q_3	$\{3\}$



q_0 I: $\{1, 2\}$, dar numai 1 urmeaza lui 0

q_0 d: $\{3\}$, dar nu urmeaza lui 0

q_1 I: $\{1, 2\}$, dar numai 2 urmeaza lui 1

q_1 d: $\{3\}$ si 3 urmeaza lui 1

q_2 I: $\{1, 2\}$, dar numai 2 urmeaza lui 2

q_2 d: $\{3\}$, si 3 urmeaza lui 2

q_3 I: $\{1, 2\}$, dar numai 2 urmeaza lui 3 -

q_3 d: $\{3\}$, si 3 urmeaza lui 3 -

deci $q_0 I \rightarrow q_1$

deci $q_1 d$ nu face parte din

deci $q_1 I \rightarrow q_2$

deci $q_1 d \rightarrow q_3$

deci $q_2 I \rightarrow q_2$

deci $q_2 d \rightarrow q_3$

deci $q_3 I \rightarrow q_2$

deci $q_3 d \rightarrow q_3$

Stari finale: q_1 , q_2 si q_3

Conversie expresie regulata - automat finit determinist - continuare

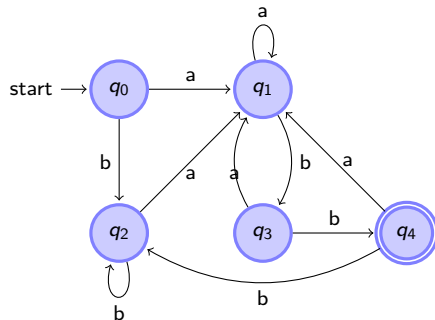
$(a + b) * abb$ becomes $0(1 + 2)^*345$

	a	b		a	follow	b	follow
q_0	q_1	q_2	0	1,3	1,3	2,4,5	2
q_1	q_1	q_3	1,3	1,3	1,3	2,4,5	2,4
q_2	q_1	q_2	2	1,3	1,3	2,4,5	2
q_3	q_1	q_4	2,4	1,3	1,3	2,4,5	2,5
q_4	q_1	q_2	2,5	1,3	1,3	2,4,5	2

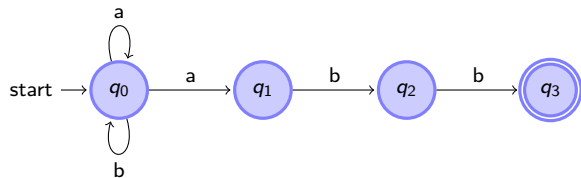
Test online: [dfa_toolbox](#), [dfa_nfa_pda](#)

DFA NFA: $(a+b)^*abb$

DFA



NFA



Alternativa

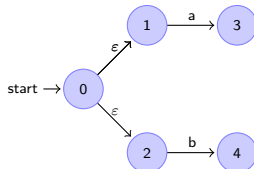
1. a



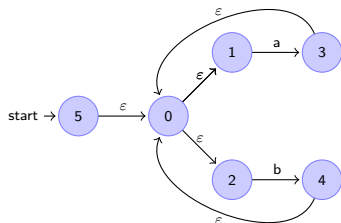
2. b



3. $a + b$



4. $(a + b)^*$



5. ab



Expresii regulate - backtracking recursiv vs automate finite deterministe

fast vs simple

Test regex in python vs lex for a string of length n

$$(a?)^n a \{n\}$$

Lex:

```
%%  
(a?){100} a{100}          printf("Easy with DFA");  
.
```

Python:

```
import re  
xx='a'*30  
print(re.match('(a?){30}a{30}',xx) )
```

test for a^{30} for python and 100 for lex

why? Concluzii

- ▶ Assuming that the **language** has been described by a **grammar**, we are interested in techniques for **automatically** generating a **recognizer from that grammar**. There are two reasons for this requirement:
 - ▶ It provides a guarantee that the language recognized by the compiler is identical to that defined by the grammar.
 - ▶ It simplifies the task of the compiler writer.

Expresiile regulate = un mod algebric de a descrie limbajele
Descriu limbajele regulate

$$a^n b^n$$

NU e un limbaj regulat: nu exista niciun automat finit care sa-l aiba ca limbaj

Dar limbajul *aaabbb*?

LEX - analiza lexicala

Caractere operator

" \ [] ^ - ? . * + | () \$ / { } % < >

Folosirea lor drept caractere text: precedate de \ sau intre "".

xyz" ++"

"xyz ++"

xyz \ + \ +

Expresii regulate in LEX

1. Clase de caractere $[a - z0 - 9 <>]$, $[-0 - 9]$, abc
2. Caracter arbitrar $.$
3. Element optional $ab?c$
4. Repetitii a^* , a^+ , $[a - z]^+$
5. Alternare $ab|cd$ $a(b|d)$
6. Doar la inceput/final de linie abc , $abc\$$
7. Context ab/cd (ab daca e urmat de cd)
8. Operator $\{ \}$: $a\{1, 5\}$, $a\{2, \}$

Analizor lexical controlat prin automat finit 3.6.2

Fie:

D [0-9]

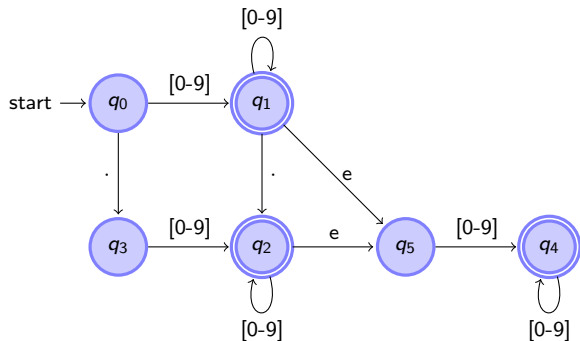
%%

{D}+

({D}*|{D}*\.{D}+|{D}+\.{D}*) (e{D}+)?

return ICON;

return FCON;



Tabel de tranzitie

Stare curenta	caracter examinat inainte (lookahead) caracter de intrare		actiune la acceptare
	.	0-9 e	
0	3	1 -	-
1	2	1 5	return ICON;
2	-	2 5	return FCON;
3	-	2 -	-
4	-	4 -	return FCON;
5	-	4 -	-

Algoritmul utilizat de LEX (greedy) 3.6.3

```
stare_curenta = 0;
stare_acceptoare_vazuta_anterior = nimic_vazut;
if (caracter lookahead este end_of_input)
    return 0;
while(caracter lookahead nu este end_of_input){
    if (exista tranzitie din starea curenta cu caracterul
        lookahead curent){
        stare_curenta = acea stare;
        avanseaza in intrare;
        if (starea curenta este o stare acceptoare){
            memoreaza pozitia curenta in intrare
            si actiunea asociata starii curente;
        }
    }
    else{
        if (nu a fost vazuta nicio stare acceptoare){
            exista o eroare:
                descarca lexemul curent si caracterul de intrare
                curent;
                stare_curenta = 0;
        }
        else {
            salveaza intrarea in pozitia in care se afla cand a
            vazut ultima stare acceptoare; realizeaza actiunea
            asociate acelei stari acceptoare;
        }
    }
}
```

Exemplu: $stare_urmatoare = array[stare_curenta][intrare]$

Stare curenta	caracter examinat inainte (lookahead) caracter de intrare		actiune la acceptare
	.	0-9 e	
0	3	1 -	-
1	2	1 5	return ICON;
2	-	2 5	return FCON;
3	-	2 -	-
4	-	4 -	return FCON;
5	-	4 -	-

Stare	Intrare	Ultima stare acceptoare	actiune	pozitie in intrare
0	1.2e4			
1	.2e4	1	ICON	.
2	2e4	2	FCON	2
2	e4	2	FCON	e
5	4			
4		4	FCON	

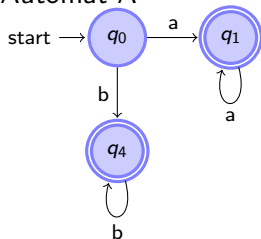
Exemplu: $stare_urmatoare = array[stare_curenta][intrare]$

Stare curenta	caracter examinat inainte (lookahead) caracter de intrare		actiune la acceptare
	.	0-9 e	
0	3	1 -	-
1	2	1 5	return ICON;
2	-	2 5	return FCON;
3	-	2 -	-
4	-	4 -	return FCON;
5	-	4 -	-

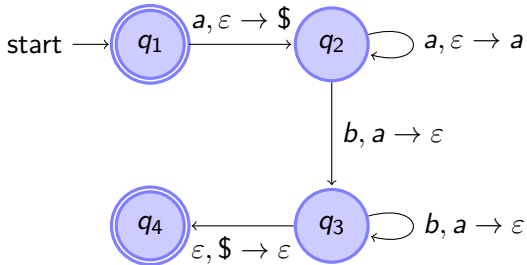
Stare	Intrare	Ultima stare acceptoare	actiune	pozitie in intrare
0	1.2e			
1	.2e	1	ICON	.
2	2e	2	FCON	2
2	e	2	FCON	e
5				
	e			

Gramatici regulate 2. Expresii regulate

Automat A



Automat B



Context free grammars - Gramatici independente de context. Automate stiva

March 29, 2021

$\{a^n b^n\}$ e regulat?

- ▶ Pt a fi limbaj regulat, ar trebui sa existe un **automat finit** care sa-l recunoasca.
- ▶ ar trebui sa tina minte cati a a citit, dar n nu este limitat
- ▶ dupa ce a citit a^m ar trebui sa fie intr-o stare ce specifica o multime de simboluri nonterminale din care sa fie derivate exact b^m . $\Rightarrow$ pt fiecare m ar trebui sa fie o stare distinctica
- ▶ deci automatul ar trebui sa aiba evidenta unui numar nelimitat de posibilitati

acest lucru nu se poate face cu un numar finit de stari

- ▶ dar, nu tot ce pare a avea nevoie de memorie nelimitata, chiar are:
 - ▶ $C = \{w \mid w \text{ are un numar egal de } 0 \text{ si } 1\}$
 - ▶ $D = \{w \mid w \text{ are un numar egal de aparitii } 01 \text{ si } 10\}$

$\{a^n b^n\}$ e regulat?

- ▶ Pt a fi limbaj regulat, ar trebui sa existe un **automat finit** care sa-l recunoasca.
- ▶ ar trebui sa tina minte cati a a citit, dar n nu este limitat
- ▶ dupa ce a citit a^m ar trebui sa fie intr-o stare ce specifica o multime de simboluri nonterminale din care sa fie derivate exact b^m . $\Rightarrow$ pt fiecare m ar trebui sa fie o stare distinctica
- ▶ deci automatul ar trebui sa aiba evidenta unui numar nelimitat de posibilitati

acest lucru nu se poate face cu un numar finit de stari

- ▶ dar, nu tot ce pare a avea nevoie de memorie nelimitata, chiar are:
 - ▶ $C = \{w \mid w \text{ are un numar egal de } 0 \text{ si } 1\}$
 - ▶ $D = \{w \mid w \text{ are un numar egal de aparitii } 01 \text{ si } 10\}$

D este limbaj regulat

$D = \{w \mid w \text{ are un numar egal de aparitii } 01 \text{ si } 10 \text{ ca substringuri}\}$

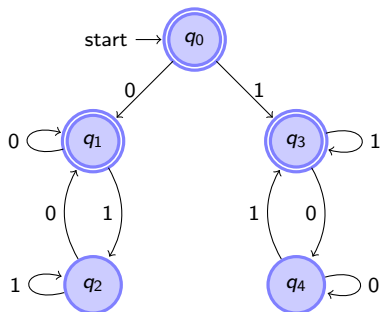
$$D = \{0, 1, \varepsilon$$

w daca incepe cu 0 se termina cu 0

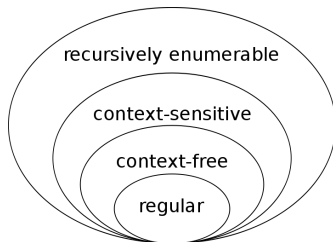
w daca incepe cu 1 se termina cu 1}

? 101, 1010, 0110

$$1 + 0 + \varepsilon + 0(0 + 1)^*0 + 1(0 + 1)^*1$$



Gramatici independente de context. Context-free grammars



- ▶ $G = (T, N, Z, P)$ e independenta de context daca
- ▶ fiecare productie are forma

$$X \rightarrow \chi, X \in N, \chi \in V^*$$

Un limbaj care e definit de o gramatica independenta de context este limbaj independent de context.

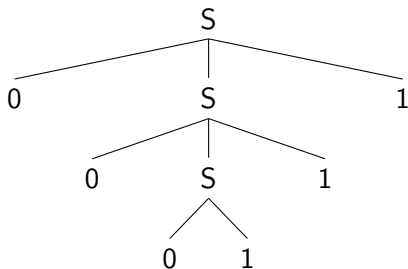
Exista CFL care nu sunt Regular languages

$0^n 1^n$

Fie $G = (\{0, 1\}, \{S\}, \{S \rightarrow 01|0S1\}, S)$

► $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$

Arbore de parsare pentru 000111:



Limbajul parantezelor

Fie $G = (\{(,)\}, \{S\}, \{S \rightarrow SS|(S)|()\}, S)$

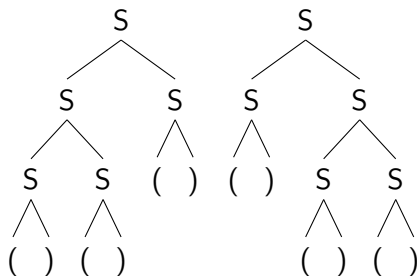
- ▶ left-most derivation: $S \Rightarrow \mathbf{SS} \Rightarrow (\mathbf{S})S \Rightarrow (())S \Rightarrow (())()$
Deci $S \Rightarrow^L (())()$
- ▶ right-most derivation: $S \Rightarrow \mathbf{SS} \Rightarrow \mathbf{S}() \Rightarrow (S)() \Rightarrow (())()$
Deci $S \Rightarrow^R (())()$

Gramatica ambigua - reamintire

O gramatică este ambiguă dacă există un string în limbaj care este parsat în două arbori de derivare.

Pentru $G = (\{(,)\}, \{S\}, \{S \rightarrow SS|(S)|()\}, S)$

Derivarea $()()()$:



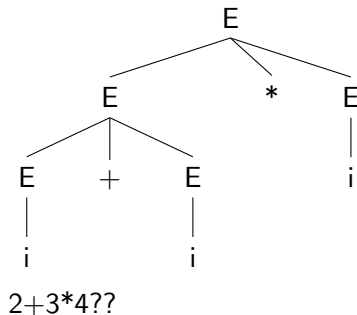
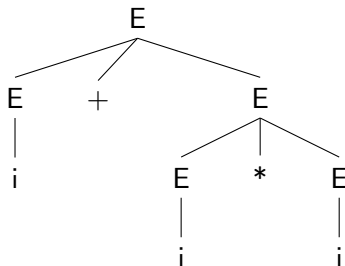
Ambiguitate

Fie $G_4 = (\{+, *, i\}, \{E\}, E, P)$ Doua derivari distincte stanga, doua derivari distincte dreapta

► $E \rightarrow E + E$

► $E \rightarrow E * E$

► $E \rightarrow i$



CFG pentru Engleza

- ▶ $T = \{eats, saw, man, woman, telescope, the, with, at\}$
- ▶ $N = \{S, NP, VP, PP, DT, Vi, Vt, NN, IN\}$

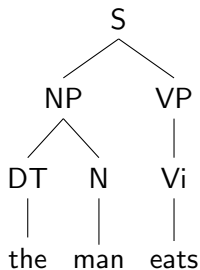
▶ P=	S	→	NP VP	Vi	→	eats
	VP	→	Vi	Vt	→	saw
	VP	→	Vt NP	N	→	man
	VP	→	VP PP	N	→	woman
	NP	→	DT N	N	→	telescope
	NP	→	NP PP	DT	→	the
	PP	→	IN NP	IN	→	with
				IN	→	at

S = sentence, VP = verb phrase, NP = noun phrase, PP = prepositional phrase, DT = determiner, Vi = intransitive verb, Vt = transitive verb, N = noun, IN = preposition

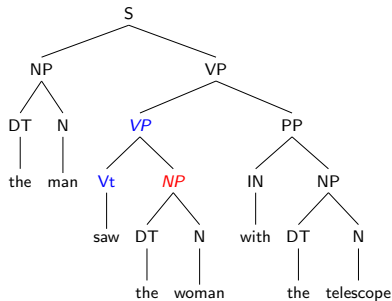
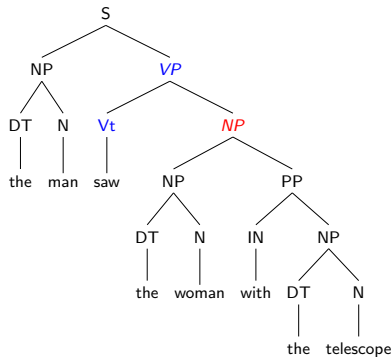
Arbore de derivare

Derivarea stanga:

$S \Rightarrow \mathbf{NP VP} \Rightarrow \mathbf{DT N VP} \Rightarrow \textit{the N VP} \Rightarrow \textit{the man VP}$
 $\Rightarrow \textit{the man Vi} \Rightarrow \textit{the man eats}$

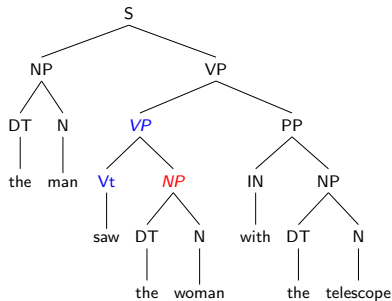
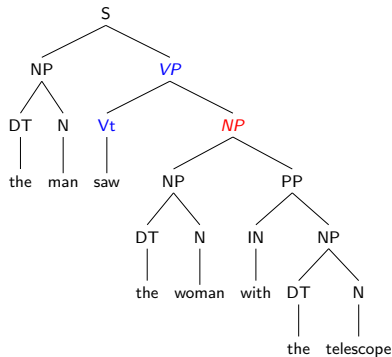


The man saw the woman with the telescope.



Ce a vazut "the man"?

The man saw the woman with the telescope.



Ce a vazut "the man"?

The telescope at the man saw the woman $\in? L(G)$

if then else grammar

► $T = \{if, then, else, E1, E2, S1, S2, S3\}$, $N = \{stmt, expr\}$

► $P =$

$stmt \rightarrow if\ expr\ then\ stmt$

$stmt \rightarrow if\ expr\ then\ stmt\ else\ stmt$

$stmt \rightarrow S1 \mid S2 \mid S3$

$expr \rightarrow E1 \mid E2$

if then else grammar

► $T = \{if, then, else, E1, E2, S1, S2, S3\}$, $N = \{stmt, expr\}$

► $P =$

stmt $\rightarrow$ if *expr* then *stmt*

stmt $\rightarrow$ if *expr* then *stmt* else *stmt*

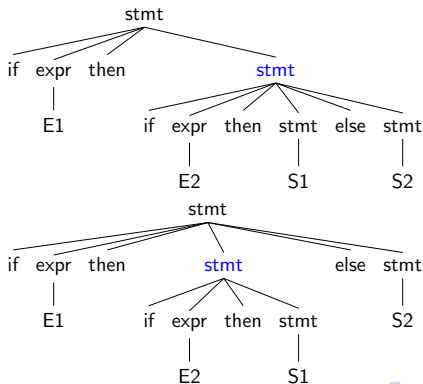
stmt $\rightarrow$ S1 | S2 | S3

expr $\rightarrow$ E1 | E2

if E1	
then	if E2
	then S1
	else S2

sau?

if E1	
then	if E2
	then S1
else S2	



if then else - rezolvare ambiguitate

- ▶ $T = \{if, then, else, E1, E2, S1, S2, S3\},$
 $N = \{stmt, matched_stmt, unmatched_stmt, expr\}$
- ▶ $P =$

$stmt$	$\rightarrow$	m_stmt
	$ $	um_stmt
m_stmt	$\rightarrow$	if $expr$ then m_stmt else um_stmt
	$ $	$stmt1$
um_stmt	$\rightarrow$	if $expr$ then $stmt$
	$ $	if $expr$ then m_stmt else um_stmt
$stmt1$	$\rightarrow$	$S1 \mid S2 \mid S3$
$expr$	$\rightarrow$	$E1 \mid E2$

Intre un *then* si un *else* e permis doar *matched_stmt*.
 $m_stmt = matched_stmt$ (if cu ambele *then* si *else*),
 $um_stmt = unmatched_stmt$

Exemple de gramatici: Liste de *stmt*

► Recursivitate dreapta

stmt_list → *stmt stmt_list*
 | *stmt*

stmt → A|B|C

Arbore pt A B C?

► Recursivitate stanga

stmt_list → *stmt_list stmt*
 | *stmt*

stmt → A|B|C

Exemple de gramatici: Liste de elemente cu separator/marcaj de final

- ▶ Separator între elemente

$$\begin{array}{lcl} \text{elem_list} & \rightarrow & \text{elem_list } ',' \text{ elem} \\ & | & \text{elem} \\ \text{elem} & \rightarrow & A|B|C \end{array}$$

Arbore pt A, B, C?

- ▶ Semn de punctuație la final

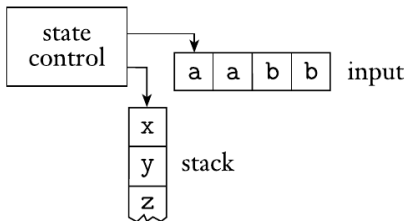
$$\begin{array}{lcl} \text{stmt_list} & \rightarrow & \text{stmt_list stmt } ';;' \\ & | & \text{stmt } ';;' \\ \text{stmt} & \rightarrow & A|B|C \end{array}$$

Arbore pt A; B; C;?

Letia and Chifu. 2.3, 2.3.1 Sipser - 2.1,2.2

Automat stiva. Push down automaton (PDA)

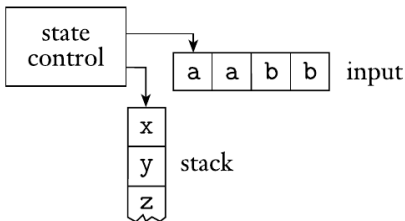
- ▶ Niciun automat finit nu poate fi construit pt a recunoaste $a^n b^n$ sau limbajul parantezelor - structuri imbricate
- ▶ Se creste puterea automatelor finite prin adaugarea unei stive drept structura aditionala de memorie



- ▶ Daca gramaticile regulate sunt o subclasa a gramaticilor independente de context, de ce se dezvoltă metode specifice gramaticilor regulate si nu se aplica pt acestea cele de la gramaticile independente?

Automat stiva. Push down automaton (PDA)

- ▶ Niciun automat finit nu poate fi construit pt a recunoaste $a^n b^n$ sau limbajul parantezelor - structuri imbricate
- ▶ Se creste puterea automatelor finite prin adaugarea unei stive drept structura aditionala de memorie



- ▶ Daca gramaticile regulate sunt o subclasa a gramaticilor independente de context, de ce se dezvoltă metode specifice gramaticilor regulate si nu se aplica pt acestea cele de la gramaticile independente?
- ▶ Datorita complexitatii analizei gramaticilor independente de context: gramaticile regulate sunt mai simplu de analizat

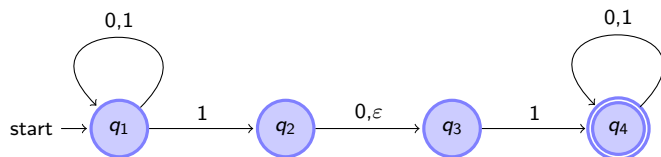
Idee: Push down automata: 00001111

1. citește simboluri de la intrare
2. la fiecare 0 citit, împinge-l pe stivă
3. la fiecare 1 citit, scoate de pe stivă un 0
4. dacă citirea stringului se termină când stivă se golește, acceptă stringul. Dacă stivă devine goală când mai sunt 1 de citit sau s-a terminat sirul și în stivă încă mai sunt 0-uri, respinge stringul

NFA- reamintire

Un automat finit nedeterminist este $(Q, \Sigma, \delta, q_0, F)$, unde:

1. Q este setul de stari
2. Σ un alfabet finit de intrare
3. $\delta : Q \times \Sigma_{\epsilon} \rightarrow P(Q)$ este o functie de tranzitie
4. $q_0 \in Q$ este starea de start
5. $F \subseteq Q$ setul de stari finale



$$\delta(q_1, 1) = \{q_1, q_2\}$$

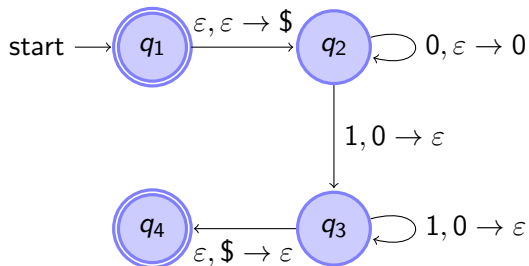
Definitie formală 1 a automatului stivă (Sipser)

Un automat stivă este $(Q, \Sigma, \Gamma, \delta, q_0, F)$, unde Q, Σ, Γ, F sunt seturi finite:

1. Q este setul de stări
2. Σ un alfabet de intrare
3. Γ este alfabetul stivei
4. $\delta : Q \times \Sigma_{\varepsilon} \times \Gamma_{\varepsilon} \rightarrow P(Q \times \Gamma_{\varepsilon})$ este o funcție de tranziție
5. $q_0 \in Q$ este starea de start
6. $F \subseteq Q$ setul de stări finale

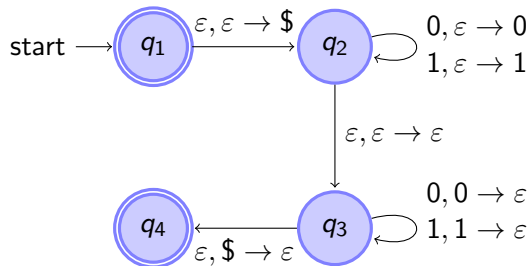
PDA for 0^n1^n

Let $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, F)$



PDA for ?

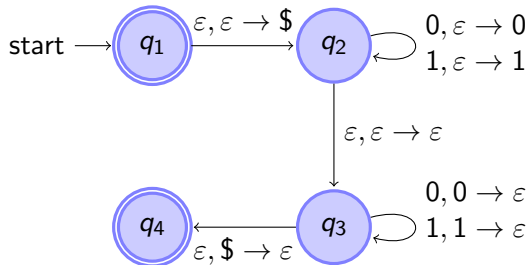
File $M_1 = (Q, \{0, 1\}, \{\$, 0, 1\}, \delta, q_1, \{q_1, q_4\})$



PDA for $\{ww^R \mid w \in \{0,1\}^*\}$

$w^R = w$ scris invers

Fie $M_1 = (Q, \{0,1\}, \{\$,0,1\}, \delta, q_1, \{q_1, q_4\})$



la fiecare pas, ghiceste daca a ajuns la mijlocul stringului sau nu

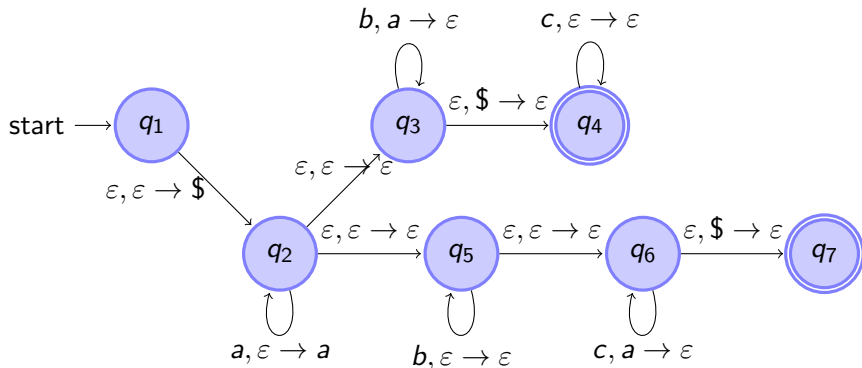
Gramatica palindrom par

► Palindrom: (T, N, P, A) , $P =$

$$\{A \rightarrow 0A0 | 1A1 \\ A \rightarrow \varepsilon\}$$

PDA for $\{a^i b^j c^k \mid i, j, k \geq 0, i = j \text{ sau } i = k\}$

Fie $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, F)$



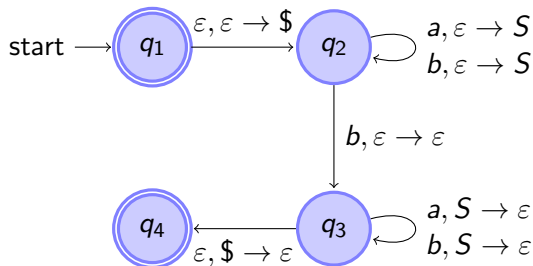
ghiceste daca e acelasi numar de a si b sau a si c

$$\{vbw \mid v, w \in \{a, b\}^*, |v| = |w|\}$$

$$M = (Q, \{a, b\}, \{\$, S\}, \delta, q_1, \{q_4\})$$

$$\{vbw \mid v, w \in \{a, b\}^*, |v| = |w|\}$$

$$M = (Q, \{a, b\}, \{\$, S\}, \delta, q_1, \{q_4\})$$



Exemplu Gramatica independenta de context

- Palindrom: (T, N, P, A) , $P =$

$$\begin{aligned} &\{A \rightarrow 0A0|1A1 \\ &A \rightarrow \varepsilon\} \end{aligned}$$

- Acelasi numar de 0 si 1: $(\{0, 1\}, \{A\}, P, A)$, $P =$

$$\begin{aligned} &\{A \rightarrow 0A1A|1A0A \\ &A \rightarrow \varepsilon\} \end{aligned}$$

Automat finit - reamintire

Automat finit (finite automaton, finite state acceptor):

$$A = (T, Q, R, q_0, F)$$

- ▶ Q set nevid - setul starilor interne
- ▶ $(T \cup Q, R)$ sistem de rescriere; $T \cap Q = \emptyset$
- ▶ $q_0 \in Q$ - starea initiala
- ▶ $F \subseteq Q$ - stari finale
- ▶ fiecare element din R are forma $qt \rightarrow q', q, q' \in Q, t \in T$

$$L(A) = \{\tau \in T^* \mid q_0\tau \Rightarrow^* q, q \in F\}$$

Automat stiva - definitie sistem de rescriere

Automat stiva

$$A = (T, Q, R, q_0, F, S, s_0)$$

, unde:

- ▶ Q set nevid - setul starilor interne
- ▶ $(T \cup Q \cup S, R)$ sistem de rescriere; $T \cap Q = \emptyset$
- ▶ $q_0 \in Q$ - starea initiala
- ▶ $s_0 \in S \cup \{\varepsilon\}$ - simboluri stiva, s_0 continutul initial al stivei
- ▶ $F \subseteq Q$ - stari finale
- ▶ fiecare element din R are forma $\sigma q t \tau \rightarrow \sigma' q' \tau$,
 $\sigma, \sigma' \in S^*$, $q, q' \in Q$, $t \in T \cup \varepsilon$, $\tau \in T^*$

Daca automatul e la configuratia $s_1 \dots s_n q \tau$ intr-o derivare, automatul e in starea q , τ este partea necitita din input, $s_1, \dots, s_n$ este continutul pe stiva, s_n in varf.

Limbaj acceptat

Daca automatul e la configuratia $s_1...s_nq\tau$ intr-o derivare, automatul e in starea q , τ este partea necitita din input, $s_1, \dots, s_n$ este continutul pe stiva, s_n in varf.

$$L(A) = \{\tau | s_0q_0\tau\# \Rightarrow^* q\#, q \in F, \tau \in T^*\}$$

$0^n 1^n$

$M_1 = (\{0, 1\}, \{q_2, q_3\}, R, q_2, \{q_3\}, \{0, 1\}, \varepsilon), R = \{$

1. $\varepsilon q_2 0 \rightarrow 0 q_2$
2. $0 q_2 1 \rightarrow \varepsilon q_3$
3. $0 q_3 1 \rightarrow \varepsilon q_3\}$

$??\varepsilon q_2 0011 \Rightarrow^* q_3$

Pe stiva pot fi alte simboluri decat cele din alfabetul de intrare.

CFG - PDA

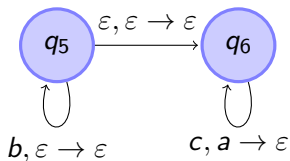
Pentru fiecare gramatica independenta de context G exista un automat stiva A a.i. $L(A)=L(G)$.

Care din urmatoarele afirmatii sunt adevarate pt gramatica

$G = (\{(,)\}, \{S\}, \{S \rightarrow SS|(S)|()\}, S)$

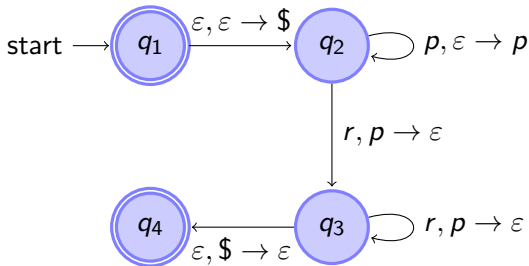
1. $S \Rightarrow^L ()S$
2. $S \Rightarrow^R ()S$
3. $S \Rightarrow^* (())(S)$
4. $()$ nu e generat de gramatica

Pentru fragmentul dintr-un automat stiva A , care afirmatii sunt adevarate?



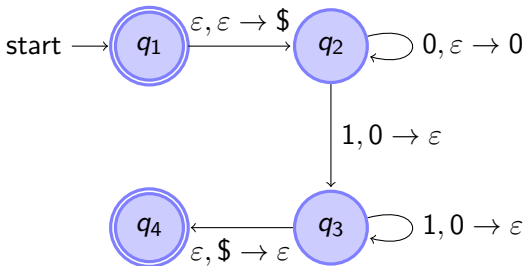
1. daca la intrare este c , starea este q_6 , in varful stivei este a ; A va ramane in q_6
2. din q_5 in q_6 A trece fara a citi niciun simbol de la intrare si fara a modifica stiva
3. tranzitia din q_5 in q_6 are loc doar daca se poate scoate din stiva a si la intrare nu conteaza
4. daca la intrare este b , starea este q_5 ; automatul va ramane cu siguranta in q_5

Pentru automatul din figura, ce este posibil sa avem in stiva?



1. oricat de multi de p , dar doar p
2. oricat de multi de r , dar doar r
3. orice combinatie de p si r
4. secvente de pr

Pentru automatul din figura, daca in stiva la un moment dat avem 0, care afirmatie e adevarata



1. daca A este in q3 si la intrare este 0, ramane in q3
2. stringul acceptat incepe cu 0 si se termina cu 1
3. va fi acceptat stringul care incepe cu 1 si se termin cu 0
4. daca A este in q2 si la intrare este 0, trece in q3

Analiza descendenta - prima parte

April 5, 2021

Automat finit - reamintire

Automat finit (finite automaton, finite state acceptor):

$$A = (T, Q, R, q_0, F)$$

- ▶ Q set nevid - setul starilor interne
- ▶ $(T \cup Q, R)$ sistem de rescriere; $T \cap Q = \emptyset$
- ▶ $q_0 \in Q$ - starea initiala
- ▶ $F \subseteq Q$ - stari finale
- ▶ fiecare element din R are forma $qt \rightarrow q', q, q' \in Q, t \in T$

$$L(A) = \{\tau \in T^* \mid q_0\tau \Rightarrow^* q, q \in F\}$$

Automat stiva - definitie sistem de rescriere -reamintire

Automat stiva

$$A = (T, Q, R, q_0, F, S, s_0)$$

, unde:

- ▶ Q set nevid - setul starilor interne
- ▶ $(T \cup Q \cup S, R)$ sistem de rescriere; $T \cap Q = \emptyset$
- ▶ $q_0 \in Q$ - starea initiala
- ▶ $s_0 \in S \cup \{\varepsilon\}$ - simboluri stiva, s_0 continutul initial al stivei
- ▶ $F \subseteq Q$ - stari finale
- ▶ fiecare element din R are forma $\sigma q t \tau \rightarrow \sigma' q' \tau$,
 $\sigma, \sigma' \in S^*$, $q, q' \in Q$, $t \in T \cup \varepsilon$, $\tau \in T^*$

Daca automatul e la configuratia $s_1 \dots s_n q \tau$ intr-o derivare, automatul e in starea q , τ este partea necitita din input, $s_1, \dots, s_n$ este continutul pe stiva, s_n in varf.

Limbaj acceptat

Daca automatul e la configuratia $s_1 \dots s_n q \tau$ intr-o derivare, automatul e in starea q , τ este partea necitita din input, $s_1, \dots, s_n$ este continutul pe stiva, s_n in varf.

$$L(A) = \{\tau \mid s_0 q_0 \tau \Rightarrow^* q, q \in F, \tau \in T^*\}$$

CFG - PDA

Pentru fiecare gramatica independenta de context G exista un automat stiva A a.i. $L(A)=L(G)$.

Parsing

- ▶ Rolul parsarii: reconstruirea derivarii prin care o CFG poate genera un input string dat.
- ▶ Echivalent cu construirea arborelui de parsare care reprezintă derivarea
- ▶ Directii:
 - ▶ Top-Down - constructia incepe de la radacina; derivarea stanga
 - ▶ Bottom-up - constructia incepe de la frunze; mai greu de construit manual, dar pot fi generate

Rezumat

Recursive descent parsing

Predictive parsing

Structuri ajutatoare: FIRST, FOLLOW

Definire Gramatici LL(k)

Algoritmul LL(K)

Exemplu aplicare LL(k)

Exemplu aplicare LL(3)

1. Recursive descent - top down parse

Arborele de parsare e construit:

- ▶ de la simbolul de start
- ▶ de la stanga la dreapta
- ▶ se incearca regulile in ordinea in care apar
- ▶ revenire si incercare alternative

$$E \rightarrow T | T + E$$

$$T \rightarrow int | int * T | (E)$$

Parse tree pt: $(\quad int \quad)$
 $\uparrow$

E
 |
 T

$$E \rightarrow T \mid T + E$$

$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

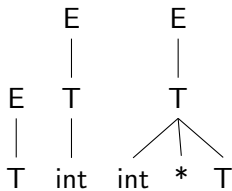
Parse tree pt: (int)
 $\uparrow$



$$E \rightarrow T \mid T + E$$

$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

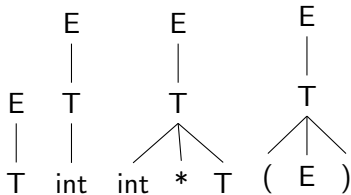
Parse tree pt: (int)
 $\uparrow$



$$E \rightarrow T \mid T + E$$

$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

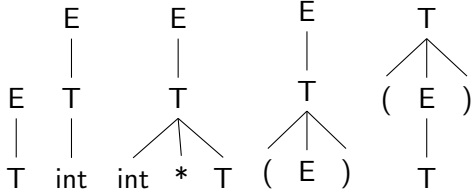
Parse tree pt: (int)
 $\uparrow$



$$E \rightarrow T \mid T + E$$

$$T \rightarrow int \mid int * T \mid (E)$$

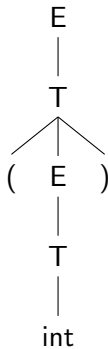
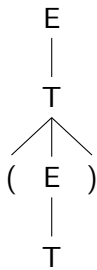
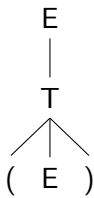
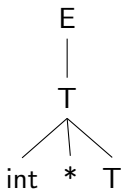
Parse tree pt: (int)
 ↑



$$E \rightarrow T \mid T + E$$

$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

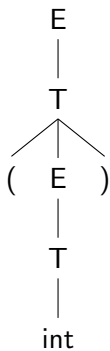
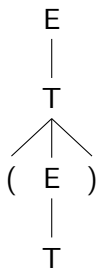
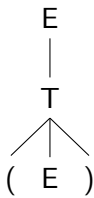
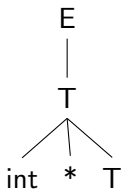
Parse tree pt: (int)
 $\uparrow$



$$E \rightarrow T \mid T + E$$

$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

Parse tree pt: (int)
 $\uparrow$



Recursive-descent parser - neștiind care dintre producțiile alternative pt un nonterminal trebuie aplicată, există posibilitatea de eșec

- Predictive parser: dat fiind sirul de intrare a (primul din sirul ramas) și nonterminalul A care trebuie expandat, am putea determina care producție alternativă e cea care derivatează stringul ramas după a

idee: alternativă corectă trebuie detectată uitându-ne înainte la k simboluri din stringul care trebuie derivate: $LL(1)$ și $LR(1)$

2. Parser predictiv¹

Fie $G = (T, N, P, Z)$ o CFG si automatul stiva

$A = (T, \{q\}, R, q, \{q\}, V, Z)$ cu $V = T \cup N$ si R :

(alfabet, stari, productii, stare initiala, stari finale, alfabet stiva, continut initial stiva)

$$\{tgt \rightarrow q | t \in T\} \cup \\ \{Xq \rightarrow x_n \dots x_1 q | X \rightarrow x_1 x_2 \dots x_n \in P, n \geq 0, X \in N, X_i \in V\}$$

Automatul accepta un sir din $L(G)$ prin

- ▶ construirea unei **derivari cea mai din stanga** a aceluia sir si
- ▶ compararea simbolurilor generate (de la stanga la dreapta) si incarcate pe stiva cu simbolurile care apar in sir.

¹Letia& Chifu 4.2

exemplu 1

Fie $G_1 = (T, N, S, P)$

- ▶ $T = \{+, (,), i\}$, $N = \{S, F\}$
- ▶ cu productiile P
 - ▶ $S \rightarrow F$
 - ▶ $S \rightarrow (S + F)$
 - ▶ $F \rightarrow i$

Care e automatul pentru analiza descendenta?

Care e derivarea stanga pentru $(i + i)$?

Automatul accepta $(i + i)$?

exemplu 2

Fie $G_1 = (T, N, E, P)$

- ▶ $T = \{+, *, (,), i\}$, $N = \{E, T, F\}$
- ▶ cu productiile P
 - ▶ $(1, 2) E \rightarrow T \mid E + T$
 - ▶ $(3, 4) T \rightarrow F \mid T * F$
 - ▶ $(5, 6) F \rightarrow i \mid (E)$

Automatul stiva construit pentru analiza descendenta:

- ▶ $T = \{+, *, (,), i\}$, $Q = \{q\}$,
 $q_0 = q$, $F = \{q\}$, $S = \{+, -, *, (,), i, E, T, F\}$, $s_0 = E$
- ▶ cu productiile R
 1. $Eq \rightarrow Tq$, $Eq \rightarrow T + Eq$,
 2. $Tq \rightarrow Fq$, $Tq \rightarrow F * Tq$,
 3. $Fq \rightarrow iq$, $Fq \rightarrow)E(q$,
 4. $+q+ \rightarrow q$, $*q* \rightarrow q$, $(q(\rightarrow q,)q) \rightarrow q$, $iqi \rightarrow q\}$

Derivarea gasita: $i+i*i$

stiva	stare	intrare	derivarea cea mai din stanga
E	q	$i + i * i$	E
T+E	q	$i + i * i$	E+T
T+T	q	$i + i * i$	T+T
T+F	q	$i + i * i$	F+T
T+i	q	$i + i * i$	i+T
T+	q	$+ i * i$	
T	q	$i * i$	
F*T	q	$i * i$	i+T*F
F*F	q	$i * i$	i+F*F
F*i	q	$i * i$	i+i*F
F*	q	$* i$	
F	q	i	
i	q	i	i+i*i
	q		

Exemplul 3

Fie $G_1 = (T, N, E, P)$

- ▶ $T = \{a, b, c\}$, $N = \{Z, X, Y\}$
- ▶ cu productiile P
 - ▶ (1) $Z \rightarrow X$
 - ▶ (2,3) $X \rightarrow Y|bYa$
 - ▶ (4,5) $Y \rightarrow c|ca$

Automatul $(\{a, b, c\}, \{q\}, R, q, \{q\}, \{a, b, c, X, Y, Z\}, Z)$:

- ▶ $aq a \rightarrow q$
- ▶ $bq b \rightarrow q$
- ▶ $cq c \rightarrow q$
- ▶ $Zq \rightarrow Xq$
- ▶ $Xq \rightarrow Yq$
- ▶ $Xq \rightarrow aYbq$
- ▶ $Yq \rightarrow cq$
- ▶ $Yq \rightarrow acq$

bcaa?

3. No backtracking

Analiza descendenta sau predictiva - traseaza derivarea de la simbolul de start la propozitie, prezicand simbolurile care trebuie sa fie prezente.

- ▶ stiva precizeaza sirul din V^* utilizat pentru derivarea restului sirului de la intrare
- ▶ **automat stiva determinist**: pentru gramatici $LL(k)$

Asumptii si structuri ajutatoare: CFG

Presupunem ca CFG (T, N, P, Z) contin

- ▶ $Z \rightarrow S$ singura in care apare Z
- ▶ fiecare propozitie se termina cu $\#$ - indica finalul propozitiei
- ▶ productia i are forma

$$X_i \rightarrow \chi_i, \text{ unde } \chi_i = x_{i,1}x_{i,2}\dots x_{i,m}$$

- ▶ $k : \omega$ primele $\min(k, |\omega| + 1)$ simboluri din $\omega\#$

$$k : \omega = \begin{cases} \omega\#, & \text{daca } |\omega| < k \\ \alpha, & \text{daca } \omega = \alpha\gamma \text{ si } |\alpha| = k \end{cases}$$

- ▶ $FIRST_k(\omega)$ setul tuturor capetelor $k : \omega$ terminale ale sirurilor derivabile din ω

$$FIRST_k(\omega) = \{\tau | \exists \nu \in T^* \text{ a.i. } \omega \Rightarrow^* \nu, \tau = k : \nu\}$$

- ▶ $EFF_k(\omega)$ (ε - free first, primul fara ε) - toate sirurile din $FIRST_k(\omega)$ pentru care nu s-a aplicat nicio productie ε in ultimul pas din derivarea cea mai din dreapta

$$EFF_k(\omega) = \{\tau \in FIRST_k(\omega) | \nexists A \in N, \nu \in T^* \text{ a.i. } \omega \Rightarrow^R A\tau\nu \Rightarrow \tau\nu\}$$

- ▶ $FOLLOW(\omega)$ capete k terminale care ar putea urma lui ω ;
 $FOLLOW_k(Z) = \{\#\}$

$$FOLLOW_k(\omega) = \{\tau | \exists \nu \in V^* \text{ a.i. } Z \Rightarrow^* \mu\omega\nu, \tau \in FIRST_k(\nu)\}$$

Exemplu de valori FIRST, FOLLOW pt $k = 1$

- ▶ $T = \{id, *, +, (,)\}$, $N = \{E, E', T, T', F\}$
- ▶ cu productiile P
 - ▶ $Z \rightarrow E$
 - ▶ $E \rightarrow TE'$
 - ▶ $E' \rightarrow +TE' | \varepsilon$
 - ▶ $T \rightarrow FT'$
 - ▶ $T' \rightarrow *FT' | \varepsilon$
 - ▶ $F \rightarrow (E) | id$

simbol	$FIRST_1(X)$	$FOLLOW_1(X)$
E	$\{(, id\}$	$\{), \#\}$
E'	$\{+, \varepsilon\}$	$\{), \#\}$
T	$\{(, id\}$	$\{+, \#,)\}$
T'	$\{*, \varepsilon\}$	$\{+, \#,)\}$
F	$\{(, id\}$	$\{*, +, \#,)\}$

Exemplu

$$\begin{aligned} E &\Rightarrow TE' \Rightarrow FT'E' \Rightarrow (E)T'E' \Rightarrow^+ (id) * FT'E' \Rightarrow \\ &(id) * F * T' + TE' \Rightarrow (id) * id * id + id \end{aligned}$$

Exemplu de valori FIRST, FOLLOW pt $k = 1$

- ▶ $T = \{id, *, +, (,)\}$, $N = \{E, E', T, T', F\}$
- ▶ cu productiile P
 - ▶ $Z \rightarrow E$
 - ▶ $E \rightarrow TE'$
 - ▶ $E' \rightarrow +TE' | \varepsilon$
 - ▶ $T \rightarrow FT'$
 - ▶ $T' \rightarrow *FT' | \varepsilon$
 - ▶ $F \rightarrow (E) | id$

simbol	$FIRST_1(X)$	$FOLLOW_1(X)$
E	$\{(, id\}$	$\{), \#\}$
E'	$\{+, \varepsilon\}$	$\{), \#\}$
T	$\{(, id\}$	$\{+, \#,)\}$
T'	$\{*, \varepsilon\}$	$\{+, \#,)\}$
F	$\{(, id\}$	$\{*, +, \#,)\}$

Exemplu

$$\begin{aligned} E &\Rightarrow TE' \Rightarrow FT'E' \Rightarrow (E)T'E' \Rightarrow^+ (id) * FT'E' \Rightarrow \\ &(id) * F * T' + TE' \Rightarrow (id) * id * id + id \end{aligned}$$

Gramatici $LL(k)$

O gramatică independentă de context $G = (T, N, P, Z)$ este $LL(k)$ pentru un $k \geq 0$ dacă pentru derivări arbitrare

$$Z \Rightarrow^L \mu X \chi \Rightarrow \mu \nu \chi \Rightarrow^* \mu \gamma$$

$$Z \Rightarrow^L \mu X \chi \Rightarrow \mu \omega \chi \Rightarrow^* \mu \gamma'$$

$$\text{unde } \mu, \gamma, \gamma' \in T^*, \nu, \chi, \omega \in V^*, X \in N$$

avem următoarea proprietate: $k : \gamma = k : \gamma'$ implică $\nu = \omega$

Fie gramatica $G = (\{i, (, +,)\}, \{Z, E, F\}, P, Z)$ cu productiile

► $Z \rightarrow E$

► $E \rightarrow F$

► $E \rightarrow (E + F)$

► $F \rightarrow i$

$$\begin{aligned} Z \Rightarrow E \Rightarrow (E+F) &\Rightarrow (F+F) \Rightarrow^* (i+i) \\ Z \Rightarrow E \Rightarrow (E+F) &\Rightarrow ((E+F)+F) \Rightarrow^* ((i+i)+i) \end{aligned}$$

$$\begin{aligned} Z \Rightarrow^L \mu X \chi &\Rightarrow \mu \nu \chi \Rightarrow^* \mu \gamma \\ Z \Rightarrow^L \mu X \chi &\Rightarrow \mu \omega \chi \Rightarrow^* \mu \gamma' \end{aligned}$$

unde $\mu, \gamma, \gamma' \in T^*, \nu, \chi, \omega \in V^*, X \in N$

Exemplul 3 - reluat

Fie $G_1 = (T, N, E, P)$

▶ $T = \{a, b, c\}$, $N = \{Z, X, Y\}$

▶ cu productiile P

▶ (1) $Z \rightarrow X$

▶ (2,3) $X \rightarrow Y|bYa$

▶ (4,5) $Y \rightarrow c|ca$

1) $Z \Rightarrow X \Rightarrow Y \Rightarrow c$

2) $Z \Rightarrow X \Rightarrow Y \Rightarrow ca$

3) $Z \Rightarrow X \Rightarrow bYa \Rightarrow bca$

4) $Z \Rightarrow X \Rightarrow bYa \Rightarrow bcaa$

Pentru 1 si 2, s-au aplicat $Y \rightarrow c$ si $Y \rightarrow ca$ dar $1:c=1:ca$.

Pentru 3 si 4, s-au aplicat $Y \rightarrow c$ si $Y \rightarrow ca$ dar $1:ca=1:caa$, la fel si $2:ca = 2:caa$.

Situatie

$$[X_p \rightarrow \mu.\nu; \Omega]$$

$$\mu = x_{p,1} \dots x_{p,j}, \nu = x_{p,j+1} \dots x_{p,n_p},$$

$$|\mu| = j, |\nu| = n_p - j$$

Punctul nu face parte din vocabular. Marcheaza pozitia curenta a analizei in partea dreapta a productiei

ex: $q_7 = [X \rightarrow b.Ya; \#]$

Algoritmul LL(k)

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$

Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.

Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.

Algoritmul LL(k)

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$

Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.

Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.

2. fie $q = [X \rightarrow \mu.\nu; \Omega]$ un element al lui Q care inca nu a fost tratat

Algoritmul LL(k)

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$

Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.

Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.

2. fie $q = [X \rightarrow \mu.\nu; \Omega]$ un element al lui Q care inca nu a fost tratat
3. Daca $\nu = \varepsilon$ atunci se include $q\varepsilon \rightarrow \varepsilon$ in R .

Algoritmul LL(k)

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$

Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.

Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.

2. fie $q = [X \rightarrow \mu.\nu; \Omega]$ un element al lui Q care inca nu a fost tratat
3. Daca $\nu = \varepsilon$ atunci se include $q\varepsilon \rightarrow \varepsilon$ in R .
4. Daca $\nu = t\gamma$, $t \in T$ si $\gamma \in V^*$, fie $q' = [X \rightarrow \mu t.\gamma; \Omega]$.
Adauga q' in Q si $qt \rightarrow q'$ in R .

Algoritmul LL(k)

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$
Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.
Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.
2. fie $q = [X \rightarrow \mu.\nu; \Omega]$ un element al lui Q care inca nu a fost tratat
3. Daca $\nu = \varepsilon$ atunci se include $q\varepsilon \rightarrow \varepsilon$ in R .
4. Daca $\nu = t\gamma$, $t \in T$ si $\gamma \in V^*$, fie $q' = [X \rightarrow \mu t.\gamma; \Omega]$.
Aaduga q' in Q si $qt \rightarrow q'$ in R .
5. Daca $\nu = Y\gamma$, $Y \in N$ si $\gamma \in V^*$,
 - ▶ fie $q' = [X \rightarrow \mu Y.\gamma; \Omega]$
 - ▶ si $H = \{[Y \rightarrow \beta_i; FIRST_k(\gamma\Omega)] \mid Y \rightarrow \beta_i \in P\}$.
 - ▶ actualizeaza $Q = Q \cup \{q'\} \cup H$
 - ▶ si $R = R \cup \{q\tau_i \rightarrow q'h_i\tau_i \mid h_i \in H, \tau_i \in FIRST_k(\beta_i\gamma\Omega)\}$
6. daca toate starile din q au fost analizate, stop. Altfel continua cu 2.

Construirea automatului se termina datorita numarului finit de situatii.

Automatul rezultat este determinist daca si numai daca G este o gramatica $LL(k)$.

Exemplu de construire (gramatica e CFG dar si regulata)

- ▶ $Z \rightarrow S$
- ▶ $S \rightarrow 0S$
- ▶ $S \rightarrow 1$

k=1

		stari noi	tranzitii noi
			$q_0 = [Z \rightarrow .S; \{\#\}]$
q_0	5	$q' = [Z \rightarrow S.; \#] = q_1$ $H = \{[S \rightarrow .0S; \#] = q_2,$ $[S \rightarrow .1; \#] = q_3\}$	$q_0\tau? \rightarrow q_1h?\tau?$ $q_00 \rightarrow q_1q_20$ $q_01 \rightarrow q_1q_31$
q_1	3	-	$q_1\varepsilon \rightarrow \varepsilon$
q_2	4	$q' = [S \rightarrow 0.S; \#] = q_4$	$q_20 \rightarrow q_4$
q_3	4	$q' = [S \rightarrow 1.; \#] = q_5$	$q_31 \rightarrow q_5$
q_4	5	$q' = [S \rightarrow 0S.; \#] = q_6$ $H = \text{la fel cu analiza pt } q_0$	$?$ $q_40 \rightarrow q_6q_20$ $q_41 \rightarrow q_6q_31$
q_5	3	-	$q_5\varepsilon \rightarrow \varepsilon$
q_6	3	-	$q_6\varepsilon \rightarrow \varepsilon$

derivare

Care e derivarea pt $001\#$?

$$q_0 q_0 001\# \Rightarrow ?$$

Inercare cu $k = 1$; $Z \rightarrow S$, $S \rightarrow 0S1$, $S \rightarrow 01$

		stari noi	No	tranzitii noi
		$q_0 = [Z \rightarrow .S; \{\#\}]$		
q_0	5	$q' = [Z \rightarrow S.; \#] = q_1$ $H = \{[S \rightarrow .0S1; \#] = q_2,$ $[S \rightarrow .01; \#] = q_3\}$	1 2	$q_0 \tau? \rightarrow q_1 h? \tau?$ $\tau \in FIRST_1(0S1\#);$ $q_0 0 \rightarrow q_1 q_2 0$ $\tau \in FIRST_1(01\#);$ $q_0 0 \rightarrow q_1 q_3 0$
q_1	3	-	3	$q_1 \epsilon \rightarrow \epsilon$
q_2	4	$q' = [S \rightarrow 0.S1; \#] = q_4$	4	$q_2 0 \rightarrow q_4$
q_3	4	$q' = [S \rightarrow 0.1; \#] = q_5$	5	$q_3 0 \rightarrow q_5$
q_4	5	$q' = [S \rightarrow 0S.1; \#] = q_6$ $H = \{[S \rightarrow .0S1; FIRST_1(1\#)]$ $= q_7,$ $[S \rightarrow .01; FIRST_1(1\#)] = q_8\}$	6 7	$\tau \in FIRST_1(0S11\#);$ $q_4 0 \rightarrow q_6 q_7 0$ $\tau \in FIRST_1(011\#);$ $q_4 0 \rightarrow q_6 q_8 0$
q_5	4	$[S \rightarrow 01.; \#] = q_9$	8	$q_5 1 \rightarrow q_9$
q_6	4	$[S \rightarrow 0S1.; \#] = q_{10}$	9	$q_6 1 \rightarrow q_{10}$
q_9	...			
	3		10	$q_9 \epsilon \rightarrow \epsilon$
				

$k=1$: automat nedeterminist: $q_00 \rightarrow q_1q_20$ si $q_00 \rightarrow q_1q_30$

Derivare 01: cu un lookahead de 1 nu stim pe care productie sa o aplicam

$$q_0q_001\# \xRightarrow{1} q_0q_1q_201\# \xRightarrow{4} q_0q_1q_41\# \quad \text{deadend}$$

$$q_0q_001\# \xRightarrow{2} q_0q_1q_301\# \xRightarrow{5} q_0q_1q_51\# \xRightarrow{8} q_0q_1q_9\# \xRightarrow{10} q_0q_1\varepsilon\# \Rightarrow q_0\#$$

Inercare cu $k = 2$; $Z \rightarrow S$, $S \rightarrow 0S1$, $S \rightarrow 01$

		stari noi	No	tranzitii noi
		$q_0 = [Z \rightarrow .S; \{\#\}]$		
q_0	5	$q' = [Z \rightarrow S.; \#] = q_1$ $H = \{[S \rightarrow .0S1; \#] = q_2,$ $[S \rightarrow .01; \#] = q_3\}$	1 2	$q_0\tau? \rightarrow q_1h?\tau?$ $\tau \in FIRST_2(0S1\#);$ $q_000 \rightarrow q_1q_200$ $\tau \in FIRST_2(01\#);$ $q_001 \rightarrow q_1q_301$
q_1	3	-	3	$q_1\varepsilon \rightarrow \varepsilon$
q_2	4	$q' = [S \rightarrow 0.S1; \#] = q_4$	4	$q_20 \rightarrow q_4$
q_3	4	$q' = [S \rightarrow 0.1; \#] = q_5$	5	$q_30 \rightarrow q_5$
q_4	5	$q' = [S \rightarrow 0S.1; \#] = q_6$ $H = \{[S \rightarrow .0S1; FIRST_2(1\#)]$ $= q_7,$ $[S \rightarrow .01; FIRST_2(1\#)] = q_8\}$	6 7	$\tau \in FIRST_2(0S11\#);$ $q_400 \rightarrow q_6q_700$ $\tau \in FIRST_2(011\#);$ $q_401 \rightarrow q_6q_801$
q_5	4	$[S \rightarrow 01.; \#] = q_9$	8	$q_51 \rightarrow q_9$
q_6	4	$[S \rightarrow 0S1.; \#] = q_{10}$	9	$q_61 \rightarrow q_{10}$
q_7	4	$[S \rightarrow 0.S1; 1\#] = q_{11}$	10	$q_70 \rightarrow q_{11}$
q_8	4	$[S \rightarrow 0.1; 1\#] = q_{12}$	11	$q_80 \rightarrow q_{12}$
q_9	3		10	$q_9\varepsilon \rightarrow \varepsilon$
q_{10}	3		10	$q_{10}\varepsilon \rightarrow \varepsilon$
q_{11}	5	$q' = [S \rightarrow 0S.1; \{1\#}] = q_{13}$ $H = \{[S \rightarrow .0S1; FIRST_2(11\#)]$ $= q_{14},$ $[S \rightarrow .01; FIRST_2(11\#)] = q_{15}\}$	6 7	$\tau \in FIRST_2(0S111\#);$ $q_{11}00 \rightarrow q_{13}q_{14}00$ $\tau \in FIRST_2(0111\#);$ $q_{11}01 \rightarrow q_{13}q_{15}01$

Derivare $Z \Rightarrow 0011$

Stiva	Stare	Intrare	Derivarea cea mai din stanga
q_0	q_0	0011#	Z
$q_0 q_1$	q_2	0011#	S
$q_0 q_1$	q_4	011#	0S1
$q_0 q_1 q_6$	q_8	011#	0011
$q_0 q_1 q_6$	q_{12}	11#	
....			

Gramatica $LL(3)$

- ▶ $Z \rightarrow X$
- ▶ $X \rightarrow Y|bYa$
- ▶ $Y \rightarrow c|ca$

		stari noi		tranzitii noi
		$q_0 = [Z \rightarrow .X; \#]$		
q_0	5	$q' = [Z \rightarrow X.; \#] = q_1$ $H = \{[X \rightarrow .Y; \#] = q_2,$ $[X \rightarrow .bYa; \#] = q_3\}$	1 2 3	$\tau \in FIRST_3(Y\#) = \{c\#, ca\#$ $q_0c\# \rightarrow q_1q_2c\#$ $q_0ca\# \rightarrow q_1q_2ca\#$ $\tau \in FIRST_3(bYa) = \{bca\}$ $q_0bca \rightarrow q_1q_3bca$
q_2	5	$q' = [X \rightarrow Y.; \#] = q_4$ $H = \{[Y \rightarrow .c; \#] = q_5,$ $Y \rightarrow .ca; \#] = q_6\}$	4 5	$\tau \in FIRST_3(c\#) = \{c\#$ $q_2c\# \rightarrow q_4q_5c\#$ $q_2ca\# \rightarrow q_4q_6ca\#$

Gramatica $LL(3)$

- ▶ $Z \rightarrow X$
- ▶ $X \rightarrow Y|bYa$
- ▶ $Y \rightarrow c|ca$

$$q_0 = [Z \rightarrow \bullet X; \#]$$

$$q_1 = [Z \rightarrow X \bullet; \#]$$

$$q_2 = [X \rightarrow \bullet Y; \#]$$

$$q_3 = [X \rightarrow \bullet bYa; \#]$$

$$q_4 = [X \rightarrow Y \bullet; \#]$$

$$q_5 = [Y \rightarrow \bullet c; \#]$$

$$q_6 = [Y \rightarrow \bullet ca; \#]$$

$$q_7 = [X \rightarrow b \bullet Ya; \#]$$

$$q_8 = [Y \rightarrow c \bullet; \#]$$

$$q_9 = [Y \rightarrow c \bullet a; \#]$$

$$q_{10} = [X \rightarrow bY \bullet a; \#]$$

$$q_{11} = [Y \rightarrow \bullet c; a\#]$$

$$q_{12} = [Y \rightarrow \bullet ca; a\#]$$

$$q_{13} = [Y \rightarrow ca \bullet; \#]$$

$$q_{14} = [X \rightarrow bYa \bullet; \#]$$

$$q_{15} = [Y \rightarrow c \bullet; a\#]$$

$$q_{16} = [Y \rightarrow c \bullet a; a\#]$$

$$q_{17} = [Y \rightarrow ca \bullet; a\#]$$

$q_0 = [Z \rightarrow \bullet X; \#]$	$q_9 = [Y \rightarrow c \bullet a; \#]$
$q_1 = [Z \rightarrow X \bullet; \#]$	$q_{10} = [X \rightarrow bY \bullet a; \#]$
$q_2 = [X \rightarrow \bullet Y; \#]$	$q_{11} = [Y \rightarrow \bullet c; a\#]$
$q_3 = [X \rightarrow \bullet bYa; \#]$	$q_{12} = [Y \rightarrow \bullet ca; a\#]$
$q_4 = [X \rightarrow Y \bullet; \#]$	$q_{13} = [Y \rightarrow ca \bullet; \#]$
$q_5 = [Y \rightarrow \bullet c; \#]$	$q_{14} = [X \rightarrow bYa \bullet; \#]$
$q_6 = [Y \rightarrow \bullet ca; \#]$	$q_{15} = [Y \rightarrow c \bullet; a\#]$
$q_7 = [X \rightarrow b \bullet Ya; \#]$	$q_{16} = [Y \rightarrow c \bullet a; a\#]$
$q_8 = [Y \rightarrow c \bullet; \#]$	$q_{17} = [Y \rightarrow ca \bullet; a\#]$

$$\begin{aligned}
R = \{ & q_0c\# \rightarrow q_1q_2c\#, & q_7ca\# \rightarrow q_{10}q_{11}ca\# \\
& q_0ca\# \rightarrow q_1q_2ca\#, & q_7caa \rightarrow q_{10}q_{12}caa, \\
& q_0bca \rightarrow q_1q_3bca, & q_8 \rightarrow \epsilon, \\
& q_1 \rightarrow \epsilon, & q_9a \rightarrow q_{13}, \\
& q_2c\# \rightarrow q_4q_5c\#, & q_{10}a \rightarrow q_{14}, \\
& q_2ca\# \rightarrow q_4q_6ca\#, & q_{11}c \rightarrow q_{15}, \\
& q_3b \rightarrow q_7, & q_{12}c \rightarrow q_{16}, q_{13} \rightarrow \epsilon, \\
& q_4 \rightarrow \epsilon, & q_{14} \rightarrow \epsilon, \\
& q_5c \rightarrow q_8, & q_{15} \rightarrow \epsilon, \\
& q_6c \rightarrow q_9, & q_{16}a \rightarrow q_{17}, q_{17} \rightarrow \epsilon \}
\end{aligned}$$

aceeasi gramatica dar cu $k = 2$

$$q_7ca \rightarrow q_{10}q_{11}ca$$

$$q_7ca \rightarrow q_{10}q_{12}ca$$

Cu $k = 3$

$$q_7ca\# \rightarrow q_{10}q_{11}ca\#$$

$$q_7caa \rightarrow q_{10}q_{12}caa$$

unde pt $k = 3$

- ▶ $q_7 = [X \rightarrow b.Ya; \#]$
- ▶ $q_{10} = [X \rightarrow bY.a; \#]$
- ▶ $q_{11} = [Y \rightarrow .c; a\#]$
- ▶ $q_{12} = [Y \rightarrow .ca; a\#]$

Derivare $Z \Rightarrow X \Rightarrow bYa \Rightarrow bcaa$

Stiva	Stare	Intrare	Derivarea cea mai din stanga
q_0	q_0	$bcaa\#$	Z
q_0q_1	q_3	$bcaa\#$	X
q_0q_1	q_7	$caa\#$	bYa
$q_0q_1q_{10}$	q_{12}	$caa\#$	$bcaa$
$q_0q_1q_{10}$	q_{16}	$aa\#$	
$q_0q_1q_{10}$	q_{17}	$a\#$	
q_0q_1	q_{10}	$a\#$	
q_0q_1	q_{14}	$\#$	
q_0	q_1	$\#$	
	q_0	$\#$	

- ▶ La **tranzitiile de stivuire** sunt examinate simbolurile dinainte (lookaheads symbols).
- ▶ Aceste tranzitii corespund **intrarii intr-o productie noua**
- ▶ **Citirea simbolurilor terminale si decizia de terminare a productiei printr-o tranzitie de destivuire** se realizeaza fara inspectarea simbolurilor dinainte

Rezumat

Recursive descent parsing

Predictive parsing

Structuri ajutatoare: FIRST, FOLLOW

Definire Gramatici LL(k)

Algoritmul LL(K)

Exemplu aplicare LL(k)

Exemplu aplicare LL(3)

Ce e adevarat pentru automatul stiva A construit pt analiza descendenta a G ?

$G = (\{a, b\}, \{S\}, \{S \rightarrow ab|aSb\}, S),$

1. stiva contine initial a
2. are o singura stare
3. stiva contine initial S
4. vocabularul stivei este $\{a, b, S\}$
5. $bqa \rightarrow q$ este o productie
6. $Sq \rightarrow bSaq$ este o productie

Automatul stiva:

► $T = \{0, 1\}, Q = \{q\}, q_0 = q,$
 $F = \{q\}, S = \{0, 1, S\}, s_0 = S$

► cu productiile R

1. $Sq \rightarrow 1S0q$

2. $1q1 \rightarrow q,$

3. $0q0 \rightarrow q,$

4. $Sq \rightarrow q,$

Analiza descendenta. Gramatici LL(k) - eliminare
recursivitate stanga

Teorema

4.2.2, 4.2.3 Teorema. O gramatica LL(k) nu poate avea simbol nonterminal recursiv stanga.

Daca $X \Rightarrow X\omega, \omega \neq \varepsilon$ - X nonterminal recursiv stanga

► $E \rightarrow E + T \mid T$

► $T \rightarrow T * F \mid F$

► $F \rightarrow (E) \mid id$

are doua productii cu recursivitate stanga

Teorema

Teorema. Pentru orice gramatica CFG $G = (T, N, P, Z)$ cu simboluri nonterminale recursive stanga exista o gramatica echivalenta $G' = (T, N', P', Z)$ fara nonterminale recursive stanga.

Idee

$$X \rightarrow X\alpha|\beta \text{ devine } \begin{cases} X \rightarrow \beta X' \\ X' \rightarrow \alpha X'|\varepsilon \end{cases}$$

► $E \rightarrow E + T | T$

► $T \rightarrow T * F | F$

► $F \rightarrow (E) | id$

► $E \rightarrow TE'$

► $E' \rightarrow +TE'|\varepsilon$

► $T \rightarrow FT'$

► $T' \rightarrow *FT'|\varepsilon$

► $F \rightarrow (E) | id$

Dar...

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

$$S \Rightarrow Aa \Rightarrow Sda$$

ne trebuie un algoritm care sa elimine toate nonterminalele cu recursivitate stanga

- ▶ Consideram ca $N = \{X_1, X_2, \dots, X_n\}$ - simbolurile nonterminale sunt numerotate consecutiv.
- ▶ Daca putem alege indicii a.i. indicii sa respecte $i < j$ pentru toate productiile $X_i \rightarrow X_j \omega$ atunci G nu are recursivitate stanga.
- ▶ Daca o astfel de numerotare nu este posibila pentru G , atunci se genereaza G' .

Exemple:

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$
- ▶ Daca S e 1, A e 2, prima productie respecta $i < j$ dar nu si a doua
- ▶ $E \rightarrow E + T$ nu respecta $i < j$

Algoritm de eliminare recursivitate stanga

1. Fie $N' = N, P' = P$. Se executa pasii 2,3 pentru $i = 1, \dots, n$
2. Pentru $j = 1, \dots, i - 1$
 $X_i \rightarrow X_j \omega \in P'$ se inlocuiesc cu $\{X_i \rightarrow \chi_j \omega \mid X_j \rightarrow \chi_j \in P'\}$.
In consecinta, $X_i \Rightarrow^+ X_j \gamma$ implica $i \leq j$.
3. Se inlocuiesc $X_i \rightarrow X_j \omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$
adaugand un nou simbol Y_i la N' .
+ se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i \gamma$ cu $X_i \rightarrow \chi Y_i$.
Simbolurile noi se numeroteaza cu $n+1, n+2, \dots$

$$\blacktriangleright E \rightarrow E + T | T$$

$$\blacktriangleright T \rightarrow T * F | F$$

$$\blacktriangleright F \rightarrow (E) | id$$

presupunem ordinea $E(1) < T(2) < F(3)$

i	pasul 2	pasul 3	variabila noua
1	nu se executa	$E \rightarrow E + T T$ devin $E' \rightarrow +TE' \varepsilon$ si $E \rightarrow TE';$	$E'(4)$
2	$j = 1$ $T \rightarrow E\omega$ nu exista	$T \rightarrow T * F F$ devin $T' \rightarrow *FT' \varepsilon$ $T \rightarrow FT'$	$T'(5)$
3	$j = 1, 2$ $F \rightarrow E\omega$ sau $F \rightarrow T\omega$ nu exista	$F \rightarrow F\omega$ nu exista	
4,5	nu se modifica nimic		

Rezultat:

- ▶ $E \rightarrow E + T \mid T$
 - ▶ $T \rightarrow T * F \mid F$
 - ▶ $F \rightarrow (E) \mid id$
- ▶ $E \rightarrow TE'$
 - ▶ $E' \rightarrow +TE' \mid \varepsilon$
 - ▶ $T \rightarrow FT'$
 - ▶ $T' \rightarrow *FT' \mid \varepsilon$
 - ▶ $F \rightarrow (E) \mid id$

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

...pasul 2 al algoritmului: Pentru $j = 1, \dots, i - 1$ $X_i \rightarrow X_j\omega \in P'$ se inlocuiesc cu $\{X_i \rightarrow \chi_j\omega | X_j \rightarrow \chi_j \in P'\}$.

Daca $S(1) < A(2)$

- ▶ $i = 1$ nimic
- ▶ $i = 2$ la pasul 2 $A \rightarrow Sd$ se inlocuieste cu $\{A \rightarrow Aad|bd\}$

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

...pasul 3 al algoritmului: Se inlocuiesc $X_i \rightarrow X_i\omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$ adaugand un nou simbol Y_i la N' .
 + se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i\gamma$ cu $X_i \rightarrow \chi Y_i$.

Daca $S(1) < A(2)$

- ▶ $i = 1$ nimic
- ▶ $i = 2$ la pasul 2 $A \rightarrow Sd$ se inlocuieste cu $\{A \rightarrow Aad|bd\}$
- ▶ $i = 2$ la pasul 3 $A \rightarrow Ac|Aad|bd|\varepsilon$ se inlocuieste cu $A' \rightarrow cA', A' \rightarrow adA', A' \rightarrow \varepsilon$ si
 $A \rightarrow bdA', A \rightarrow A'$

Teorema. Daca sirul ω din $X_i \rightarrow X_i\omega$ nu incepe cu $X_j, j \leq i$ atunci $X_i \rightarrow X_i\omega$ se poate inlocui cu $\{Y_i \rightarrow \omega, Y_i \rightarrow \omega Y_i\}$ si $X_i \rightarrow \chi$ cu $\{X_i \rightarrow \chi, X_i \rightarrow \chi Y_i\}$ la pasul 3.

pasul 3 anterior ...se inlocuiesc $X_i \rightarrow X_i\omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$ adaugand un nou simbol Y_i la N' .
+ se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i\gamma$ cu $X_i \rightarrow \chi Y_i$.

Se evita introducerea productiilor ε .

- ▶ $E \rightarrow E + T \mid T$
- ▶ $T \rightarrow T * F \mid F$
- ▶ $F \rightarrow (E) \mid id$

Cu productii ε

- ▶ $E \rightarrow TE'$
- ▶ $E' \rightarrow +TE' \mid \varepsilon$
- ▶ $T \rightarrow FT'$
- ▶ $T' \rightarrow *FT' \mid \varepsilon$
- ▶ $F \rightarrow (E) \mid id$

Fara productii ε

- ▶ $E \rightarrow TE' \mid T$
- ▶ $E' \rightarrow +T \mid +TE'$
- ▶ $T \rightarrow FT' \mid F$
- ▶ $T' \rightarrow *F \mid *FT'$
- ▶ $F \rightarrow (E) \mid id$

Observatii

- ▶ Recursivitatea stanga precum $E \rightarrow T|E + T$ - utilizata pentru a reflecta asociativitatea stanga a operatorilor.
- ▶ Aceeasi proprietate avem si in $E \rightarrow TE', E' \rightarrow +TE', E' \rightarrow \varepsilon$
- ▶ Insa asociativitate dreapta $E \rightarrow T, E \rightarrow T + E$.

Factorizare stanga

Productiile ε se pot elimina intotdeauna dintr-o gramatica LL(k),
dar aceasta poate mari valoare lui k.

4.2.3

Teorema

TEOREMA. Pentru orice gramatica LL(k) cu productii ε exista o gramatica LL($k+1$) fara productii ε care genereaza limbajul $L(G) - \varepsilon$.

Prin introducerea productiilor ε se poate reduce k .

Teorema

TEOREMA. Pentru orice gramatica $LL(k+1)$, $k > 0$ fara productii ε exista o gramatica $LL(k)$ echivalenta cu productii ε .

Factorizare stanga

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow Yc \mid Yd \\ Y \rightarrow a \mid bY \end{array} \}$$

Productiile $X \rightarrow Yc$ si $X \rightarrow Yd$ nu pot fi distinse chiar prin examinarea oricarui numar fix de simboluri din sirul de intrare deoarece din Y se poate deriva un sir de lungime si mai mare.

Solutie: evitarea problemei prin amanarea deciziei. Ambele incep cu Y , nu trebuie facuta distinctie intre ele decat dupa ce Y a fost recunoscut.

Factorizare stanga

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow Yc|Yd \\ Y \rightarrow a|bY \end{array} \}$$

devine

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow YX' \\ X' \rightarrow c|d \\ Y \rightarrow a|bY \end{array} \}$$

Se poate examina un singur caracter inainte pt a face diferente
intre cele doua variante c sau d .

Factorizare stanga

Fie $P = \{$

- $Z \rightarrow X$
- $X \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$
- $E \rightarrow b\}$

devine

Fie $P = \{$

- $Z \rightarrow X$
- $X \rightarrow \text{if } E \text{ then } S \ S' \mid a$
- $S' \rightarrow \text{else } S \mid \varepsilon$
- $E \rightarrow b\}$

Kahoot

- ▶ Pentru $G = (\{id, (,), +\}, \{E, E', T\}, E, P)$ si P:
- ▶ $E \rightarrow TE'$
- ▶ $E' \rightarrow +TE' | \varepsilon$
- ▶ $T \rightarrow (E) | id$

O gramatica G este $LL(k)$ daca exista un automat stiva determinist A cu $L(A)=L(G)$ care

1. citește inputul de la stanga la dreapta
2. citește inputul de la dreapta la stanga
3. trasează derivarea cea mai din dreapta
4. trasează derivarea cea mai din stanga
5. examinează mai mult de k simboluri de intrare la fiecare pas
6. nu examinează mai mult de k simboluri de intrare la fiecare pas

Situatia $[S \rightarrow 0.S1; \#]$ exprima o stare a automatului stiva in care:

1. Din productia $S \rightarrow 0S1$ s-a analizat deja $0S$, se asteapta un sir derivat din 1 , urmat de $\#$
2. Din productia $S \rightarrow 0S1$ s-a analizat deja 0 , se asteapta un sir derivat din $S1$, urmat de $\#$
3. Din productia $S \rightarrow 0S1$ se asteapta un 0 , urmat de un sir derivat din $S1$, urmat de $\#$
4. Din productia $S \rightarrow 0S1$ s-a analizat deja 0 , se asteapta un sir derivat din S , urmat de $1\#$

► $Z \rightarrow X$

► $X \rightarrow Y|bYa$

► $Y \rightarrow c|ca$

		stari noi	tranzitii noi
		$q_0 = [Z \rightarrow .X; \#]$	
q_0	5	$q' = ?$	

- ▶ $Z \rightarrow X$
- ▶ $X \rightarrow Y|bYa$
- ▶ $Y \rightarrow c|ca$

		stari noi	tranzitii noi
		$q_0 = [Z \rightarrow .X; \#]$	
q_0	5	$q' = [Z \rightarrow X.; \#] = q_1$	

- ▶ $Z \rightarrow X$
- ▶ $X \rightarrow Y|bYa$
- ▶ $Y \rightarrow c|ca$

		stari noi	tranzitii noi
		$q_0 = [Z \rightarrow .X; \#]$	
q_0	5	$q' = [Z \rightarrow X.; \#] = q_1$ $H = \{[X \rightarrow .Y; \#] = q_2,$ $[X \rightarrow .bYa; \#] = q_3\}$	$q_0 \tau_2 \rightarrow q_1 q_2 \tau_2$ $q_0 \tau_3 \rightarrow q_1 q_3 \tau_3$

- ▶ $Z \rightarrow X$
- ▶ $X \rightarrow Y|bYa$
- ▶ $Y \rightarrow c|ca$

		stari noi	tranzitii noi
		$q_0 = [Z \rightarrow .X; \#]$	
q_0	5	$q' = [Z \rightarrow X.; \#] = q_1$ $H = \{[X \rightarrow .Y; \#] = q_2,$ $\quad [X \rightarrow .bYa; \#] = q_3\}$	$\tau_2 \in FIRST_3(Y\#) = \{c\#, ca\# \}$ $q_0\tau_2 \rightarrow q_1q_2\tau_2$ $q_0\tau_3 \rightarrow q_1q_3\tau_3$ $\tau_3 \in FIRST_3(bYa) = \{bca\}$

Gramatici LL(k) tari. Derivare descendent recursiva

Ce e gramatica $LL(k)$? - reaminitire

O gramatica independenta de context $G = (T, N, P, Z)$ este $LL(k)$ pentru un $k \geq 0$ daca pentru derivari arbitrare

$$Z \Rightarrow^L \mu X \chi \Rightarrow \mu \nu \chi \Rightarrow^* \mu \gamma$$

$$Z \Rightarrow^L \mu X \chi \Rightarrow \mu \omega \chi \Rightarrow^* \mu \gamma'$$

$$\text{unde } \mu, \gamma, \gamma' \in T^*, \nu, \chi, \omega \in V^*, X \in N$$

avem urmatoarea proprietate: $k : \gamma = k : \gamma'$ implica $\nu = \omega$

Observatie: Dependenta de μ obliga pastrarea in situatiile $[X \rightarrow \alpha.\beta; \omega]$ a contextului dreapta. Daca se elimina aceasta dependenta: gramatici **$LL(k)$ tari**

Gramatici $LL(k)$ tari

O gramatică independentă de context $G = (T, N, P, Z)$ este o gramatică $LL(k)$ tare pentru un $k > 0$ dacă pentru derivări arbitrare

$$Z \Rightarrow^L \mu X \chi \Rightarrow \mu \nu \chi \Rightarrow^* \mu \gamma$$

$$Z \Rightarrow^L \mu' X \chi' \Rightarrow \mu' \omega \chi' \Rightarrow^* \mu' \gamma'$$

$$\text{unde } \mu, \mu', \gamma, \gamma' \in T^*, \nu, \chi, \omega \in V^*, X \in N$$

avem următoarea proprietate: $k : \gamma = k : \gamma'$ implică $\nu = \omega$

Fie G cu $P = \{$

$$\begin{aligned} Z &\rightarrow X \\ X &\rightarrow aAab|bAbb \\ A &\rightarrow a|\varepsilon \end{aligned}$$

$Z \Rightarrow X \Rightarrow aAab \xRightarrow{A \rightarrow \varepsilon} aab$

$Z \Rightarrow X \Rightarrow aAab \Rightarrow aaab$

$Z \Rightarrow X \Rightarrow bAbb \Rightarrow bbb$

$Z \Rightarrow X \Rightarrow bAbb \xRightarrow{A \rightarrow a} babb$

Este LL(1)? Este LL(2)? Este strong LL(2)?

Fie G cu $P = \{$

$$\begin{aligned} Z &\rightarrow X \\ X &\rightarrow aAab|bAbb \\ A &\rightarrow a|\varepsilon \end{aligned}$$

$Z \Rightarrow X \Rightarrow aAab \xRightarrow{A \rightarrow \varepsilon} aab$

$Z \Rightarrow X \Rightarrow aAab \Rightarrow aaab$

$Z \Rightarrow X \Rightarrow bAbb \Rightarrow bbb$

$Z \Rightarrow X \Rightarrow bAbb \xRightarrow{A \rightarrow a} babb$

Este LL(1)? Este LL(2)? Este strong LL(2)?

$Z \Rightarrow X \Rightarrow aAab \Rightarrow a**ab**$

$Z \Rightarrow X \Rightarrow bAbb \Rightarrow **b**abb$

pt LL(k) tare: $k : \gamma = k : \gamma \Rightarrow$ aceeași producție pt A ; dar aici contextul stanga contează

Conditia strong LL(k)

O gramatica independenta de context G este $LL(k)$ daca pentru orice pereche de productii $X \rightarrow \chi$, $X \rightarrow \chi'$, $\chi \neq \chi'$ urmatoarea conditie este adevarata:

$$FIRST_k(\chi FOLLOW_k(X)) \cap FIRST_k(\chi' FOLLOW_k(X)) = \emptyset$$

Fie G cu $P = \{$
exemplu $\begin{array}{l} Z \rightarrow X \\ X \rightarrow aAab|bAbb \\ A \rightarrow a|\varepsilon \end{array}$
pt $A : FIRST_2(a\{ab, bb\}) \cap FIRST_2(\varepsilon\{ab, bb\}) = \{ab\}$

Strong LL(k)

NU e necesar niciun context pt a decide productia pentru nonterminalul X . Nu trebuie tinuti minte pasii anteriori din derivarea stanga, cei care au condus la nonterminalul X .

Algoritmul LL(k) - reamintire

Fie $G = (T, N, P, Z)$. Pt automatul stiva se determina Q si tranzitiile R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = [Z \rightarrow .S, \{\#\}]$
Obs: $FOLLOW_k(Z) = \{\#\}$. q_0 starea initiala si a stivei.
Automatul se opreste daca aceasta stare se intalneste din nou, stiva este vida, simbolul de intrare urmator este $\#$.
2. fie $q = [X \rightarrow \mu.\nu; \Omega]$ un element al lui Q care inca nu a fost tratat
3. Daca $\nu = \varepsilon$ atunci se include $q\varepsilon \rightarrow \varepsilon$ in R .
4. Daca $\nu = t\gamma$, $t \in T$ si $\gamma \in V^*$, fie $q' = [X \rightarrow \mu t.\gamma; \Omega]$.
Aaduga q' in Q si $qt \rightarrow q'$ in R .
5. Daca $\nu = Y\gamma$, $Y \in N$ si $\gamma \in V^*$,
 - ▶ fie $q' = [X \rightarrow \mu Y.\gamma; \Omega]$
 - ▶ si $H = \{[Y \rightarrow \beta_i; FIRST_k(\gamma\Omega)] \mid Y \rightarrow \beta_i \in P\}$.
 - ▶ actualizeaza $Q = Q \cup \{q'\} \cup H$
 - ▶ si $R = R \cup \{q\tau_i \rightarrow q'h_i\tau_i \mid h_i \in H, \tau_i \in FIRST_k(\beta_i\gamma\Omega)\}$
6. daca toate starile din q au fost analizate, stop. Altfel continua cu 2.

Algoritm LL(k) tare

Daca $\nu = Y\gamma$, $Y \in N$ si $\gamma \in V^*$ in loc de pasul 5 din LL(k)

- ▶ fie $q' = [X \rightarrow \mu Y.\gamma; \Omega]$
- ▶ si $H = \{[Y \rightarrow \beta_i; \textcolor{red}{FIRST}_k(\gamma\Omega)] \mid Y \rightarrow \beta_i \in P\}$.
- ▶ actualizeaza $Q = Q \cup \{q'\} \cup H$ si
- ▶ $R = R \cup \{q\tau_i \rightarrow q'h_i\tau_i \mid h_i \in H, \tau_i \in \textcolor{red}{FIRST}_k(\beta_i\gamma\Omega)\}$

se poate folosi

- ▶ fie $q' = [X \rightarrow \mu Y.\gamma; \Omega]$
- ▶ si $H = \{[Y \rightarrow \beta_i; \textcolor{red}{FOLLOW}_k(Y)] \mid Y \rightarrow \beta_i \in P\}$.
- ▶ actualizeaza $Q = Q \cup \{q'\} \cup H$ si
- ▶ $R = R \cup \{q\tau_i \rightarrow q'h_i\tau_i \mid h_i \in H, \tau_i \in \textcolor{red}{FIRST}_k(\beta_i\textcolor{red}{FOLLOW}_k(Y))\}$

Toate situatiile distincte anterior doar prin context dreapta apartin intotdeauna aceleiasi stari.

LL(1) tare

Fie $Z \rightarrow E$, $E \rightarrow E + F | F$, $F \rightarrow i|(E)$

Prin eliminarea recursivitatii stanga:

$Z \rightarrow E$, $E \rightarrow FE_1$, $E_1 \rightarrow \varepsilon | + FE_1$, $F \rightarrow i|(E)$

simbol	$FIRST_1(X)$	$FOLLOW_1(X)$
E	$\{(, i\}$	$\}, \#\}$
E_1	$\{+, \varepsilon\}$	$\}, \#\}$
F	$\{(, i\}$	$\{+, \#,)\}$

Conditie LL(1) tare:

pt E_1 :

$$FIRST_1(\varepsilon FOLLOW(E_1)) \cap FIRST_1(+FE_1 FOLLOW(E_1)) = \emptyset$$

pt F :

$$FIRST_1(iFOLLOW(F)) \cap FIRST_1((E)FOLLOW(F)) = \emptyset$$

stari noi

$q_0 = [Z \rightarrow .E; \#]$

tranzitii noi

	stari noi	tranzitii noi
	$q_0 = [Z \rightarrow .E; \#]$	
q_0	$q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($

	stari noi	tranzitii noi
	$q_0 = [Z \rightarrow .E; \#]$	
q_0	$q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1		$q_1 \varepsilon \rightarrow \varepsilon$

	stari noi	tranzitii noi
	$q_0 = [Z \rightarrow .E; \#]$	
q_0	$q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1		$q_1 \varepsilon \rightarrow \varepsilon$
q_2	$[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($

fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
q_0 $q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
q_2 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$

	stari noi	tranzitii noi
	$q_0 = [Z \rightarrow .E; \#]$	
q_0	$q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1		$q_1 \varepsilon \rightarrow \varepsilon$
q_2	$[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)		
q_3	$[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4	$[F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
q_0 $q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
q_2 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4	$[F \rightarrow i.] = q_9$
q_5	$[F \rightarrow (.E)] = q_{10}$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
q_0 $q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
q_2 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4	$q_4 i \rightarrow q_9$
q_5	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
q_0 $q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
q_2 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4	$q_4 i \rightarrow q_9$
q_5	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
$q_0 \quad q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
$q_2 \quad [E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
$q_3 \quad [E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
$q_4 \quad [F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$
$q_5 \quad [F \rightarrow (.E)] = q_{10}$	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
$q_8 \quad [E_1 \rightarrow +.FE_1] = q_{11}$	$q_8 + \rightarrow q_{11}$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$	
$q_0 \quad q' = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1	$q_1 \varepsilon \rightarrow \varepsilon$
$q_2 \quad [E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
$q_3 \quad [E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
$q_4 \quad [F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$
$q_5 \quad [F \rightarrow (.E)] = q_{10}$	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
$q_8 \quad [E_1 \rightarrow +.FE_1] = q_{11}$	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$ $q'_0 = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$q_1 \varepsilon \rightarrow \varepsilon$ $\tau \in FIRST_1(i FOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4 $[F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$
q_5 $[F \rightarrow (.E)] = q_{10}$	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
q_8 $[E_1 \rightarrow +.FE_1] = q_{11}$	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$
q_{10} $[F \rightarrow (E.)] = q_{12}$ $H = \{[E \rightarrow .FE_1] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW(E))$ $q_{10} (\rightarrow q_{12} q_2 ($ $q_{10} i \rightarrow q_{12} q_2 i$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$ $q'_0 = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$q_1 \varepsilon \rightarrow \varepsilon$ $\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4 $[F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$
q_5 $[F \rightarrow .(E)] = q_{10}$	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
q_8 $[E_1 \rightarrow +.FE_1] = q_{11}$	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$
q_{10} $[F \rightarrow (E.)] = q_{12}$ $H = \{[E \rightarrow .FE_1] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW(E))$ $q_{10} (\rightarrow q_{12} q_2 ($ $q_{10} i \rightarrow q_{12} q_2 i$
q_{11} $[E_1 \rightarrow +F.E_1] = q_{13}$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_{11} i \rightarrow q_{13} q_4 i$ $q_{11} (\rightarrow q_{13} q_5 ($

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$ $q'_0 = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1 $[E \rightarrow F.E_1] = q_3$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$q_1 \varepsilon \rightarrow \varepsilon$ $\tau \in FIRST_1(iFOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3 $[E \rightarrow FE_1.] = q_6$ $H = \{[E_1 \rightarrow .\varepsilon] = q_7$ $[E_1 \rightarrow . + FE_1] = q_8\}$	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4 $[F \rightarrow i.] = q_9$	$q_4 i \rightarrow q_9$
q_5 $[F \rightarrow .(E)] = q_{10}$	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
q_8 $[E_1 \rightarrow +.FE_1] = q_{11}$	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$
q_{10} $[F \rightarrow (E.)] = q_{12}$ $H = \{[E \rightarrow .FE_1] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW(E))$ $q_{10} (\rightarrow q_{12} q_2 ($ $q_{10} i \rightarrow q_{12} q_2 i$
q_{11} $[E_1 \rightarrow +F.E_1] = q_{13}$ $H = \{[F \rightarrow .i, FOLLOW_1(F)] = q_4$ $[F \rightarrow .(E); FOLLOW_1(F)] = q_5\}$	$\tau \in FIRST_1(iFOLLOW_1(F))$ $q_{11} i \rightarrow q_{13} q_4 i$ $q_{11} (\rightarrow q_{13} q_5 ($
q_{12} $[F \rightarrow (E).] = q_{14}$	$q_{12}) \rightarrow q_{14}$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$ $q'_0 = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1 q_2	$q_1 \varepsilon \rightarrow \varepsilon$ $\tau \in FIRST_1(i FOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
q_4	$q_4 i \rightarrow q_9$
q_5	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
q_8	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$
q_{10}	$\tau \in FIRST_1(FE_1 FOLLOW(E))$ $q_{10} (\rightarrow q_{12} q_2 ($ $q_{10} i \rightarrow q_{12} q_2 i$
q_{11}	$\tau \in FIRST_1(i FOLLOW_1(F))$ $q_{11} i \rightarrow q_{13} q_4 i$ $q_{11} (\rightarrow q_{13} q_5 ($
q_{12}	$q_{12}) \rightarrow q_{14}$
q_{13}	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_{13}) \rightarrow q_{15} q_7)$ $q_{13} \# \rightarrow q_6 q_7 \#$ $q_{13} + \rightarrow q_{15} q_8 +$

stari noi	tranzitii noi
$q_0 = [Z \rightarrow .E; \#]$ $q'_0 = [Z \rightarrow E.; \#] = q_1$ $H = \{[E \rightarrow .FE_1; \#] = q_2\}$	$\tau \in FIRST_1(FE_1 FOLLOW_1(E)) = \{i, (\}$ $q_0 i \rightarrow q_1 q_2 i$ $q_0 (\rightarrow q_1 q_2 ($
q_1 q_2	$q_1 \varepsilon \rightarrow \varepsilon$ $\tau \in FIRST_1(i FOLLOW_1(F))$ $q_2 i \rightarrow q_3 q_4 i$ $q_2 (\rightarrow q_3 q_5 ($
fiind LL(1) strong, capetele din situatii nu le mai pastram (se pot deduce din situatie)	
q_3	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_3) \rightarrow q_6 q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_3 + \rightarrow q_6 q_8 +$
$[E_1 \rightarrow . + FE_1] = q_8$	
q_4	$q_4 i \rightarrow q_9$
q_5	$q_5 (\rightarrow q_{10}$
q_6	$q_6 \varepsilon \rightarrow \varepsilon$
q_7	$q_7 \varepsilon \rightarrow \varepsilon$
q_8	$q_8 + \rightarrow q_{11}$
q_9	$q_9 \varepsilon \rightarrow \varepsilon$
q_{10}	$\tau \in FIRST_1(FE_1 FOLLOW(E))$ $q_{10} (\rightarrow q_{12} q_2 ($ $q_{10} i \rightarrow q_{12} q_2 i$
q_{11}	$\tau \in FIRST_1(i FOLLOW_1(F))$ $q_{11} i \rightarrow q_{13} q_4 i$ $q_{11} (\rightarrow q_{13} q_5 ($
q_{12}	$q_{12}) \rightarrow q_{14}$
q_{13}	$\tau \in FIRST_1(\varepsilon FOLLOW(E_1))$ $q_{13}) \rightarrow q_{15} q_7)$ $q_3 \# \rightarrow q_6 q_7 \#$ $q_{13} + \rightarrow q_{15} q_8 +$ $q_{14} \varepsilon \rightarrow \varepsilon$ $q_{15} \varepsilon \rightarrow \varepsilon$
$[E_1 \rightarrow . + FE_1] = q_8$	
q_{14} q_{15}	

$$\begin{array}{ll}
q_0 : [Z \rightarrow \bullet E] & q_8 : [E_1 \rightarrow \bullet + F E_1] \\
q_1 : [Z \rightarrow E \bullet] & q_9 : [F \rightarrow i \bullet] \\
q_2 : [E \rightarrow \bullet F E_1] & q_{10} : [F \rightarrow (\bullet E)] \\
q_3 : [E \rightarrow F \bullet E_1] & q_{11} : [E_1 \rightarrow + \bullet F E_1] \\
q_4 : [F \rightarrow \bullet i] & q_{12} : [F \rightarrow (E \bullet)] \\
q_5 : [F \rightarrow \bullet (E)] & q_{13} : [E_1 \rightarrow + F \bullet E_1] \\
q_6 : [E \rightarrow F E_1 \bullet] & q_{14} : [F \rightarrow (E) \bullet] \\
q_7 : [E_1 \rightarrow \bullet \epsilon] & q_{15} : [E_1 \rightarrow + F E_1 \bullet]
\end{array}$$

$$\begin{array}{lll}
q_0 i \rightarrow q_1 q_2 i, & q_0 (\rightarrow q_1 q_2 (, & \\
q_1 \rightarrow \epsilon, & & \\
q_2 i \rightarrow q_3 q_4 i, & q_2 (\rightarrow q_3 q_5 (, & \\
q_3 \# \rightarrow q_6 q_7 \#, & q_3) \rightarrow q_6 q_7), & q_3 + \rightarrow q_6 q_8 +, \\
q_4 i \rightarrow q_9, & & \\
q_5 (\rightarrow q_{10}, & & \\
q_6 \rightarrow \epsilon, & & \\
q_7 \rightarrow \epsilon, & & \\
q_8 + \rightarrow q_{11}, & & \\
q_9 \rightarrow \epsilon, & & \\
q_{10} i \rightarrow q_{12} q_2 i, & q_{10} (\rightarrow q_{12} q_2 (, & \\
q_{11} i \rightarrow q_{13} q_4 i, & q_{11} (\rightarrow q_{13} q_5 (, & \\
q_{12}) \rightarrow q_{14}, & & \\
q_{13} \# \rightarrow q_{15} q_7 \#, & q_{13}) \rightarrow q_{15} q_7), & q_{13} + \rightarrow q_{15} q_8 +, \\
q_{14} \rightarrow \epsilon, & & \\
q_{15} \rightarrow \epsilon & &
\end{array}$$

$q_0 q_0$	$(i + i)\#$	$q_0(\rightarrow q_1 q_2($
$q_0 q_1 q_2$	$(i + i)\#$	$q_2(\rightarrow q_3 q_5($
$q_0 q_1 q_3 q_5$	$(i + i)\#$	$q_5(\rightarrow q_{10}$
$q_0 q_1 q_3 q_{10}$	$i + i)\#$	$q_{10} i \rightarrow q_{12} q_2 i$
$q_0 q_1 q_3 q_{12} q_2$	$i + i)\#$	$q_2 i \rightarrow q_3 q_4 i$
$q_0 q_1 q_3 q_{12} q_3 q_4$	$i + i)\#$	$q_4 i \rightarrow q_9$
$q_0 q_1 q_3 q_{12} q_3 q_9$	$+i)\#$	$q_9 \rightarrow \varepsilon$
$q_0 q_1 q_3 q_{12} q_3$	$+i)\#$	$q_3 + \rightarrow q_6 q_8 +$
$q_0 q_1 q_3 q_{12} q_6 q_8$	$+i)\#$	$q_8 + \rightarrow q_{11}$
$q_0 q_1 q_3 q_{12} q_6 q_{11}$	$i)\#$	$q_{11} i + \rightarrow q_{13} q_4 i$
$q_0 q_1 q_3 q_{12} q_6 q_{13} q_4$	$i)\#$	$q_4 i \rightarrow q_9$
$q_0 q_1 q_3 q_{12} q_6 q_{13} q_9$	$)\#$	$q_9 \rightarrow \varepsilon$
$q_0 q_1 q_3 q_{12} q_6 q_{13}$	$)\#$	$q_{13}) \rightarrow q_{15} q_7)$
$q_0 q_1 q_3 q_{12} q_6 q_{15} q_7$	$)\#$	$q_7 \rightarrow \varepsilon$
$q_0 q_1 q_3 q_{12} q_6 q_{15}$	$)\#$	$q_{15} \rightarrow \varepsilon$
$q_0 q_1 q_3 q_{12} q_6$	$)\#$	$q_6 \rightarrow \varepsilon$
$q_0 q_1 q_3 q_{12}$	$)\#$	$q_{12}) \rightarrow q_{14}$
$q_0 q_1 q_3 q_{14}$	$\#$	$q_{14} \rightarrow \varepsilon$
$q_0 q_1 q_3$	$\#$	$q_3 \# \rightarrow q_6 q_7 \#$
$q_0 q_1 q_6 q_7$	$\#$	$q_7 \rightarrow \varepsilon$
$q_0 q_1 q_6$	$\#$	$q_6 \rightarrow \varepsilon$
$q_0 q_1$	$\#$	$q_1 \rightarrow \varepsilon$
q_0	$\#$	$q_1 \rightarrow \varepsilon$

$q_0 = [Z \rightarrow .E]$
 $q_1 = [Z \rightarrow E.], q_2 = [E \rightarrow .FE_1]$
 $q_3 = [E \rightarrow F.E_1], q_5 = [F \rightarrow .(E)]$
 $q_{10} = [F \rightarrow .(E)]$
 $q_{12} = [F \rightarrow (E.)], q_2 = [E \rightarrow .FE_1]$
 $q_3 = [E \rightarrow F.E_1], \dots$

Algoritm derivator LL(1)

Convertirea automatului LL(1) in proceduri recursive: Descendenta recursiva (Recursive descent)

- ▶ derivator descendent recursiv: starea automatului este o pozitie din derivator
- ▶ stiva - locatii de unde derivatorul poate relua executia
- ▶ daca starea e $[X \rightarrow \mu.B\nu; \omega]$, $B \in N$: se pune pe stiva informatia despre $[X \rightarrow \mu.B.\nu; \omega]$ inainte de a lua in considerare $B \rightarrow \beta$.
- ▶ daca folosim limbaje de programare cu suport pt recursivitate: **procedura** pt fiecare nonterminal B + mecanismul standard de **recursivitate** pentru a implementa stiva automatului

Schema de program

$q \rightarrow \varepsilon$	q: end
$qt \rightarrow q'$	q: if symbol = t then next_symbol else error; q'
	$q : X; q' : \dots$
	
$qt_1 \rightarrow q'q_1t_1$	proc X:
....	begin
$qt_m \rightarrow q'q_mt_m$	case symbol of
	$t_1 : \text{begin } q_1 : \dots \text{ end};$
	
	$t_m : \text{begin } q_m : \dots \text{ end};$
unde	otherwise error
$q = [Y \rightarrow \mu.X\nu;]$	end
	end

Reguli de transformare

1. nonterminal X - procedura X ; simbolul de start - programul principal
2. corpul functiei X :
 - ▶ ramificare case pt productiile cu X in partea stanga
 - ▶ fiecare nonterminal din partea dreapta a productiei - apel al procedurii corespunzatoare
 - ▶ fiecare terminal din partea dreapta a productiei - verificare a prezentei terminalului, urmat de apel al *next_symbol*
3. daca niciunul dintre terminalele asteptate nu e prezent - apel functia de tratare a erorilor

- ▶ Pt tranzitii $qt_1 \rightarrow q'q_1t1...$
- ▶ schema program indica:
 $q : F(); q'$
 procedura $F()$ - case pt toate t_i
- ▶ $q_2i \rightarrow q_3q_4i, q_2(\rightarrow q_3q_5($
 $q_4i \rightarrow q_9, q_9 \rightarrow \varepsilon,$
 $q_5(\rightarrow q_{10}, q_{10}i \rightarrow q_{12}q_2i, q_{10}(\rightarrow q_{12}q_2(,$
- ▶ $q_2 = [E \rightarrow .FE_1], q_3 = [E \rightarrow F.E_1], q_{10} = [F \rightarrow (.E)]$

q2: F(); q3

```

procedure F()
{ case symbol of
  'i' : { q4:  if (symbol == 'i') then next_symbol else
           error();
           q9:  ;}
  '(' : { q5:  if (symbol == '(') then next_symbol else
           error();
           q10: E();
           q12: if (symbol == ')') then next_symbol else
                  error();
           q14: ;}
  otherwise error(); }

```

```

derivator()
{ q0: E()
  q1: if (symbol != '#')
      error();
}
procedure E1()
{ case symbol of
  '#', ')': q7: ;
  '+': {
    q8: if (symbol == '+') next_symbol(); else error
        ();
    q11: F();
    q13: E1;
    q15: ;
  }
  otherwise : error();
}
procedure F()
{ case symbol of
  'i': { q4: if (symbol == 'i') then next_symbol else
        error();
        q9: ;}
  '(': { q5: if (symbol == '(') then next_symbol else
        error();
        q10: E();
        q12: if (symbol == ')') then next_symbol else
              error();
        q14: ;}
  otherwise error(); }

```

Parsing table - tabel de derivare

- ▶ Ullman 4.4 . Nonrecursive predictive parsing
- ▶ Table-driven predictive parsing: input, stiva, parsing table.
- ▶ Tabel de derivare: $M[A,a]$ - A nonterminal, a - terminal sau #

Exemplu de tabel de derivare

	lookahead					
	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

$$\begin{aligned}
 P = \{ & E \rightarrow TE' \\
 & E' \rightarrow +TE' | \varepsilon \\
 & T \rightarrow FT' \\
 & T' \rightarrow *FT' | \varepsilon \\
 & F \rightarrow (E) | id \}
 \end{aligned}$$

Algoritm de derivare predictiva cu tabel de derivare

```
#S (simbol de start) pe stiva, string# la intrare
set ip to point to the first symbol of input string
repeat
  let X be the top stack symbol and a the symbol pointed to
  by ip
  if X is a terminal or # then
    if X = a then
      pop X from the stack and advance ip
    else error()
  else
    if M[X,a] = X-> Y1 Y2 ...Yk then begin
      pop X fro the stack
      push Yk, Yk-1, ...Y1 onto the stack, with Y1 on top
      output the production X-> Y1 Y2 ...Yk
    end
    else error()
until X=#
```

Algoritm de derivare predictiva cu tabel de derivare

```
#S (simbol de start) pe stiva, string# la intrare
set ip to point to the first symbol of input string
repeat
  let X be the top stack symbol and a the symbol pointed to
  by ip
  if X is a terminal or # then
    if X = a then
      pop X from the stack and advance ip
    else error()
  else
    if M[X,a] = X-> Y1 Y2 ...Yk then begin
      pop X fro the stack
      push Yk, Yk-1, ...Y1 onto the stack, with Y1 on top
      output the production X-> Y1 Y2 ...Yk
    end
    else error()
until X=#
```

$$\{tqt \rightarrow q | t \in T\} \cup$$

$$\{Xq \rightarrow x_n \dots x_1 q | X \rightarrow x_1 x_2 \dots x_n \in P, n \geq 0, X \in N, X_i \in V\}$$

Exemplu de tabel de derivare

	lookahead					
	id	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

$$\begin{aligned}
 P = \{ & E \rightarrow TE' \\
 & E' \rightarrow +TE' | \varepsilon \\
 & T \rightarrow FT' \\
 & T' \rightarrow *FT' | \varepsilon \\
 & F \rightarrow (E) | id \}
 \end{aligned}$$

simbol	$FIRST_1(X)$	$FOLLOW_1(X)$
E	$\{ (, id \}$	$\{), \# \}$
E'	$\{ +, \varepsilon \}$	$\{), \# \}$
T	$\{ (, id \}$	$\{ +, \#,) \}$
T'	$\{ *, \varepsilon \}$	$\{ +, \#,) \}$
F	$\{ (, id \}$	$\{ *, +, \#,) \}$

- for each production $A \rightarrow \alpha$ do steps 2 and 3
- for each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
- if $\varepsilon \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal $b \in FOLLOW(A)$. if $\varepsilon \in FIRST(\alpha)$ and $\# \in FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \#]$
- Make each undefined entry of M be error

1. for each production $A \rightarrow \alpha$ do steps 2 and 3
2. for each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
3. if $\varepsilon \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal $b \in FOLLOW(A)$. if $\varepsilon \in FIRST(\alpha)$ and $\# \in FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \#]$
4. Make each undefined entry of M be error

test it online

First, follow sets, predict set

Algoritm $First_1$ - gramatici fara recursivitate stanga

Se aplica urmatoarele reguli pana cand nu se mai poate reduce nimic

- ▶ $FIRST(a) = \{a\}$
- ▶ $X \rightarrow \varepsilon: FIRST(X) = \varepsilon$
- ▶ $X \rightarrow Y_1 Y_2 Y_3$:
 - ▶ daca $\varepsilon \notin FIRST(Y_1)$ then $FIRST(X) = FIRST(Y_1)$
 - ▶ daca $\varepsilon \in FIRST(Y_1)$ then
 $FIRST(X) = (FIRST(Y_1) - \{\varepsilon\}) \cup FIRST(Y_2 Y_3)$

Algorithm *Follow*₁

- ▶ pentru simbolul de start: se adauga $\{\#\}$ in $\text{follow}(Z)$
- ▶ $X \rightarrow \alpha Y$: $\text{FOLLOW}(Y) = \text{FOLLOW}(X)$
- ▶ $X \rightarrow \alpha Y \beta$:
 - ▶ daca $\varepsilon \notin \text{FIRST}(\beta)$ then $\text{FOLLOW}(Y) = \text{FIRST}(\beta)$
 - ▶ daca $\varepsilon \in \text{FIRST}(\beta)$ then
$$\text{FOLLOW}(Y) = (\text{FIRST}(\beta) - \{\varepsilon\}) \cup \text{FOLLOW}(X)$$

vezi exemple

Parsare ascendenta (Bottom-up) 1.

Parsare ascendenta

- ▶ necesita luarea unei decizii **dupa** analiza sirului derivat dintr-o productie
- ▶ mai multa informatie pt decizie →
 - ▶ clasa larga de gramatici
 - ▶ pret: cresterea complexitatii procedurii de analiza si a automatului rezultat

Automat stiva - analiza ascendenta

Fie $G = (T, N, P, Z)$ o CFG si automatul stiva

$A = (T, \{q\}, R, q, \{q\}, V, \epsilon)$ cu $V = T \cup N$ si R :

(alfabet, stari, productii, stare initiala, stari finale, alfabet stiva, continut initial stiva)

$$\{x_1x_2...x_nq \rightarrow Xq \mid X \rightarrow x_1x_2...x_n \in P, n \geq 0, X \in N, X_i \in V\} \cup$$

$$\{qt \rightarrow tq \mid t \in T\} \cup$$

$$\{Zq \rightarrow q\}$$

Automatul accepta un sir din $L(G)$ lucrând inapoi printr-o derivare cea mai din dreapta a sirului.

Comparatie automat stiva in analiza descendenta vs ascendenta pt $G = (T, N, P, Z)$ o CFG

- ▶ **descendenta** - cursuri anterioare

$A = (T, \{q\}, R, q, \{q\}, V, Z)$ cu $V = T \cup N$ si R :

$$\{tqt \rightarrow q \mid t \in T\} \cup$$

$$\{Xq \rightarrow x_n \dots x_1 q \mid X \rightarrow x_1 x_2 \dots x_n \in P, n \geq 0, X \in N, X_i \in V\}$$

- ▶ **ascendenta**

$A = (T, \{q\}, R, q, \{q\}, V, \epsilon)$ cu $V = T \cup N$ si R :

$$\{x_1 x_2 \dots x_n q \rightarrow Xq \mid X \rightarrow x_1 x_2 \dots x_n \in P, n \geq 0, X \in N, X_i \in V\} \cup$$

$$\{qt \rightarrow tq \mid t \in T\} \cup$$

$$\{Zq \rightarrow q\}$$

Exemplu

Fie $G_1 = (T, N, E, P)$

- ▶ $T = \{+, *, (,), i\}$, $N = \{E, T, F\}$
- ▶ cu productiile P
 - ▶ $(1, 2) E \rightarrow T | E + T$
 - ▶ $(3, 4) T \rightarrow F | T * F$
 - ▶ $(5, 6) F \rightarrow i | (E)$

Automatul stiva:

- ▶ $T = \{+, *, (,), i\}$, $Q = \{q\}$,
 $q_0 = q$, $F = \{q\}$, $S = \{+, -, *, (,), i, E, T, F\}$, $s_0 = E$
- ▶ cu productiile R
 1. $Tq \rightarrow Eq$, $E + Tq \rightarrow Eq$,
 2. $Fq \rightarrow Tq$, $T * Fq \rightarrow Tq$,
 3. $iq \rightarrow Fq$, $(E)q \rightarrow Fq$,
 4. $q+ \rightarrow +q$, $q* \rightarrow *q$, $q(\rightarrow (q, q) \rightarrow q)$, $qi \rightarrow iq\}$
 5. $Eq \rightarrow q\}$

Derivarea gasita: $i+i*i$

stiva	stare	intrare	derivarea cea mai din dreapta
	q	$i + i * i$	$i+i*i$
i	q	$+i * i$	
F	q	$+i * i$	$F+i*i$
T	q	$+i * i$	$T+i*i$
E	q	$+i * i$	$E+i*i$
E+	q	$i * i$	
E+i	q	$*i$	
E+F	q	$*i$	$E+F*i$
E+T	q	$*i$	$E+T*i$
E+T*	q	i	
E+T*i	q	i	
E+T*F	q		
E+T	q		$E+T*F$
E	q		$E+T$
	q		E

Observatii

- ▶ coloana din dreapta este inversul derivarii celei mai din dreapta
- ▶ ascendenta - traseaza derivarea de jos la simbolul de start
- ▶ stiva contine la fiecare pas un **sir** din care **se poate deriva** portiunea de **sir deja citita**
- ▶ informatia semnificativa: perechea (ρ, σ) , unde
 - ▶ $\rho \in V^*$ - continutul stivei,
 - ▶ $\sigma \in T^*$ - restul sirului de la intrare

LL	LR
Does a leftmost derivation.	Does a rightmost derivation in reverse.
Starts with the root nonterminal on the stack.	The last nonterminal on the stack is the root nonterminal.
Ends when the stack is empty.	Starts with an empty stack.
Uses the stack for designating what is still to be expected.	Uses the stack for designating what is already seen.
Builds the parse tree top-down.	Builds the parse tree bottom-up.
Continuously pops a nonterminal off the stack, and pushes the corresponding right hand side.	Tries to recognize a right hand side on the stack, pops it, and pushes the corresponding nonterminal.
Expands the non-terminals.	Reduces the non-terminals.
Reads the terminals when it pops one off the stack.	Reads the terminals while it pushes them on the stack.
Pre-order traversal of the parse tree.	Post-order traversal of the parse tree.

Clase de echivalență pentru perechile (ρ, σ)

Pentru $p \in 1..n$, fie $X_p \rightarrow \chi_p$ producția a p -a a gramaticii independente de context $G = (T, N, P, Z)$. Clasele de reducere $R_j, j \in 0, ..n$ sunt definite de

$$R_0 = \{(\rho, \sigma) | \rho = \mu\gamma, \sigma = \nu\omega \text{ a.i. } Z \Rightarrow^R \mu Y \omega, Y \Rightarrow^{R'} \gamma \nu, \nu \neq \varepsilon\}$$

$$R_p = \{(\rho, \sigma) | \rho = \mu\chi_p, Z \Rightarrow^R \mu X_p \sigma, X_p \Rightarrow \chi_p\}$$

unde $Y \Rightarrow^{R'} \alpha$ este $Y \Rightarrow^R \alpha$ si ultimul pas din derivare nu ia forma $Y_1 \alpha \Rightarrow \alpha$

Clase de reducere - continuare

- ▶ clasele de reducere - perechile de siruri care ar putea sa apara in timpul analizei ascendente a unei propozitii din $L(G)$ de catre automatul stiva
- ▶ clasa de reducere careia ii apartine o pereche caracterizeaza tranzitia efectuata de catre automat cand acea pereche apare ca o configuratie
 1. $(\rho, \sigma) \in R_0$ - fraza simpla χ nu e complet in stiva; se aplica $qt \rightarrow tq$ cu $t = 1 : \sigma$ **tranzitie de deplasare**
 2. $(\rho, \sigma) \in R_p, p \in 1..n$ - fraza simpla χ_p e complet in stiva; se aplica $\chi_p q \rightarrow X_p q$ **tranzitie de reducere**
Obs: pt $p = 1$ tranzitia $Zq \rightarrow q$ si automatul se opreste
 3. $(\rho, \sigma) \notin R_j, j \in 0..n$. nu mai sunt posibile alte tranzitii; sirul de intrare nu apartine $L(G)$

Clase stiva k

- ▶ Pentru un $k \geq 0$, multimile $R_{j,k}$, $k \in 0..n$ se numesc clase stiva k al gramaticii G daca

$$R_{j,k} = \{(\rho, \tau) | \exists(\rho, \sigma) \in R_j, \tau = k : \sigma\}$$

- ▶ Daca clasele stiva k sunt mutual disjuncte atunci automatul stiva este determinist chiar si cand examinarea inainte este limitata la k simboluri

Gramatica LR(k)

O gramatica independenta de context $G = (T, N, P, Z)$ este $LR(k)$ pentru un $k \geq 0$ dat daca pentru derivari arbitrare

$$Z \Rightarrow^R \mu X \omega \Rightarrow \mu \chi \omega \quad \mu \in V^*, \omega \in T^*, X \rightarrow \chi \in P$$

$$Z \Rightarrow^R \mu' Y \omega' \Rightarrow \mu' \gamma \omega' \quad \mu' \in V^*, \omega' \in T^*, Y \rightarrow \gamma \in P$$

$(|\mu\chi| + k) : \mu\chi\omega = (|\mu'\chi| + k) : \mu'\gamma\omega'$ implica
 $\mu = \mu', X = Y, \chi = \gamma$

LR(k)

Automatul

- ▶ baleiaza sirul de intrare de la stanga la dreapta (Left to right)
- ▶ traversand inversa celei mai din dreapta derivari (Right)
- ▶ fara sa examineze mai mult de k simboluri de intrare intr-un pas

Teorema

O gramatica independenta de context este LR(k) daca si numai
daca clasele sale stiva k sunt mutual disjuncte.

Verificarea proprietatii LR(k) prin intersectarea claselor stiva

- ▶ clasele stiva k contin o infinitate de perechi (ρ, τ) , $\# \tau$ fiind finit datorita limitei de lungime, insa lungimea stivei nefiind limitata, $\# \rho$ este infinitate
- ▶ pentru fiecare clasa stiva k $R_{j,k}$ se poate preciza o gramatica regulata G_j a.i.

$$L(G_j) = \{(\rho \& \tau) | (\rho, \tau) \in R_{j,k}\}$$

- ▶ exista algoritmi pt a determina daca doua limbaje regulate sunt distincte

Situatii si inchidere nonterminal

Gramaticile regulate care genereaza clase stiva k:
Simbolurile nonterminale:

$$W = \{[X \rightarrow \mu.\nu; \omega] | X \rightarrow \mu\nu \in P, \omega \in FOLLOW_k(X)\}$$

Gramatici care genereaza clasele stiva k, fara a fi regulate

$$G'_j = (V \cup \{\&, \#\}, W, P' \cup P'' \cup P_j, [Z \rightarrow .S; \#])$$

$$P' = \{[X \rightarrow \mu.\nu\gamma; \omega] \rightarrow \nu[X \rightarrow \mu\nu.\gamma; \omega] \quad | \nu \in V\}$$

$$P'' = \{[X \rightarrow \mu.Y\gamma; \omega] \rightarrow [Y \rightarrow .\beta; \tau] \quad | Y \rightarrow \beta \in P, \tau \in EFF_k(\gamma\omega)\}$$

$$P_0 = \{[X \rightarrow \mu.\nu; \omega] \rightarrow \&\tau \quad | \nu \neq \varepsilon, \tau \in EFF_k(\nu\omega)\}$$

$$P_p = \{[X_p \rightarrow \chi_{p.}; \omega] \rightarrow \&\omega \quad p \in 1..n\}$$

Care productii sunt permise in gramatica regulata?

Lungimile $\&\tau$, $\&\omega$ sunt finite datorita lui k;

Gramatici care genereaza clasele stiva k, fara a fi regulate

$$G'_j = (V \cup \{\&, \#\}, W, P' \cup P'' \cup P_j, [Z \rightarrow .S; \#])$$

$$P' = \{[X \rightarrow \mu.\nu\gamma; \omega] \rightarrow \nu[X \rightarrow \mu\nu.\gamma; \omega] \quad | \nu \in V\}$$

$$P'' = \{[X \rightarrow \mu.Y\gamma; \omega] \rightarrow [Y \rightarrow .\beta; \tau] \quad | Y \rightarrow \beta \in P, \tau \in EFF_k(\gamma\omega)\}$$

$$P_0 = \{[X \rightarrow \mu.\nu; \omega] \rightarrow \&\tau \quad | \nu \neq \varepsilon, \tau \in EFF_k(\nu\omega)\}$$

$$P_p = \{[X_p \rightarrow \chi_{p.}; \omega] \rightarrow \&\omega \quad p \in 1..n\}$$

Care productii sunt permise in gramatica regulata?

P' si P_j unde $j \in 0..n$

Lungimile $\&\tau$, $\&\omega$ sunt finite datorita lui k; sunt considerate simboluri terminale

Inchiderea nonterminalului

o gramatica se poate rescrie a.i. sa nu contina productii precum cele din P''

- Inchiderea unui nonterminal

$$H(X) = \{X\} \cup \{Y \mid Y_l \rightarrow Y \in P, Y_l \in H(X)\}$$

Algoritm de rescriere a gramaticii

1. se selecteaza un $X \in N$ pentru care $H(X) \neq \{X\}$.
2. $P = P - \{X \rightarrow Y \mid Y \in N\}$
3. $P = P \cup \{X \rightarrow \beta \mid Y \rightarrow \beta \in P, Y \in H(X), \beta \notin N\}$

Alg se termina cand nu se mai poate face nicio selectie la pasul 1

din G'_j rezulta G_j . Sirurile β sunt toate de forma $\nu[..]$, $\&\tau$ sau $\&\omega$:
deci **gramatica regulata**

Teorema

Pentru orice gramatica G de tipul $LR(k)$ exista un automat stiva determinist A a.i. $L(A) = L(G)$.

Constructia automatului se bazeaza pe gramaticile G_j :

- ▶ automatul genereaza clasele stiva k
- ▶ si le verifica fata de inversa celei mai din dreapta derivari a sirului
- ▶ in functie de clasa stiva k particulara, automatul
 - ▶ **stivuieste** simbolul de intrare, sau
 - ▶ **reduce** un numar de simboluri stivuite la un nonterminal

..continuare construire automat LR

- ▶ alg de construire genereaza treptat situatiile necesare si utilizeaza operatia de inchidere pentru evitarea productiilor din P'' .
- ▶ o stare - o multime de situatii:
 - ▶ fiecare situatie dintr-o stare poate fi utilizata pentru derivarea clasei stiva k curente
- ▶ o alta formulare a inchiderii direct in functie de o multime de situatii M:

$$H(M) = M \cup \{[Y \rightarrow \cdot\beta; \tau] \mid$$

$$\exists [X \rightarrow \mu.Y\gamma; \omega] \in H(M),$$

$$Y \rightarrow \beta \in P,$$

$$\tau \in FIRST_k(\gamma\omega)\}$$

Algoritm LR(k)-determinare Q si R :

1. $Q = \{q_0\}$ si $R = \emptyset$ cu $q_0 = H([Z \rightarrow .S; \#])$
2. pt orice $q \in Q$ se efectueaza pasii 3-5 pt fiecare $\nu \in V$
3. fie $basis(q, \nu) = \{[X \rightarrow \mu\nu.\gamma; \omega] \mid [X \rightarrow \mu.\nu\gamma; \omega] \in q\}$
4. daca $basis(q, \nu) \neq \emptyset$ atunci $next(q, \nu) = H(basis(q, \nu))$. Se include $q' = next(q, \nu)$ in Q
5. daca $basis(q, \nu) \neq \emptyset$ si $\nu \in T$ se actualizeaza
 $R = R \cup$
 $\begin{cases} \{q\nu \rightarrow qq'\}, & k \leq 1 \\ \{q\nu\tau \rightarrow qq'\tau \mid [X \rightarrow \mu.\nu\gamma; \omega] \in q, \tau \in FIRST_{k-1}(\gamma\omega)\}, & k > 1 \end{cases}$
6. daca toate elementele lui Q au fost tratate se executa pasul 7 pt fiecare $q \in Q$ si alg se termina; altfel se continua pasul 2
7. pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$\begin{aligned} R = R \cup \{ & q_1..q_nq\omega \rightarrow q_1q'\omega \mid [X \rightarrow .\chi; \omega] \in q_1, \\ & q_{i+1} = next(q_i, x_i) (i \in 1..n-1), \\ & q = next(q_n, x_n), \\ & q' = next(q_1, X) \} \end{aligned}$$

Exemplu LR(k) cu $k=2$

$$T = \{a, b, c\}, N = \{Z.X, Y\}$$

$$P = \{(1)Z \rightarrow X,$$
$$(2, 3)X \rightarrow Y|bYa,$$
$$(4, 5)Y \rightarrow c|ca\}$$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$ R	q_1	q_2		q_3	q_4
				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$
q_1 $H([Z \rightarrow X.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$ R	q_1	q_2		q_3	q_4
				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$
q_1 $H([Z \rightarrow X.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_2 $H([X \rightarrow Y.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$next(q_0, \dots)$	q_1	q_2		q_3	q_4
R				$q_0bc \rightarrow q_0q_3c$ $FIRST_1(Ya\#)$	$q_0c\# \rightarrow q_0q_4\#$ $FIRST_1(\#)$ $q_0ca \rightarrow q_0q_4a$ $FIRST_1(a\#)$
q_1 $H([Z \rightarrow X.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_2 $H([X \rightarrow Y.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$q_3 : H([X \rightarrow$ $\quad b.Ya; \#])$ $[X \rightarrow b.Ya; \#]$ $[Y \rightarrow .c; a\#]$ $[Y \rightarrow .ca; a\#]$	$\emptyset$	$[X \rightarrow bY.a.; \#]$	$\emptyset$	$\emptyset$	$[Y \rightarrow c.; a\#]$ $[Y \rightarrow c.a; a\#]$
$next(q_3, \dots)$		q_5			q_6
R					$q_3ca \rightarrow q_3q_6a$

Stare	X	Y	a	b	c
q_4 $H([Y \rightarrow c.; \#],)$ $[Y \rightarrow c.a; \#]$	$\emptyset$	$\emptyset$	$[Y \rightarrow ca.; \#]$	$\emptyset$	$\emptyset$
$next(q_5...)$			q_7		
R			$q_4 a \# \rightarrow q_4 q_7 \#$		
q_5 $H([X \rightarrow bY.a; \#],)$ $next(q_5, ...)$	$\emptyset$	$\emptyset$	$[X \rightarrow bYa.; \#]$	$\emptyset$	$\emptyset$
			q_8		
R			$q_5 a \# \rightarrow q_5 q_8 \#$		
q_6 $H([Y \rightarrow c.; a\#],)$ $[Y \rightarrow c.a; a\#]$	$\emptyset$	$\emptyset$	$[Y \rightarrow ca.; a\#]$	$\emptyset$	$\emptyset$
$next(q_6, ...)$			q_9		
R			$q_6 a a \rightarrow q_6 q_9 a$		
q_7 $H([Y \rightarrow ca.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_8 $H([X \rightarrow bYa.; \#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_9 $H([Y \rightarrow ca.; a\#])$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

pas 7. Pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$R = R \cup \{q_1..q_n q \omega \rightarrow q_1 q' \omega \mid [X \rightarrow .\chi; \omega] \in q_1,$$

$$q_{i+1} = \text{next}(q_i, x_i) (i \in 1..n-1),$$

$$q = \text{next}(q_n, x_n),$$

$$q' = \text{next}(q_1, X)\}$$

Pt q_2 : $H([X \rightarrow Y.; \#])$

rol de q_1 $[X \rightarrow .Y; \#] \in ?$

rol de q $? = \text{next}(?, Y)$

rol de q' $? = \text{next}(?, X)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$\text{next}(q_0, \dots)$	q_1	q_2		q_3	q_4

pas 7. Pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$\begin{aligned}
 R &= R \cup \{q_1..q_n q \omega \rightarrow q_1 q' \omega \mid [X \rightarrow .\chi; \omega] \in q_1, \\
 &\quad q_{i+1} = \text{next}(q_i, x_i) (i \in 1..n-1), \\
 &\quad q = \text{next}(q_n, x_n), \\
 &\quad q' = \text{next}(q_1, X)\}
 \end{aligned}$$

Pt q_2 : $H([X \rightarrow Y.; \#])$

rol de q_1 $[X \rightarrow .Y; \#] \in q_0$

rol de q $q_2 = \text{next}(q_0, Y)$

rol de q' $q_1 = \text{next}(q_0, X)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$\text{next}(q_0, \dots)$	q_1	q_2		q_3	q_4

pas 7. Pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$\begin{aligned}
 R &= R \cup \{q_1..q_n q \omega \rightarrow q_1 q' \omega \mid [X \rightarrow .\chi; \omega] \in q_1, \\
 &\quad q_{i+1} = \text{next}(q_i, x_i) (i \in 1..n-1), \\
 &\quad q = \text{next}(q_n, x_n), \\
 &\quad q' = \text{next}(q_1, X)\}
 \end{aligned}$$

Pt q_2 : $H([X \rightarrow Y.; \#])$

rol de q_1 $[X \rightarrow .Y; \#] \in q_0$

rol de q $q_2 = \text{next}(q_0, Y)$

rol de q' $q_1 = \text{next}(q_0, X)$

$q_0 q_2 \# \rightarrow q_0 q_1 \#$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$\text{next}(q_0, \dots)$	q_1	q_2		q_3	q_4

pas 7. Pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$\begin{aligned}
 R &= R \cup \{q_1..q_n q \omega \rightarrow q_1 q' \omega \mid [X \rightarrow .\chi; \omega] \in q_1, \\
 &\quad q_{i+1} = \text{next}(q_i, x_i) (i \in 1..n-1), \\
 &\quad q = \text{next}(q_n, x_n), \\
 &\quad q' = \text{next}(q_1, X)\}
 \end{aligned}$$

Pt q_4 : $H([Y \rightarrow c.; \#], [Y \rightarrow c.a; \#])$

q_1 $[Y \rightarrow .c; \#] \in q_0$

q $q_4 = \text{next}(q_0, c)$

q' $q_2 = \text{next}(q_0, Y)$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$\text{next}(q_0, \dots)$	q_1	q_2		q_3	q_4

pas 7. Pentru fiecare $[X \rightarrow \chi.; \omega] \in q$, unde $\chi = x_1..x_n$ se face

$$\begin{aligned}
 R &= R \cup \{q_1..q_n q \omega \rightarrow q_1 q' \omega \mid [X \rightarrow .\chi; \omega] \in q_1, \\
 &\quad q_{i+1} = \text{next}(q_i, x_i) (i \in 1..n-1), \\
 &\quad q = \text{next}(q_n, x_n), \\
 &\quad q' = \text{next}(q_1, X)\}
 \end{aligned}$$

Pt q_4 : $H([Y \rightarrow c.; \#], [Y \rightarrow c.a; \#])$

q_1 $[Y \rightarrow .c; \#] \in q_0$

q $q_4 = \text{next}(q_0, c)$

q' $q_2 = \text{next}(q_0, Y)$

$q_0 q_4 \# \rightarrow q_0 q_2 \#$

Stare	X	Y	a	b	c
q_0 $H([Z \rightarrow .X; \#])$ $[Z \rightarrow .X; \#],$ $[X \rightarrow .Y; \#],$ $[X \rightarrow .bYa; \#],$ $[Y \rightarrow .c; \#],$ $[Y \rightarrow .ca; \#]$	$[Z \rightarrow X.; \#]$	$[X \rightarrow Y.; \#]$	-	$[X \rightarrow b.Ya; \#]$	$[Y \rightarrow c.; \#]$ $[Y \rightarrow c.a; \#]$
$\text{next}(q_0, \dots)$	q_1	q_2		q_3	q_4

Pt q_6 : $H([Y \rightarrow c.; a\#], [Y \rightarrow c.a; a\#])$

q_1 $[Y \rightarrow .c; a\#] \in q_3$

q $q_6 = next(q_3, c)$

q' $q_5 = next(q_3, Y)$

$q_3 q_6 a\# \rightarrow q_3 q_5 a\#$

Pt q_7 : $H([Y \rightarrow ca.; \#])$

q_1 $[Y \rightarrow .ca; \#] \in q_0$

q_2 $q_4 = next(q_0, c)$

q $q_7 = next(q_4, a)$

q' $q_2 = next(q_0, Y)$

$q_0 q_4 q_7 \# \rightarrow q_0 q_2 \#$

Pt q_8 : $H([X \rightarrow bYa.; \#]$

q_1 $[X \rightarrow .bYa; \#] \in q_0$

q_2 $q_3 = next(q_0, b)$

q_3 $q_5 = next(q_3, Y)$

q $q_8 = next(q_5, a)$

q' $q_1 = next(q_0, X)$

$q_0 q_3 q_5 q_8 \# \rightarrow q_0 q_1 \#$

Pt q_9 : $H([Y \rightarrow \text{ca.}; a\#])$

q_1 $[Y \rightarrow .ca; a\#] \in q_3$

q_2 $q_6 = next(q_3, c)$

q $q_9 = next(q_6, a)$

q' $q_5 = next(q_3, Y)$

$q_3 q_6 q_9 a\# \rightarrow q_3 q_5 a\#$

$q_0:$ $[Z \rightarrow \bullet X; \#]$
 $[X \rightarrow \bullet Y; \#]$
 $[X \rightarrow \bullet bY a; \#]$
 $[Y \rightarrow \bullet c; \#]$
 $[Y \rightarrow \bullet ca; \#]$
 $q_1:$ $[Z \rightarrow X \bullet; \#]$
 $q_2:$ $[X \rightarrow Y \bullet; \#]$
 $q_3:$ $[X \rightarrow b \bullet Y a; \#]$
 $[Y \rightarrow \bullet c; a \#]$
 $[Y \rightarrow \bullet ca; a \#]$

$q_4:$ $[Y \rightarrow c \bullet; \#]$
 $[Y \rightarrow c \bullet a; \#]$
 $q_5:$ $[X \rightarrow bY \bullet a; \#]$
 $q_6:$ $[Y \rightarrow c \bullet; a \#]$
 $[Y \rightarrow c \bullet a; a \#]$
 $q_7:$ $[Y \rightarrow ca \bullet; \#]$
 $q_8:$ $[X \rightarrow bY a \bullet; \#]$
 $q_9:$ $[Y \rightarrow ca \bullet; a \#]$

$$\begin{aligned}
R = \{ & q_0bc \rightarrow q_0q_3c, \\
& q_0c\# \rightarrow q_0q_4\#, \\
& q_0ca \rightarrow q_0q_4a, \\
& q_3ca \rightarrow q_3q_6a, \\
& q_4a\# \rightarrow q_4q_7\#, \\
& q_5a\# \rightarrow q_5q_8\#, \\
& q_6aa \rightarrow q_6q_9a, \\
& q_0q_2\# \rightarrow q_0q_1\#, \\
& q_0q_4\# \rightarrow q_0q_2\#, \\
& q_3q_6a\# \rightarrow q_3q_5a\#, \\
& q_0q_4q_7\# \rightarrow q_0q_2\#, \\
& q_0q_3q_5q_8\# \rightarrow q_0q_1\#, \\
& q_3q_6q_9a\# \rightarrow q_3q_5a\# \}
\end{aligned}$$

q_0 : $[Z \rightarrow \bullet X; \#]$
 $[X \rightarrow \bullet Y; \#]$
 $[X \rightarrow \bullet bY a; \#]$
 $[Y \rightarrow \bullet c; \#]$
 $[Y \rightarrow \bullet ca; \#]$
 q_1 : $[Z \rightarrow X \bullet; \#]$
 q_2 : $[X \rightarrow Y \bullet; \#]$
 q_3 : $[X \rightarrow b \bullet Y a; \#]$
 $[Y \rightarrow \bullet c; a \#]$
 $[Y \rightarrow \bullet ca; a \#]$

q_4 : $[Y \rightarrow c \bullet; \#]$
 $[Y \rightarrow c \bullet a; \#]$
 q_5 : $[X \rightarrow bY \bullet a; \#]$
 q_6 : $[Y \rightarrow c \bullet; a \#]$
 $[Y \rightarrow c \bullet a; a \#]$
 q_7 : $[Y \rightarrow ca \bullet; \#]$
 q_8 : $[X \rightarrow bY a \bullet; \#]$
 q_9 : $[Y \rightarrow ca \bullet; a \#]$

$R = \{q_0bc \rightarrow q_0q_3c,$
 $q_0c\# \rightarrow q_0q_4\#,$
 $q_0ca \rightarrow q_0q_4a,$
 $q_3ca \rightarrow q_3q_6a,$
 $q_4a\# \rightarrow q_4q_7\#,$
 $q_5a\# \rightarrow q_5q_8\#,$
 $q_6aa \rightarrow q_6q_9a,$
 $q_0q_2\# \rightarrow q_0q_1\#,$
 $q_0q_4\# \rightarrow q_0q_2\#,$
 $q_3q_6a\# \rightarrow q_3q_5a\#,$
 $q_0q_4q_7\# \rightarrow q_0q_2\#,$
 $q_0q_3q_5q_8\# \rightarrow q_0q_1\#,$
 $q_3q_6q_9a\# \rightarrow q_3q_5a\#\}$

Cu $k = 1$: aceleasi stari; $k = 0$ ar fuziona q_4, q_6 , respectiv q_7, q_9 .
 Dar: un singur simbol inainte nu face distinctie intre tranzitiile de
 deplasare (shift) si reducere (reduce) din starea 6.

$$Z \Rightarrow X \rightarrow bYa \Rightarrow bcaa$$

Stiva	Stare	intrare	$\Rightarrow^R$	tranzitie
#	q_0	bcaa#	bcaa	1
#	$q_0 q_3$	caa#		3
#	$q_0 q_3 q_6$	aa#		6
#	$q_0 q_3 q_6 q_9$	a#	bYa	12
#	$q_0 q_3 q_5$	a#		5
#	$q_0 q_3 q_5 q_8$	#	X	11
#	$q_0 q_1$	#	Z	

Derivatoare LR

Algoritmul LR(k) - poate fi folosit atat pt a verifica daca o gramatica este LR(1) cat si pt construirea derivatorului sau

Algoritmul LR(k): numarul de stari este foarte mare

- ▶ similar cazului strong LL(k), multe tranzitii din LR(1) sunt independente de simbolul de lookahead
- ▶ $\Rightarrow$ putem construi un parser cu mai putine stari care implementeaza analiza LR(1) dar cu tranzitii mai putine, folosind lookahead doar cand este necesar
- ▶ LR(k)
- ▶ *simple* LR(k) : SLR(k)
- ▶ *lookahead* LR(k): LALR(k)

- ▶ se porneste cu $LR(0)$: nu examineaza deloc simbolurile dinainte
- ▶ si se foloseste lookahead doar la nevoie - simple $LR(1)$ ($SLR(1)$)
- ▶ Obs: nu toate $LR(1)$ sunt $SLR(1)$
- ▶ $LALR(1)$ - lookahead aplicat la $SLR(1)$

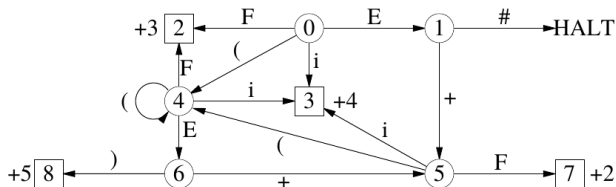
Tabel de tranzitii

(1) $Z \rightarrow E$ (2) $E \rightarrow E + F$ (3) $E \rightarrow F$ (4) $F \rightarrow i$ (5) $F \rightarrow (E)$

	i	$($	$)$	$+$	$\#$	E	F
0	3	4	.	.	.	1	2
1	.	.	.	5	*		
2	+3	+3	+3	+3	+3		
3	+4	+4	+4	+4	+4		
4	3	4	.	.	.	6	2
5	3	4	.	.	.		7
6	.	.	8	5	.		
7	+2	+2	+2	+2	+2		
8	+5	+5	+5	+5	+5		

Diagrama de tranzitii

- ▶ suprapunerea gramaticilor regulate care corespund claselor stiva k
- ▶ starile prin care trece sunt inarcate pe stiva pana cand se ajunge intr-o stare finala
- ▶ in stare finala se face reducerea pe baza productiei $X \rightarrow \chi$, se elimina $|\chi|$ stari de pe stiva si se continua **ca si cand s-ar fi citit simbolul X**



(1) $Z \rightarrow E$ (2) $E \rightarrow E + F$ (3) $E \rightarrow F$ (4) $F \rightarrow i$ (5) $F \rightarrow (E)$

Stiva	Derivare dreapta	simbol urmator	reducere cu productia	stare urmatoare
0	.i+(i+i)#	i		3
0 3	i.+(i+i)#		4	2
0 2	F.+(i+i)#		3	1
0 1	E.+(i+i)#	+		5
0 1 5	E+. (i+i)#	(		4
0 1 5 4	E+(.i+i)#	i		3
0 1 5 4 3	E+(i.+i)#		4	2
0 1 5 4 2	E+(F.+i)#		3	6
0 1 5 4 6	E+(E.+i)#	+		5
0 1 5 4 6 5	E+(E+.i)#	i		3
0 1 5 4 6 5 3	E+(E+i.)#		4	7
0 1 5 4 6 5 7	E+(E+F.)#		2	6
0 1 5 4 6	E+(E.)#	)		8
0 1 5 4 6 8	E+(E).#		5	7
0 1 5 7	E+F.#		2	1
0 1	E.#			

- ▶ Instrumente pentru reprezentare
 - ▶ Siruri de rescriere
 - ▶ Gramatici - ierarhia lui Chomsky
 - ▶ Derivari si arbori de derivare
- ▶ Gramatici regulate si automate finite
 - ▶ Automate finite
 - ▶ Diagrame de stare si expresii regulate
- ▶ Gramatici independente de context si automate stiva:
Automate stiva
- ▶ Analiza sintatica descendenta:
 - ▶ $LL(k)$
 - ▶ eliminare recursivitate stanga
 - ▶ Factorizare stanga
 - ▶ gramatici $LL(k)$ tari
 - ▶ **Derivator $LL(1)$** - segmente de program
- ▶ Analiza sintatica ascendenta
 - ▶ $LR(k)$
 - ▶ **Derivator $LR(0)$** - functia de tranzitie
 - ▶ $SLR(1)$